



Learning & Growing With Team

Help & Testing Services Any Field

CS304-Object Oriented Programming

PPT Slide Merged By LGWT

You Can Discuss Your Problems in these groups:

Whatsapp Study Groups: [Handouts & Files](#) [Study Group](#) [Community Link](#)

If Any Link Have Been Not Working then you can direct contact me on whatsapp

<https://wa.me/923196248868?text=Asslamualikum%21%20Can%20You%20Send%20me%20Group%20Link>

Paid Services Available:

- + Assignments
- + Quiz
- + GDB
- + Mid & Final Exam Preparation

[CLICK ME FOR PAID SERVICES](#)

For More Notes & Files Join Our Whatsapp Group <https://chat.whatsapp.com/CpYyqhErHGX3E8PJ9Aj2XR>

Object-Oriented Programming (OOP)

Lecture No. 19

Stream Insertion operator

► Often we need to display the data on the screen

► Example:

```
int i=1, j=2;
```

```
cout << "i= " << i << "\n";
```

```
Cout << "j= " << j << "\n";
```

Stream Insertion operator

```
Complex c1;  
  
cout << c1;  
  
cout << c1 << 2;
```

*// Compiler error: binary '<<': no
operator // defined which takes a right-
hand // operand of type 'class
Complex'*

Stream Insertion operator

```
class Complex{  
    ...  
public:  
    ...  
    void operator << (const  
                        Complex & rhs);  
};
```

Stream Insertion operator

```
int main() {  
    Complex c1;  
    cout << c1;           // Error  
    c1 << cout;  
    c1 << cout << 2; // Error  
    return 0;  
};
```

Stream Insertion operator

```
class Complex{  
    ...  
public:  
    ...  
    void operator << (ostream &) ;  
};
```

Stream Insertion operator

```
void Complex::operator <<
    (ostream & os){
    os << '(' << real
        << ',' << img << ')';
}
```

Stream Insertion operator

```
class Complex{
```

```
...
```

Note: return type
is NOT const

```
friend ostream & operator <<  
(ostream & os, const Complex  
                        & c);
```

```
};
```

Note: this object
is NOT const

Stream Insertion operator

// we want the output as: *(real, img)*

```
ostream & operator << (ostream &  
    os, const Complex & c) {  
    os << '(' << c.real  
        << ','  
        << c.img << ')';  
    return os;  
}
```

Stream Insertion operator

```
Complex c1(1.01, 20.1),  
        c2(0.01, 12.0);
```

```
cout << c1 << endl << c2;
```

Stream Insertion operator

Output:

(1.01 , 20.1)

(0.01 , 12.0)

Stream Insertion operator

```
cout << c1 << c2;
```

is equivalent to

```
operator<<(
    operator<<(cout,c1),c2);
```

Stream Extraction Operator

► Overloading “>>” operator:

```
class Complex{
```

```
...
```

```
friend istream & operator  
>> (istream & i, Complex &  
    c);
```

```
} ;
```

Note: this object
is NOT const

Stream Extraction Operator

```
istream & operator << (istream  
                        & in, Complex & c){  
    in >> c.real;  
    in >> c.img;  
    return in;  
}
```

Stream Extraction Operator

► Main Program:

```
Complex c1(1.01, 20.1);  
  
cin >> c1;  
  
// suppose we entered  
// 1.0025 for c1.real and  
// 0.0241 for c1.img  
  
cout << c1;
```

Stream Extraction Operator

Output:

(1.0025 , 0.0241)

Other Binary operators

► Overloading comparison operators:

```
class Complex{
public:
    bool operator == (const Complex & c);
    //friend bool operator == (const
    //Complex & c1, const Complex & c2);
    bool operator != (const Complex & c);
    //friend bool operator != (const
    //Complex & c1, const Complex & c2);
    ...
};
```

Other Binary operators

```
bool Complex::operator ==(const
Complex & c){
    if((real == c.real) &&
        (img == c.img)){
        return true;
    }
    else
        return false;
}
```

Other Binary operators

```
bool operator ==(const  
Complex& lhs, const Complex& rhs) {  
    if((lhs.real == rhs.real) &&  
        (lhs.img == rhs.img)) {  
        return true;  
    }  
    else  
        return false;  
}
```


Other Binary operators

```
bool Complex::operator !=(const
Complex & c){
    if((real != c.real) ||
        (img != c.img)){
        return true;
    }
    else
        return false;
}
```

Object-Oriented Programming (OOP)

Lecture No. 20

Other Binary operators

- We have seen the following string class till now:

```
class String{  
private:  
    char * bufferPtr; int size;  
public:  
    String();  
    String(char * ptr);  
    void SetString(char * ptr);  
    const char * GetString();  
    ...  
};
```

Other Binary Operators

```
int main() {  
    String str1("Test");  
    String str2;  
    str2.SetString("Ping");  
    return 0;  
}
```

Other Binary Operators

► What if we want to change the string from “*Ping*” to “*Pong*”?? {*ONLY 1 character to be changed...*}

► Possible solution:

- Call: `str2.SetString("Pong") ;`
- This will delete the current buffer and allocate a new one
- Too much overhead if string is too big

Other Binary Operators

- Or, we can add a function which changes a character at nth location

```
class String{  
    ...  
public:  
    void SetChar(char c, int pos);  
    ...  
};
```

Other Binary Operators

```
void SetChar(char c, int pos){  
    if(bufferPtr != NULL){  
        if(pos>0 && pos<=size)  
            bufferPtr[pos] = c;  
    }  
}
```

Other Binary Operators

- Now we can efficiently change a single character:

```
String str1("Ping");
```

```
str1.SetChar('o', 2);
```

```
// str1 is now changed to "Pong"
```


Subscript Operator

- ▶ An elegant solution:
- ▶ Overloading the subscript “[]” operator

Subscript Operator

```
int main() {  
    String str2;  
    str2.SetString("Ping");  
    str[2] = 'o';  
    cout << str[2];  
    return 0;  
}
```

Subscript Operator

```
class String{  
    ...  
public:  
    char & operator[] (int) ;  
    ...  
};
```

Subscript Operator

```
char & String::operator[](  
                                int pos){  
    assert(pos>0 && pos<=size);  
    return stringPtr[pos-1];  
}
```

Subscript Operator

```
int main() {  
    String s1("Ping");  
    cout <<str.GetString()<< endl;  
    s1[2] = 'o';  
    cout << str.GetString();  
    return 0;  
}
```

Subscript Operator

► Output:

Ping

Pong

Overloading ()

- ▶ Must be a member function
- ▶ Any number of parameters can be specified
- ▶ Any return type can be specified
- ▶ `Operator ( )` can perform any generic operation

Function Operator

```
class String{  
    ...  
public:  
    char & operator() (int) ;  
    ...  
};
```


Function Operator

```
char & String::operator()  
    (int pos) {  
    assert(pos>0 && pos<=size);  
    return bufferPtr[pos-1];  
}
```

Subscript Operator

```
int main(){  
    String s1("Ping");  
    char g = s1(2);    // g = 'i'  
    s1(2) = 'o';  
    cout << g << "\n";  
    cout << str.GetString();  
    return 0;  
}
```

Function Operator

► Output:

`i`

`Pong`

Function Operator

```
class String{
```

```
    ...
```

```
public:
```

```
    String operator()(int, int);
```

```
    ...
```

```
};
```

Function Operator

```
String String::operator()(int index,  
                           int subLength){  
    assert(index>0 && index+subLength-1<=size);  
    char * ptr = new char[subLength+1];  
    for (int i=0; i < subLength; ++i)  
        ptr[i] = bufferPtr[i+index-1];  
    ptr[subLength] = '\\0';  
    String str(ptr);  
    delete [] ptr;  
    return str;  
}
```

Function Operator

```
int main(){  
    String s("Hello World");  
    // "<<" is overloaded  
    cout << s(1, 5);  
    return 0;  
}
```

Function Operator

Output:

Hello

Unary Operators

► Unary operators:

- `& * + - ++ -- ! ~`

► Examples:

`--X`

`-(x++)`

`!(*ptr ++)`

Unary Operators

► Unary operators are usually prefix, except for ++ and --

► ++ and -- both act as prefix and postfix

► Example:

```
h++;
```

```
g-- + ++h - --i;
```

Unary Operators

► General syntax for unary operators:

Member Functions:

TYPE & operator OP ();

Non-member Functions:

**Friend TYPE & operator OP
(TYPE & t);**

Unary Operators

- Overloading unary '-':

```
class Complex{
```

```
...
```

```
Complex operator - ();
```

```
// friend Complex operator
```

```
//          -(Complex &);
```

```
}
```

Unary Operators

- Member function definition:

```
Complex Complex::operator -(){  
    Complex temp;  
    temp.real = -real;  
    temp.img = -img;  
    return temp;  
}
```

Unary Operators

Complex c1(1.0 , 2.0), c2;

c2 = -c1;

// c2.real = -1.0

// c2.img = -2.0

► Unary '+' is overloaded in the same way

Object-Oriented Programming (OOP)

Lecture No. 21

Unary Operators

- ▶ Unary operators are usually prefix, except for `++` and `--`
- ▶ `++` and `--` both act as prefix and postfix
- ▶ Example:
 - `h++;`
 - `g-- + ++h - --i;`

Unary Operators

► Behavior of ++ and -- for pre-defined types:

- Post-increment ++ :

- Post-increment operator ++ increments the current value and then returns the previous value

- Post-decrement -- :

- Works exactly like post ++

Unary Operators

► Example:

```
int x = 1, y = 2;  
cout << y++ << endl;  
cout << y;
```

► Output:

2

3

Unary Operators

► Example:

```
int y = 2;
```

```
y++++;           // Error
```

```
y++ = x;         // Error
```

Post-increment ++
returns by value,
hence an error while
assignment

Unary Operators

► Behavior of ++ and -- for pre-defined types:

- Pre-increment ++ :

- Pre-increment operator ++ increments the current value and then returns it's reference

- Pre-decrement -- :

- Works exactly like Pre-increment ++

Unary Operators

► Example:

```
int y = 2;  
cout << ++y << endl;  
cout << y << endl;
```

► Output:

3
3

Unary Operators

► Example:

```
int x = 2, y = 2;  
++++y;  
cout << y;  
++y = x;  
cout << y;
```

Pre-increment ++
returns by
reference, hence
NOT an error

► Output:

4
2

Unary Operators

► Example (Pre-increment):

```
class Complex{  
    double real, img;  
public:  
    ...  
    Complex & operator ++ ();  
    // friend Complex & operator  
    //      ++(Complex &);  
}
```

Unary Operators

- Member function definition:

```
Complex & Complex::operator++() {  
    real = real + 1;  
    return * this;  
}
```

Unary Operators

► Friend function definition:

```
Complex & operator ++ (Complex  
                        & h) {  
    h.real += 1;  
    return h;  
}
```


Unary Operators

```
Complex h1, h2, h3;
```

```
++h1;
```

► Function operator++ () returns a reference so that the object can be used as an *lvalue*

```
++h1 = h2 + ++h3;
```

Unary Operators

- How does a compiler know whether it is a pre-increment or a post-increment ?

Unary Operators

► A post-fix unary operator is implemented using:

Member function with 1 dummy int argument

OR

Non-member function with two arguments

Unary Operators

- ▶ In post increment, current value of the object is stored in a temporary variable
- ▶ Current object is incremented
- ▶ Value of the temporary variable is returned

Unary Operators

- Post-increment operator:

```
class Complex{  
    ...  
    Complex operator ++ (int);  
    // friend Complex operator  
    // ++(const Complex &, int);  
}
```

Unary Operators

- Member function definition:

```
Complex Complex::operator ++  
                (int) {  
  
    complex t = *this;  
    real += 1;  
    return t;  
}
```

Unary Operators

► Friend function definition:

```
Complex operator ++ (const  
                    Complex & h, int) {  
    complex t = h;  
    h.real += 1;  
    return t;  
}
```

Unary Operators

► The dummy parameter in the operator function tells compiler that it is post-increment

► Example:

```
Complex h1, h2, h3;
```

```
h1++;
```

```
h3++ = h2 + h3++; // Error...
```


Unary Operators

- The *pre* and *post* decrement operator -- is implemented in exactly the same way

Type Conversion

► The compiler automatically performs a type coercion of compatible types

► e.g:

```
int f = 0.021;
```

```
double g = 34;
```

```
// type float is automatically converted  
// into int. Compiler only issues a  
// warning...
```

Type Conversion

- The user can also explicitly convert between types:

C style type casting

```
int g = (int)0.0210;
```

```
double h = double(35);
```

```
// type float is explicitly converted  
// (casted) into int. Not even a warning  
// is issued now...
```

Type Conversion

- ▶ For user defined classes, there are two types of conversions
 - From any other type to current type
 - From current type to any other type

Type Conversion

► Conversion from any other type to current type:

- Requires a constructor with a single parameter

► Conversion from current type to any other type:

- Requires an overloaded operator

Type Conversion

- Conversion from other type to current type (int to String):

```
class String{  
    ...  
public:  
    String(int a) ;  
    char * GetStringPtr() const;  
};
```

Type Conversion

```
String::String(int a) {  
    cout << "String(int) called..." << endl;  
    char array[15];  
    itoa(a, array, 10);  
    size = strlen(array);  
    bufferPtr = new char [size + 1];  
    strcpy(bufferPtr, array);  
}  
  
char * String::GetStringPtr() const{  
    return bufferPtr;  
}
```

Type Conversion

```
int main(){  
    String s = 345;  
    cout << s.GetStringPtr() << endl;  
    return 0;  
}
```


Type Conversion

► Output:

`String(int)` called...

345

Type Conversion

- ▶ Automatic conversion has drawbacks
- ▶ Conversion takes place transparently even if the user didn't wanted the conversion

Type Conversion

User can write the following code to initialize the string with a single character:

```
int main() {  
    String s = 'A';  
    cout<< s.GetStringPtr()<< endl  
        << s.GetSize()    << endl;  
    return 0;  
}
```

Type Conversion

► Output:

`String(int)` called...

65

ASCII code
for 'A' !!!

2

String size
is also 2
instead of 1

Type Conversion

There is a mechanism in C++ to restrict automatic conversions

- ▶ Keyword `explicit`
- ▶ Casting must be explicitly performed by the user

Type Conversion

► Keyword `explicit` only works with constructors

► Example:

```
class String{  
    ...  
public:  
    ...  
    explicit String(int) ;  
};
```

Type Conversion

```
int main() {  
    String s;  
    // Error...  
    s = 'A';  
    return 0;  
}
```

Type Conversion

```
int main() {  
    String s1, s2;  
    // valid, explicit casting..  
    s1 = String(101);  
    // OR  
    s2 = (String)204;  
    return 0;  
}
```


Type Conversion

► There is another method for type conversion:

“Operator overloading”

(Converting from current type to any other type)

Type Conversion

- ▶ General Syntax:

`TYPE1 :: Operator TYPE2 ();`

- ▶ Must be a member function
- ▶ NO return type and arguments are specified
- ▶ Return type is implicitly taken to be `TYPE2` by compiler

Type Conversion

- Overloading pre-defined types:

```
class String{  
    ...  
public:  
    ...  
    operator int();  
    operator char *();  
};
```

Type Conversion

```
String::operator int() {  
    if(size > 0)  
        return atoi(bufferPtr);  
    else  
        return -1;  
}
```

Type Conversion

```
String::operator char *() {  
    return bufferPtr;  
}
```

Type Conversion

```
int main() {  
    String s("2324");  
    cout << (int)s << endl  
         << (char *)s;  
    return 0;  
}
```

Type Conversion

Output:

2324

2324

Type Conversion

- ▶ User-defined types can be overloaded in exactly the same way
- ▶ Only prototype is shown below:

```
class String{  
    ...  
    operator Complex();  
    operator HugeInt();  
    operator IntVector();  
};
```


Type Conversion

- Modifying String class:

```
class String{  
    ...  
public:  
    ...  
    String(char *);  
    operator int();  
};
```

Type Conversion

```
int main() {  
    String s("Fakhir");  
    // << is NOT overloaded  
    cout << s;  
    return 0;  
}
```

Type Conversion

Output:

Junk Returned...

Type Conversion

- Modifying String class:

```
class String{  
    ...  
public:  
    ...  
    String(char *);  
    int AsInt();  
};
```

Type Conversion

```
int String::AsInt() {  
    if(size > 0)  
        return atoi(bufferPtr);  
    else  
        return -1;  
}
```

Type Conversion

```
int main() {  
    String s("434");  
    // << is NOT overloaded  
    cout << s; //error  
    cout << s.AsInt();  
    return 0;  
}
```

Object-Oriented Programming (OOP)

Lecture No. 23

Date Class

```
class Date{  
    int day, month, year;  
    static Date defaultDate;  
public:  
    void SetDay(int aDay);  
    int GetDay() const;  
    void AddDay(int x);  
    ...  
    static void SetDefaultDate(  
        int aDay,int aMonth, int aYear);
```

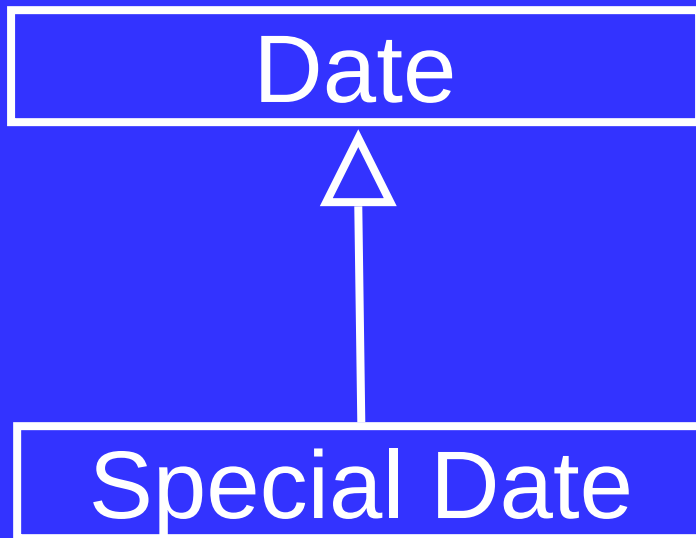


Date Class

```
...
private:
    bool IsLeapYear();
};

int main(){
    Date aDate;
    aDate.IsLeapYear(); //Error
    return 0;
}
```

Creating SpecialDate Class



Creating SpecialDate Class

```
class SpecialDate: public Date{  
    ...  
public:  
    void AddSpecialYear(int i){  
        ...  
        if(day == 29 && month == 2  
           && !IsLeapyear(year+i)){ //ERROR!  
            ...  
        }  
    }  
};
```

Modify Access Specifier

- We can modify access specifier “IsLeapYear” from private to public

Modified Date Class

```
class Date{  
public:  
    ...  
    bool IsLeapYear();  
};
```



Modified AddSpecialYear

```
void SpecialDate :: AddSpecialYear  
                        (int i){
```

```
...
```

```
if(day == 29 && month == 2  
    && !IsLeapyear(year+i)){
```

```
...
```

```
}
```

```
}
```

Protected members

- Protected members can not be accessed outside the class
- Protected members of base class become protected member of derived class in Public inheritance

Modified Date Class

```
class Date{  
    ...  
protected:  
    bool IsLeapYear();  
};  
  
int main(){  
    Date aDate;  
    aDate.IsLeapYear(); //Error  
    return 0;  
}
```



Modified AddSpecialYear

```
void SpecialDate :: AddSpecialYear  
                        (int i){
```

```
...
```

```
if(day == 29 && month == 2  
    && !IsLeapyear(year+i)){
```

```
...
```

```
}
```

```
}
```

Disadvantages

- Breaks encapsulation
 - The protected member is part of base class's implementation as well as derived class's implementation

“IS A” Relationship

- Public inheritance models the “IS A” relationship
- Derived object IS A kind of base object

Example

```
class Person {  
    char * name;  
public: ...  
    const char * GetName();  
};  
class Student: public Person{  
    int rollNo;  
public: ...  
    int GetRollNo();  
};
```

Example

```
int main()
{
    Student sobj;
    cout << sobj.GetName();
    cout << sobj.GetRollNo();
    return 0;
}
```

“IS A” Relationship

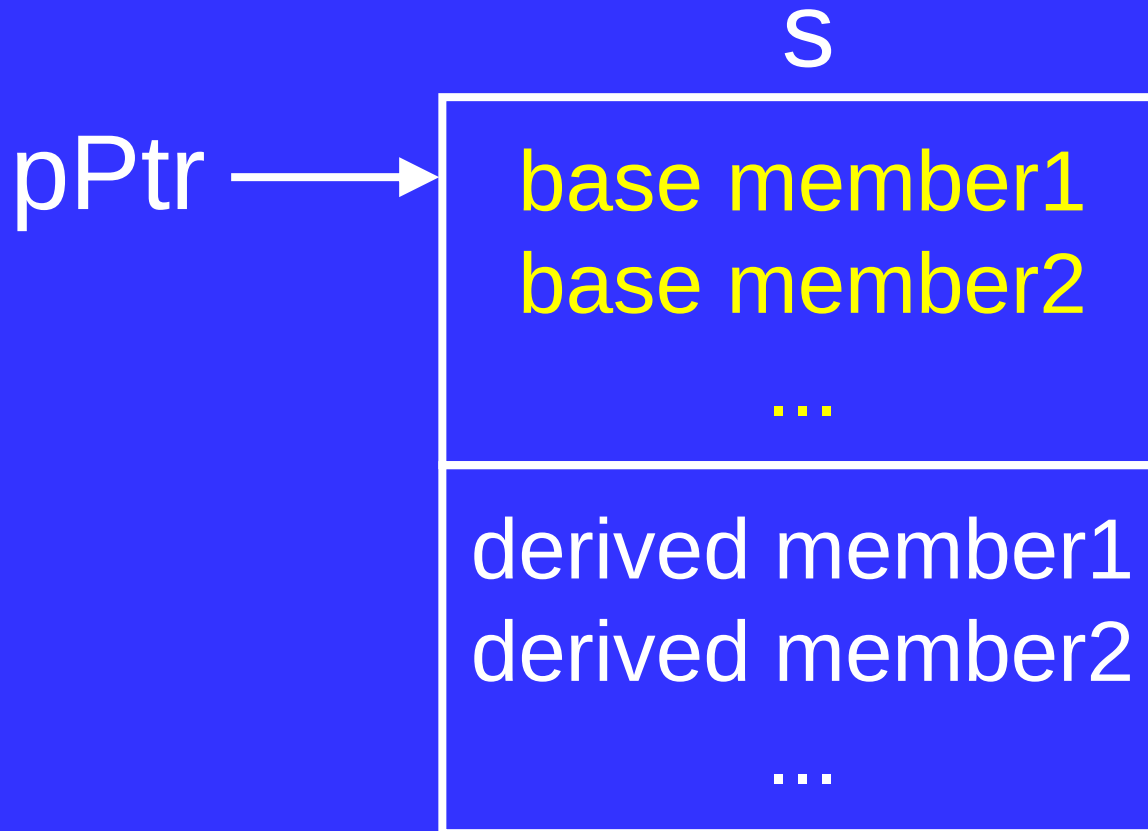
- The base class pointer can point towards an object of derived class

Example

```
int main() {  
    Person * pPtr = 0;  
    Student s;  
    pPtr = &s;  
    cout << pPtr->GetName();  
  
    return 0;  
}
```

Example

```
pPtr = &s;
```



Example

```
int main() {  
    Person * pPtr = 0;  
    Student s;  
    pPtr = &s;  
    //Error  
    cout << pPtr->GetRollNo();  
    return 0;  
}
```



Static Type

- The type that is used to declare a reference or pointer is called its static type
 - The static type of pPtr is Person
 - The static type of s is Student

Member Access

- The access to members is determined by static type
- The static type of pPtr is Person
- Following call is erroneous
`pPtr->GetRollNo();`

“IS A” Relationship

- We can use a reference of derived object where the reference of base object is required

Example

```
int main(){  
    Person p;  
    Student s;  
    Person & refp = s;  
    cout << refp.GetName();  
    cout << refp.GetRollNo(); //Error  
    return 0;  
}
```

Example

```
void Play(const Person& p){  
    cout << p.GetName()  
        << " is playing";  
}
```

```
void Study(const Student& s){  
    cout << s.GetRollNo()  
        << " is Studying";  
}
```

Example

```
int main(){  
    Person p;  
    Student s;  
    Play(p);  
    Play(s);  
    return 0;  
}
```

Object-Oriented Programming (OOP)

Lecture No. 24

Example

```
class Person{  
    char * name;  
public:  
    Person(char * = NULL);  
    const char * GetName() const;  
    ~Person();  
};
```

Example

```
class Student: public Person{  
    char* major;  
public:  
    Student(char *, char *);  
    void Print() const;  
    ~Student();  
};
```

Example

```
Student::Student(char *_name, char
*_maj) : Person(_name), major(NULL)
{
    if (_maj != NULL) {
        major = new char [strlen(_maj)+1];
        strcpy(major,_maj);
    }
}
```

Example

```
void Student::Print() const{  
    cout << "Name: " << GetName()  
        << endl;  
    cout << "Major: " << major  
        << endl;  
}
```

Example

```
int main(){
    Student subj1("Ali", "Computer Science");
    {
        Student subj2 = subj1;
        //      Student subj2(subj1);
        subj2.Print();
    }
    return 0;
}
```

Example

- The output is as follows:

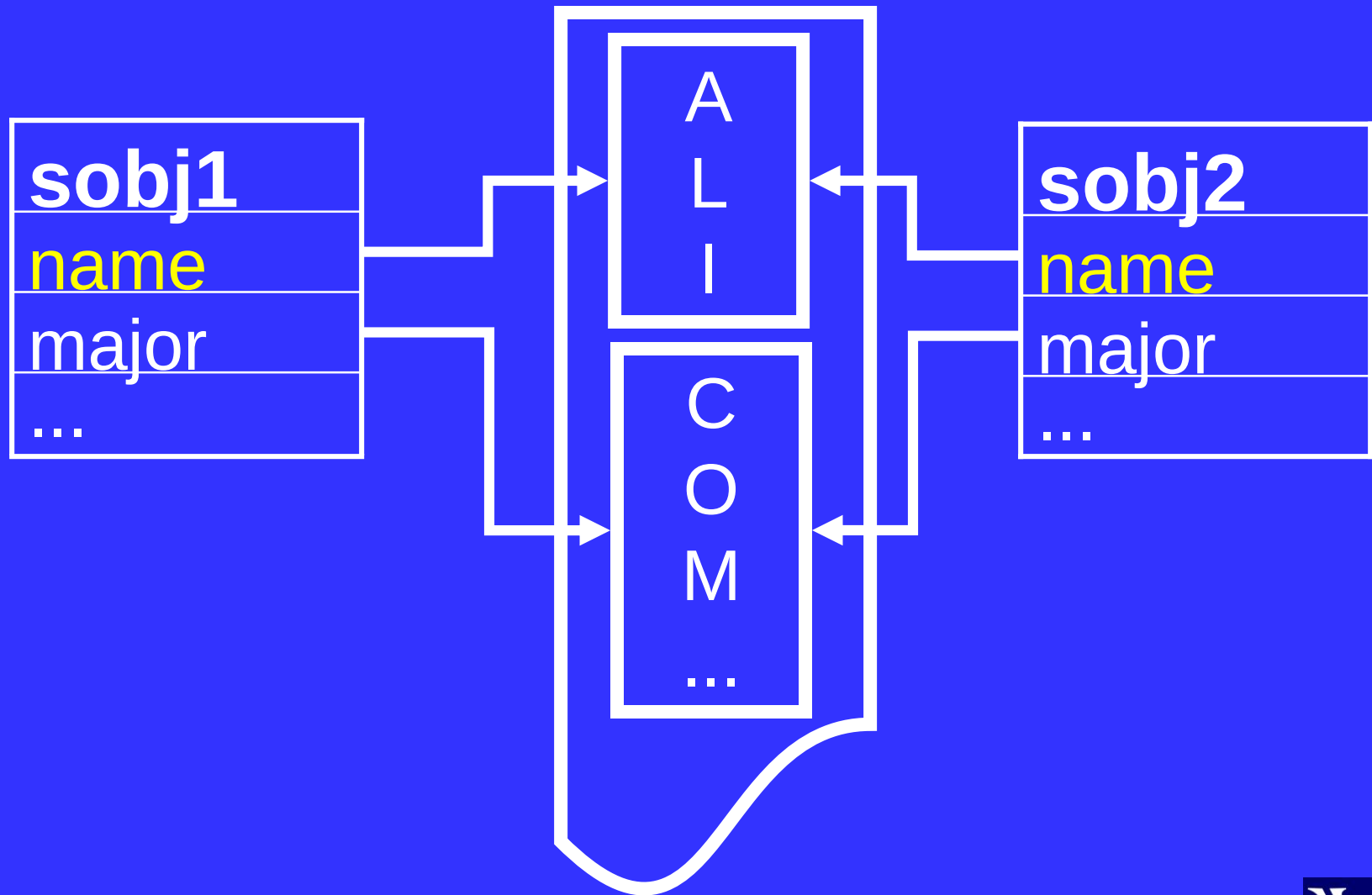
Name: Ali

Major: Computer Science

Copy Constructor

- Compiler generates copy constructor for base and derived classes, if needed
- Derived class Copy constructor is invoked which in turn calls the Copy constructor of the base class
- The base part is copied first and then the derived part

Shallow Copy



Example

```
Person::Person(const Person& rhs){  
    // Code for deep copy  
}
```

```
int main(){  
    Student subj1("Ali", "Computer Science");  
    Student subj2 = subj1;  
    subj2.Print();  
    return 0;  
}
```

Example

- The output is as follows:

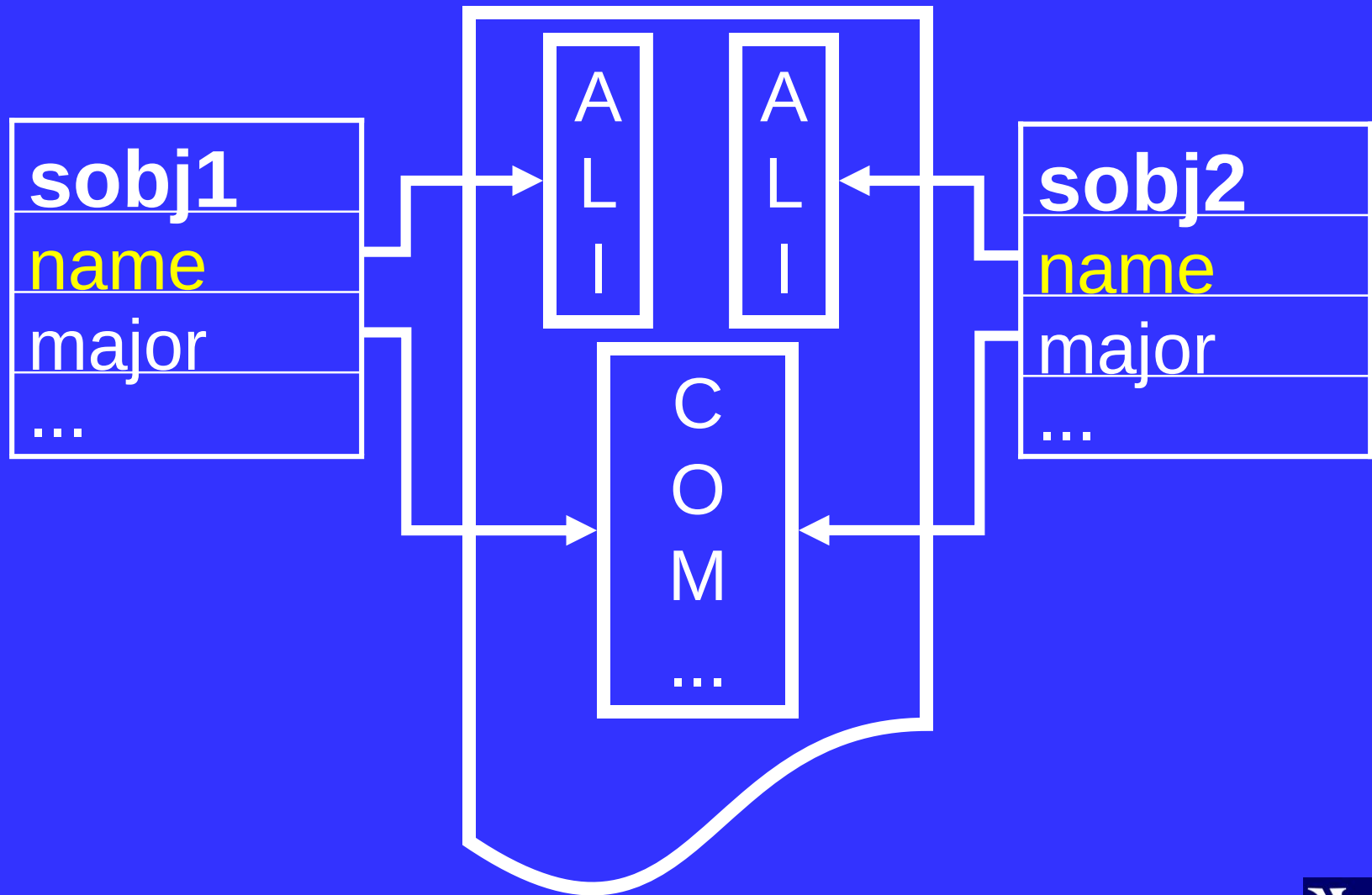
Name: Ali

Major: Computer Science

Copy Constructor

- Compiler generates copy constructor for derived class, calls the copy constructor of the base class and then performs the shallow copy of the derived class's data members

Shallow Copy



Example

```
Person::Person(const Person& rhs)
{
    // Code for deep copy
}

Student::Student
    (const Student& rhs) {
    // Code for deep copy
}
```

Example

```
int main(){  
    Student subj1("Ali",  
                  "Computer Science");  
    Student subj2 = subj1;  
    subj2.Print();  
    return 0;  
}
```

Copy Constructor

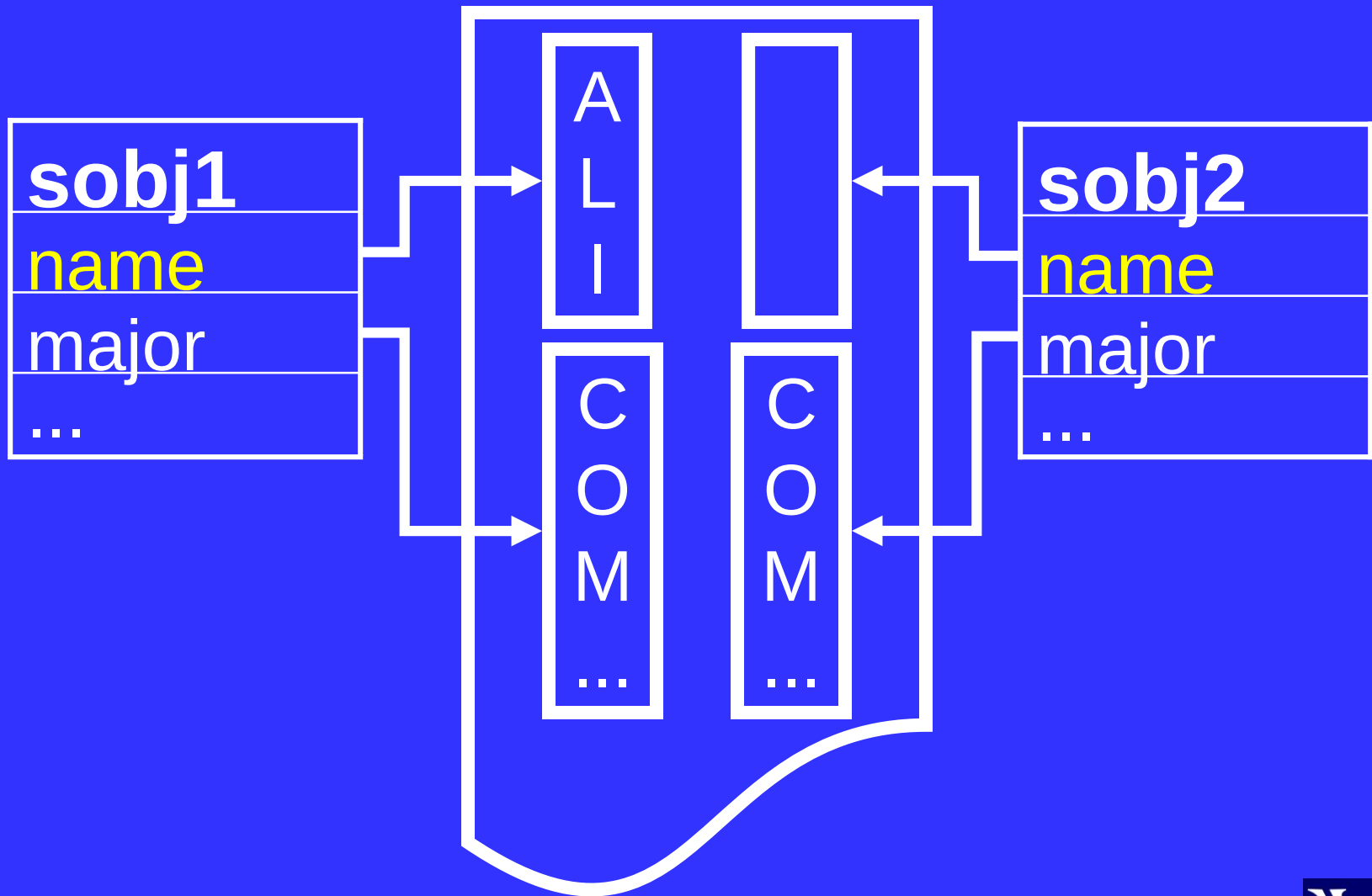
- The output will be as follows:

Name:

Major: Computer Science

- Name of subj2 was not copied from subj1

Copy



Modified Default Constructor

```
Person::Person(char * aName){  
    if(aName == NULL)  
        cout << "Person Constructor";  
  
    ...  
}  
  
int main(){  
    Student s ("Ali","Computer Science");  
  
    ...  
}
```

Copy Constructor

- The output of previous code will be as follows:

Person Constructor

Name:

Major: Computer Science

Copy Constructor

- Programmer must explicitly call the base class copy constructor from the copy constructor of derived class

Example

```
Person::Person(const Person&
                prhs) {
    // Code for deep copy
}

Student::Student(const Student
                  &srhs) :Person(srhs) {
    // Code for deep copy
}
```

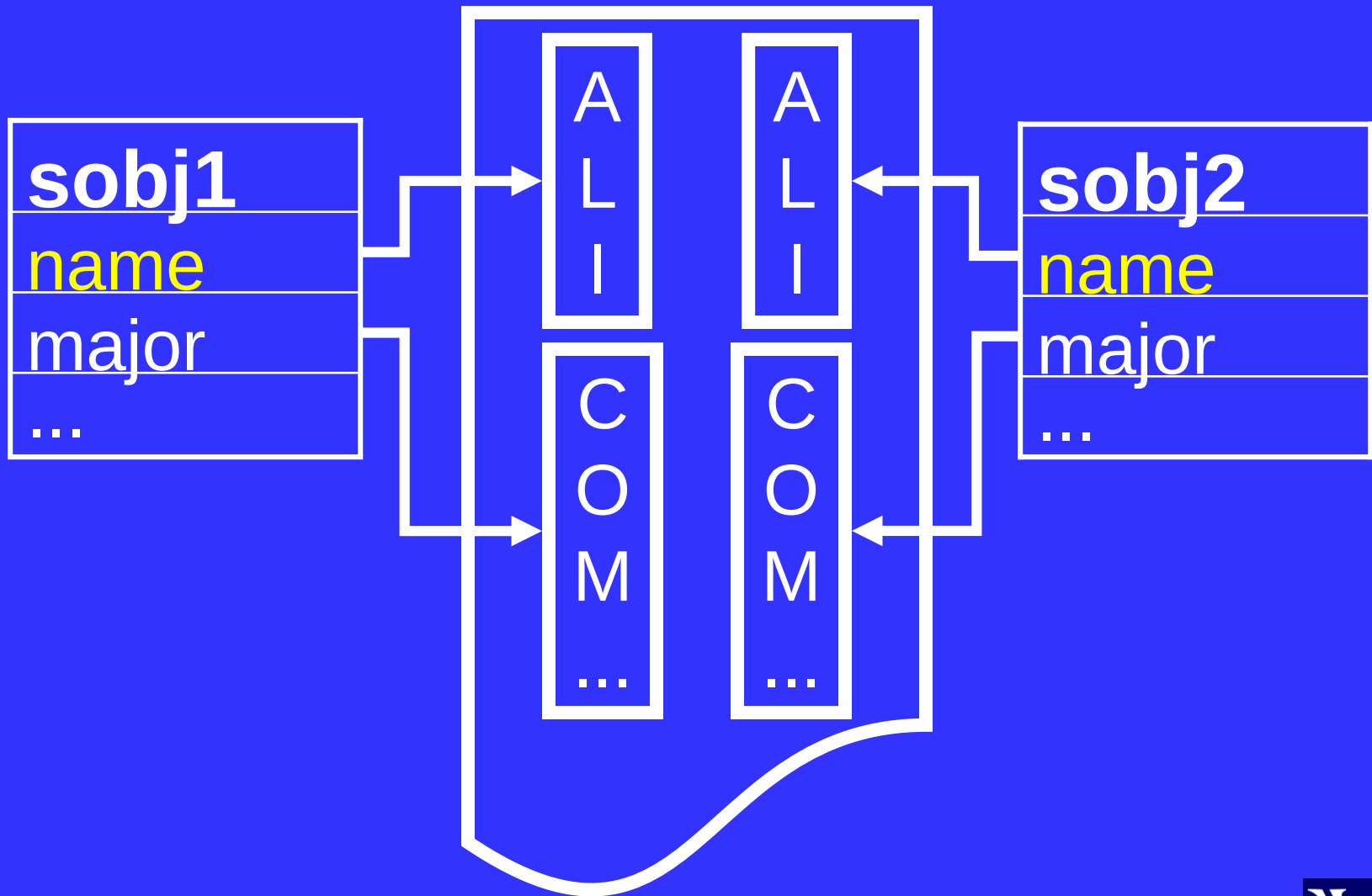
Example

- main function shown previously will give following output

Name: Ali

Major: Computer Science

Copy



Copy Constructors

```
3 | Person::Person(const Person &rhs) :  
4 |     name(NULL) {  
5 |     //code for deep copy  
   | }  
1 | Student::Student(const Student & rhs) :  
6 |     major(NULL),  
2 |     Person(rhs){  
7 |     //code for deep copy  
   | }
```

Example

```
int main()
{
    Student sobj1, sobj2("Ali", "CS");
    sobj1 = sobj2;
    return 0;
}
```


Assignment Operator

- Compiler generates copy assignment operator for base and derived classes, if needed
- Derived class copy assignment operator is invoked which in turn calls the assignment operator of the base class
- The base part is assigned first and then the derived part

Assignment Operator

- Programmer has to call operator of base class, if he is writing assignment operator of derived class

Example

```
class Person{  
public:  
    Person & operator =  
        (const Person & rhs){  
        cout << "Person Assignment";  
        // Code for deep copy assignment  
    }  
};
```

Example

```
class Student: Public Person{  
public:  
    Student & operator = (const Student  
    & rhs){  
        cout<< "Student Assignment";  
        // Code for deep copy assignment  
    }  
};
```

Example

```
int main()
{
    Student sobj1, sobj2("Ali", "CS");
    sobj1 = sobj2;
    return 0;
}
```

Example

- The assignment operator of base class is not called
- Output

Student Assignment

Assignment Operator

- There are two ways of writing assignment operator in derived class
 - Calling assignment operator of base class explicitly
 - Calling assignment operator of base class implicitly

Calling Base Class Member Function

- Base class functions can be explicitly called with reference to base class itself

```
//const char* Person::GetName() {...};  
void Student::Print()  
{  
    cout << GetName();  
    cout << Person::GetName();  
}
```


Explicitly Calling operator =

```
Person & Person::operator =  
    (const Person & prhs);
```

```
Student & Student ::operator =  
    (const Student & srhs){  
    Person::operator = (srhs);  
    ...  
    return *this;  
}
```

Implicitly Calling operator =

```
Student & Student ::operator =  
    (const Student & srhs) {  
    static_cast<Person &>(*this)=srhs;  
    // Person(*this) = srhs;  
    // (Person)*this = srhs;  
    ...  
}
```

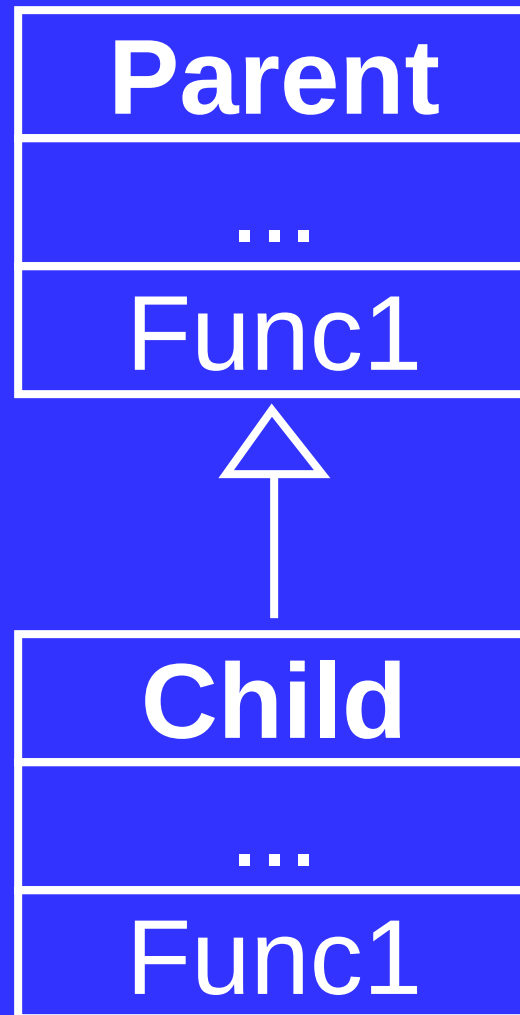
Object-Oriented Programming (OOP)

Lecture No. 25

Overriding Member Functions of Base Class

- Derived class can override the member functions of its base class
- To override a function the derived class simply provides a function with the same signature as that of its base class

Overriding



Overriding

```
class Parent {  
public:  
    void    Func1();  
    void    Func1(int);  
};
```

```
class Child: public Parent {  
public:  
    void Func1();  
};
```

Overloading vs. Overriding

- Overloading is done within the scope of one class
- Overriding is done in scope of parent and child
- Overriding within the scope of single class is error due to duplicate declaration

Overriding

```
class Parent {  
public:  
    void    Func1();  
    void    Func1();    //Error  
};
```


Overriding Member Functions of Base Class

- Derive class can override member function of base class such that the working of function is totally changed

Example

```
class Person{  
public:  
    void Walk();  
};
```

```
class ParalyzedPerson: public Person{  
public:  
    void Walk();  
};
```

Overriding Member Functions of Base Class

- Derive class can override member function of base class such that the working of function is similar to former implementation

Example

```
class Person{
    char *name;
public:
    Person(char *=NULL);
    const char *GetName() const;
    void Print(){
        cout << "Name: " << name
              << endl;
    }
};
```

Example

```
class Student : public Person{
    char * major;
public:
    Student(char * aName, char* aMajor);

    void Print(){
        cout<<"Name: "<< GetName()<<endl
            << "Major:" << major<< endl;
    }
    ...
};
```

Example

```
int main() {  
    Student a("Ahmad",  
        "Computer Science");  
    a.Print();  
    return 0;  
}
```

Output

Output:

Name: Ahmed

Major: Computer Science

Overriding Member Functions of Base Class

- Derive class can override member function of base class such that the working of function is based on former implementation

Example

```
class Student : public Person{
    char * major;
public:
    Student(char * aName, char* m);

    void Print(){
        Print();//Print of Person
        cout<<"Major:" << major <<endl;
    }
    ...
};
```

Example

```
int main() {  
    Student a("Ahmad",  
    "Computer Science");  
    a.Print();  
    return 0;  
}
```

Output

- There will be no output as the compiler will call the print of the child class from print of child class recursively
- There is no ending condition

Example

```
class Student : public Person{
    char * major;
public:
    Student(char * aName, char* m);

    void Print(){
        Person::Print();
        cout<<"Major:" << major <<endl;
    }
    ...
};
```

Example

```
int main() {  
    Student a("Ahmad",  
    "Computer Science");  
    a.Print();  
    return 0;  
}
```

Output

Output:

Name: Ahmed

Major: Computer Science

Overriding Member Functions of Base Class

- The pointer must be used with care when working with overridden member functions

Example

```
int main() {  
    Student a("Ahmad", "Computer  
              Science");  
  
    Student *sPtr = &a;  
    sPtr->Print();  
  
    Person *pPtr = sPtr;  
    pPtr->Print();  
    return 0;  
}
```


Example

Output:

Name: Ahmed

Major: Computer Science

Name: Ahmed

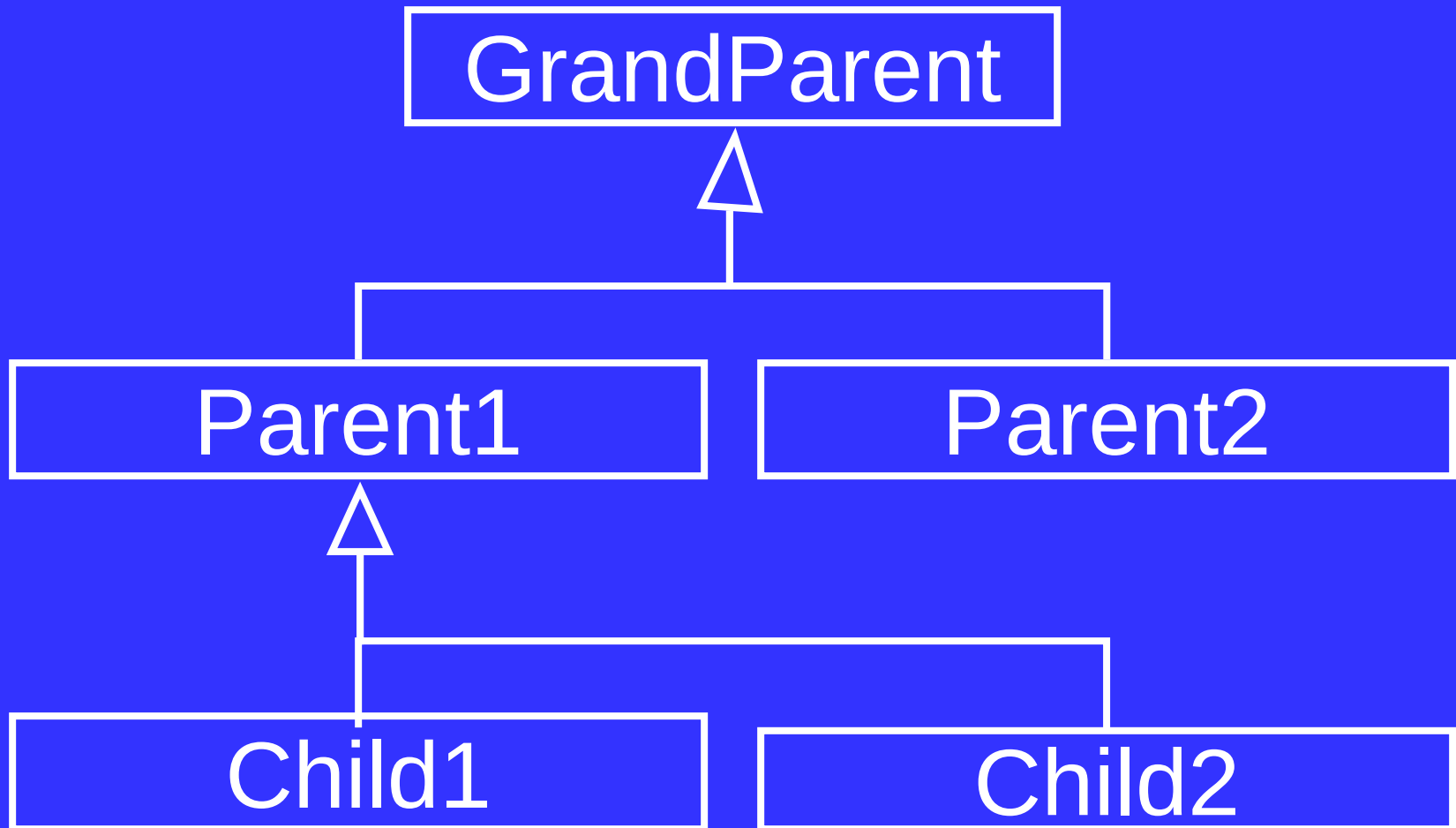
Overriding Member Functions of Base Class

- The member function is called according to static type
- The static type of pPtr is Person
- The static type of sPtr is Student

Hierarchy of Inheritance

- We represent the classes involved in inheritance relation in tree like hierarchy

Example



Direct Base Class

- A direct base class is explicitly listed in a derived class's header with a colon (:)

```
class Child1:public Parent1  
...
```

Indirect Base Class

- An indirect base class is not explicitly listed in a derived class's header with a colon (:)
- It is inherited from two or more levels up the hierarchy of inheritance

```
class GrandParent{};  
class Parent1:  
    public GrandParent {};  
class Child1:public Parent1{};
```

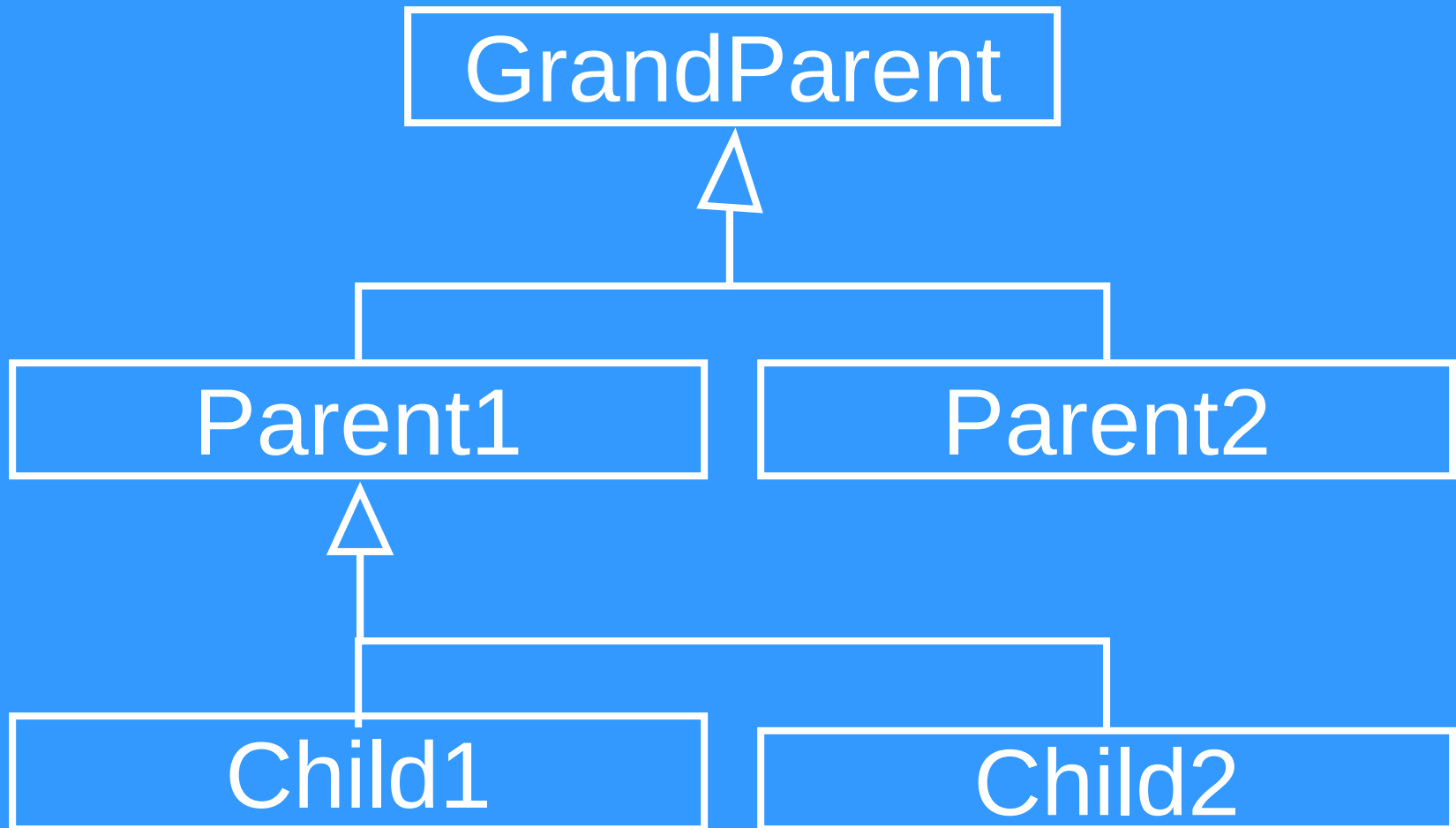
Object-Oriented Programming (OOP)

Lecture No. 26

Hierarchy of Inheritance

- We represent the classes involved in inheritance relation in tree like hierarchy

Example



Direct Base Class

- A direct base class is explicitly listed in a derived class's header with a colon (:)

```
class Child1:public Parent1  
...
```

Indirect Base Class

- An indirect base class is not explicitly listed in a derived class's header with a colon (:)
- It is inherited from two or more levels up the hierarchy of inheritance

```
class GrandParent{};  
class Parent1:  
    public GrandParent {};  
class Child1:public Parent1{};
```

Base Initialization

- The child can only perform the initialization of direct base class through *base class initialization list*
- The child can not perform the initialization of an indirect base class through *base class initialization list*

Example

```
class GrandParent{  
    int gpData;  
public:  
    GrandParent() : gpData(0){...}  
    GrandParent(int i) : gpData(i){...}  
    void Print() const;  
};
```

Example

```
class Parent1: public GrandParent{  
    int pData;  
public:  
    Parent1() : GrandParent(),  
                pData(0) {...}  
};
```

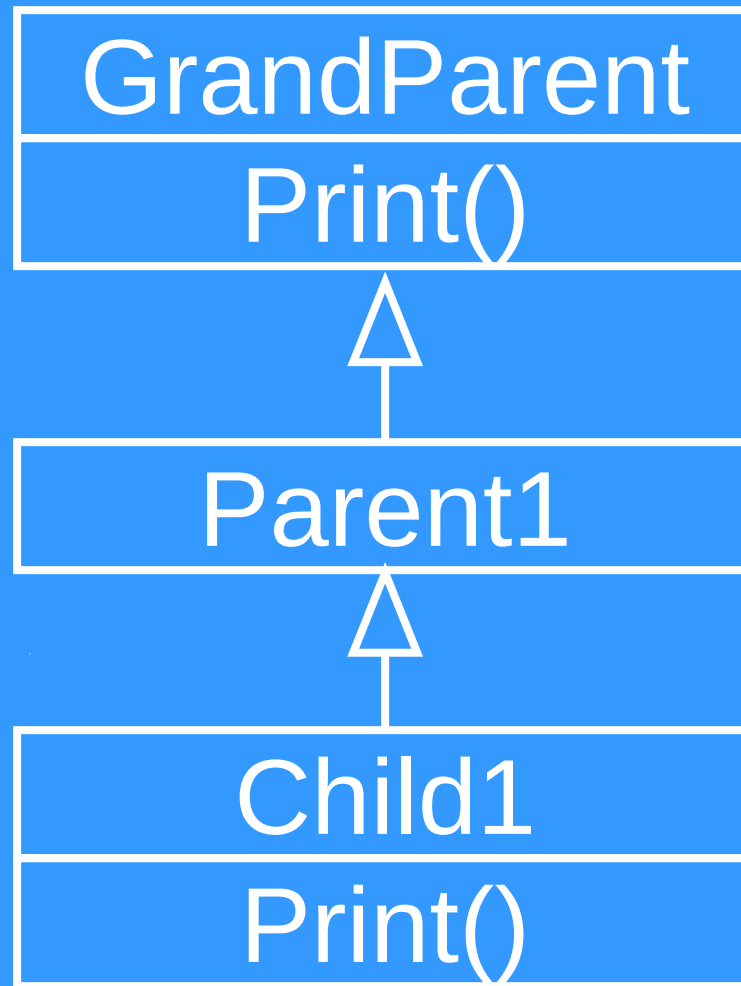
Example

```
class Child1 : public Parent1 {  
public:  
    Child1() : Parent1()    {...}  
    Child1(int i) : GrandParent (i)  
    //Error  
    {...}  
    void Print() const;  
};
```

Overriding

- Child class can override the function of GrandParent class

Example



Example

```
void GrandParent::Print() {  
    cout << "GrandParent::Print"  
        << endl;  
}
```

```
void Child1::Print() {  
    cout << "Child1::Print" << endl;  
}
```

Example

```
int main(){  
    Child1 obj;  
    obj.Print();  
    obj.Parent1::Print();  
    obj.GrandParent::Print();  
    return 0;  
}
```

Output

- Output is as follows

Child1::Print

GrandParent::Print

GrandParent::Print

Types of Inheritance

- There are three types of inheritance
 - Public
 - Protected
 - Private
- Use keyword public, private or protected to specify the type of inheritance

Public Inheritance

```
class Child: public Parent {...};
```

Member access in

Base Class	Derived Class
Public	Public
Protected	Protected
Private	Hidden

Protected Inheritance

```
class Child: protected Parent {...};
```

Member access in

Base Class	Derived Class
Public	Protected
Protected	Protected
Private	Hidden

Private Inheritance

```
class Child: private Parent {...};
```

Member access in

Base Class	Derived Class
Public	Private
Protected	Private
Private	Hidden

Private Inheritance

- If the user does not specifies the type of inheritance then the default type is private inheritance

class Child: **private** Parent {...}

is equivalent to

class Child: Parent {...}

Private Inheritance

- We use private inheritance when we want to reuse code of some class
- Private Inheritance is used to model “Implemented in terms of” relationship

Example

```
class Collection {  
...  
public:  
    void AddElement(int);  
    bool SearchElement(int);  
    bool SearchElementAgain(int);  
    bool DeleteElement(int);  
};
```

Example

- If element is not found in the Collection the function SearchElement will return false
- SearchElementAgain finds the second instance of element in the collection

Class Set

```
class Set: private Collection {  
private:  
    ...  
public:  
    void AddMember(int);  
    bool IsMember(int);  
    bool DeleteMember(int);  
};
```

Class Set

```
void Set::AddMember(int i){  
    if (! IsMember(i) )  
        AddElement(i);  
}  
  
bool Set::IsMember(int i){  
    return SearchElement(i);  
}
```

Object-Oriented Programming (OOP)

Lecture No. 27

Class Collection

```
class Collection {  
    ...  
public:  
    void AddElement(int);  
    bool SearchElement(int);  
    bool SearchElementAgain(int);  
    bool DeleteElement(int);  
};
```


Class Set

```
class Set: private Collection {  
private:  
    ...  
public:  
    void AddMember(int);  
    bool IsMember(int);  
    bool DeleteMember(int);  
};
```

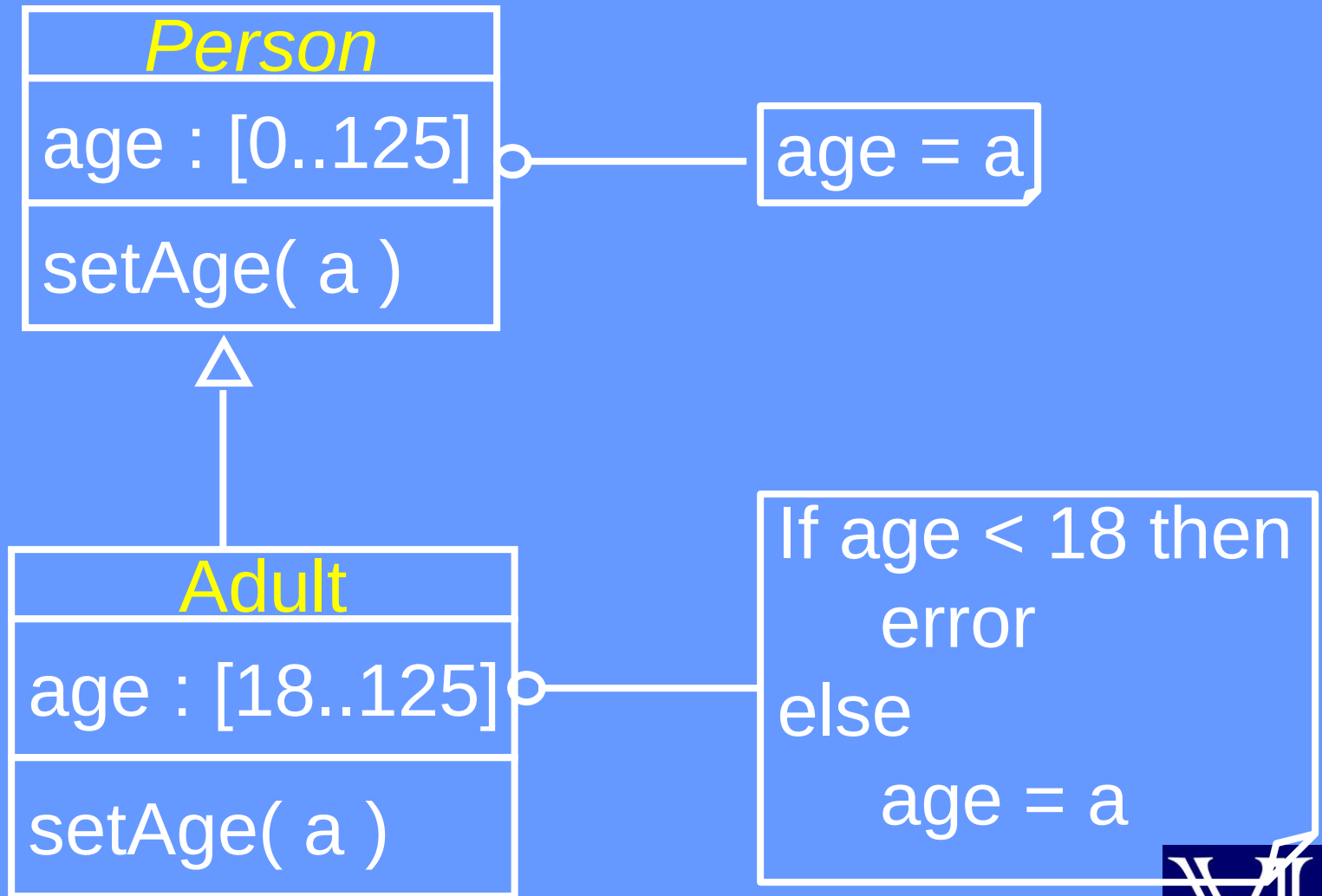
Specialization (Restriction)

- the derived class is behaviourally incompatible with the base class
- Behaviourally incompatible means that base class can't always be replaced by the derived class

Specialization (Restriction)

- Specialization (Restriction) can be implemented using private and protected inheritance

Example – Specialization (Restriction)



Example

```
class Person{  
    ...  
protected:  
    int age;  
public:  
    bool SetAge(int _age){  
        if (_age >=0 && _age <= 125) {  
            age = _age;  
            return true;  
        }  
        return false;  
    }  
};
```

Example

```
class Adult : private Person {  
    public:  
    bool SetAge(int _age){  
        if (_age >=18 && _age <= 125) {  
            age = _age;  
            return true;  
        }  
        return false;  
    }  
};
```

Private Inheritance

- Only member functions and friend functions of a derived class can convert pointer or reference of derived object to that of parent object

Example

```
class Parent{  
};  
class Child : private Parent{  
};
```

```
int main(){  
    Child cobj;  
    Parent *pptr = & cobj;    //Error  
    return 0;  
}
```


Example

```
void DoSomething(const Parent &);
```

```
Child::Child(){  
    Parent & pPtr =  
        static_cast<Parent &>(*this);  
    DoSomething(pPtr);  
    // DoSomething(*this);  
}
```

Private Inheritance

- The child class object has an anonymous object of parent class object
- The default constructor and copy constructor of parent class are called when needed

Example

```
class Parent{  
public:  
    Parent(){  
        cout << "Parent Constructor";  
    }  
  
    Parent(const Parent & prhs){  
        cout << "Parent Copy Constructor";  
    }  
};
```

Example

```
class Child: private Parent{
public:
    Child(){
        cout << "Child Constructor";
    }

    Child(const Child & crhs)
        :Parent(crhs){
        cout << "Child Copy Constructor";
    }
};
```

Example

```
int main() {  
    Child cobj1;  
    Child cobj2 = cobj1;  
    //Child cobj2(cobj1);  
    return 0;  
}
```

Example

- Output:

Parent Constructor

Child Constructor

Parent Copy Constructor

Child Copy Constructor

Private Inheritance

- The base class that is more than one level down the hierarchy cannot access the member function of parent class, if we are using private inheritance

Class Hierarchy

```
class GrandParent{  
public :  
    void DoSomething();  
};
```

```
class Parent: private GrandParent{  
    void SomeFunction(){  
        DoSomething();  
    }  
};
```


Example

```
class Child: private Parent
{
public:
    Child() {
        DoSomething();    //Error
    }
};
```

Private Inheritance

- The base class that is more than one level down the hierarchy cannot convert the pointer or reference to child object to that of parent, if we are using private inheritance

Class Hierarchy

```
void DoSomething(GrandParent&);  
class GrandParent{  
};  
class Parent: private GrandParent{  
public:  
    Parent() {DoSomething(*this);}  
};
```

Example

```
class Child: private Parent {  
public:  
    Child()  
    {  
        DoSomething(*this);    //Error  
    }  
};
```

Protected Inheritance

- Use protected inheritance if you want to build class hierarchy using “implemented in terms of”

Protected Inheritance

- If B is a protected base and D is derived class then public and protected members of B can be used by member functions and friends of classes derived from D

Class Hierarchy

```
class GrandParent{  
public :  
    void DoSomething();  
};
```

```
class Parent: protected GrandParent{  
    void SomeFunction(){  
        DoSomething();  
    }  
};
```

Example

```
class Child: protected Parent
{
public:
    Child()
    {
        DoSomething();
    }
};
```


Protected Inheritance

- If B is a protected base and D is derived class then only friends and members of D and friends and members of class derived from D can convert D^* to B^* or $D\&$ to $B\&$

Class Hierarchy

```
void DoSomething(GrandParent&);
```

```
class GrandParent{  
};
```

```
class Parent: protected  
    GrandParent{  
};
```

Example

```
class Child: protected Parent {  
public:  
    Child()  
    {  
        DoSomething(*this);  
    }  
};
```

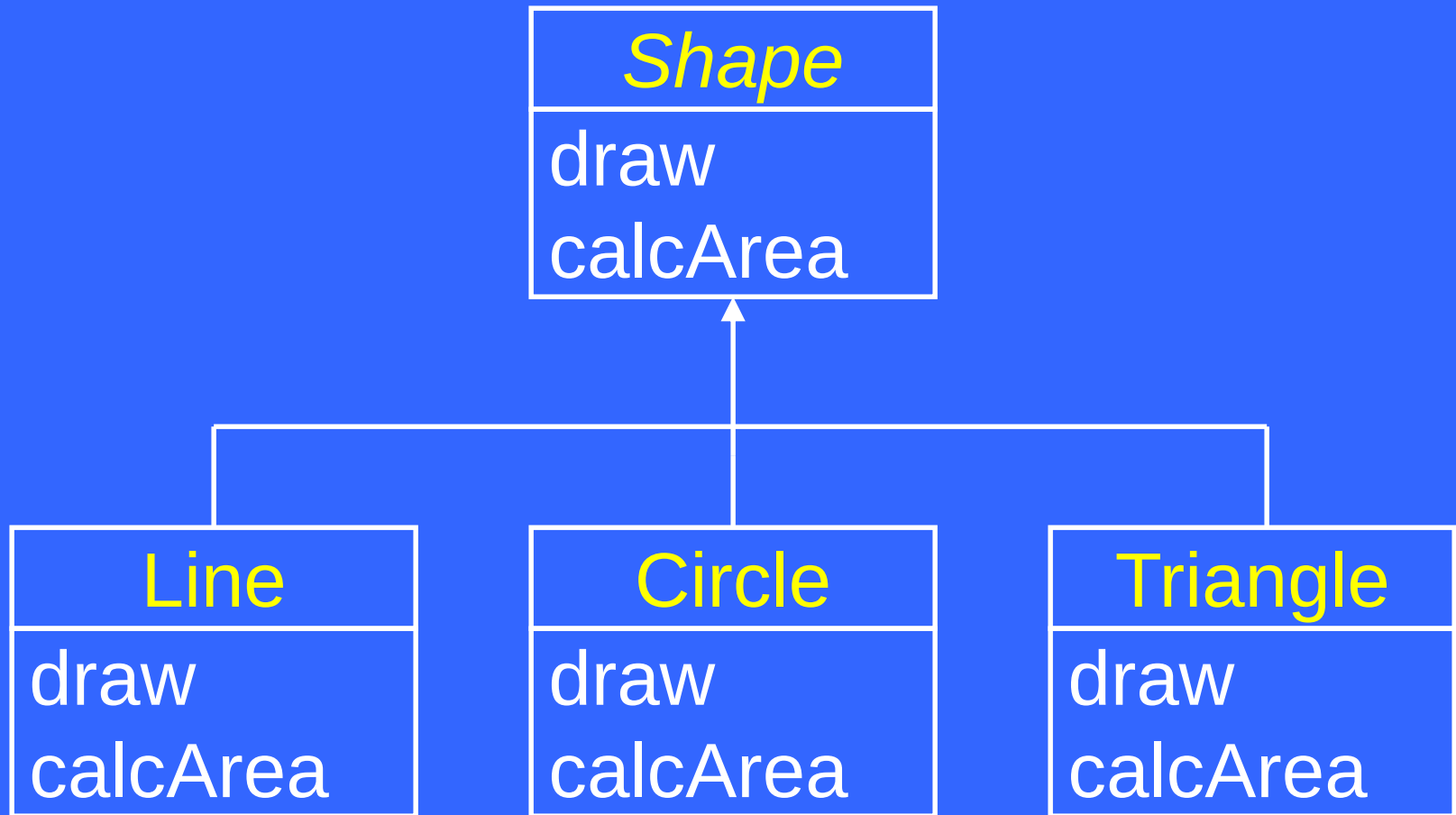
Object-Oriented Programming (OOP)

Lecture No. 28

Problem Statement

- Develop a function that can draw different types of geometric shapes from an array

Shape Hierarchy



Shape Hierarchy

```
class Shape {  
    ...  
protected:  
    char _type;  
public:  
    Shape() { }  
    void draw() { cout << "Shape\n"; }  
    int calcArea() { return 0; }  
    char getType() { return _type; }  
}
```

... Shape Hierarchy

```
class Line : public Shape {  
    ...  
public:  
    Line(Point p1, Point p2) {  
        ...  
    }  
    void draw() { cout << "Line\n"; }  
}
```


... Shape Hierarchy

```
class Circle : public Shape {  
    ...  
public:  
    Circle(Point center, double radius)  
    {  
        ...  
    }  
    void draw() { cout << "Circle\n"; }  
    int calcArea() { ... }  
}
```

... Shape Hierarchy

```
class Triangle : public Shape {  
    ...  
public:  
    Triangle(Line l1, Line l2,  
              double angle)  
    { ... }  
    void draw() { cout << "Triangle\n"; }  
    int calcArea() { ... }  
}
```

Drawing a Scene

```
int main() {  
    Shape* _shape[ 10 ];  
    Point p1(0, 0), p2(10, 10);  
    shape[1] = new Line(p1, p2);  
    shape[2] = new Circle(p1, 15);  
    ...  
    void drawShapes( shape, 10 );  
    return 0;  
}
```

Function drawShapes()

```
void drawShapes (Shape* _shape[],  
                 int size) {  
    for (int i = 0; i < size; i++)  
    {  
        _shape[i]->draw();  
    }  
}
```

Sample Output

Shape

Shape

Shape

Shape

...

Function drawShapes()

```
void drawShapes(  
    Shape* _shape[], int size) {  
    for (int i = 0; i < size; i++) {  
        // Determine object type with  
        // switch & accordingly call  
        // draw() method  
    }  
}
```

Required Switch Logic

```
switch ( _shape[i]->getType() )
{
    case 'L' :
        static_cast<Line*>(_shape[i])->draw() ;
        break;
    case 'C' :
        static_cast<Circle*>(_shape[i])
                                ->draw() ;
        break;
    ...
}
```

Equivalent If Logic

```
if ( _shape[i]->getType() == 'L' )  
    static_cast<Line*>(_shape[i])->draw();  
else if ( _shape[i]->getType() == 'C' )  
    static_cast<Circle*>(_shape[i])->draw();  
...
```


Sample Output

Line

Circle

Triangle

Circle

...

Problems with Switch Statement

...Delocalized Code

- Consider a function that prints area of each shape from an input array

Function printArea

```
void printArea(  
    Shape* _shape[], int size) {  
    for (int i = 0; i < size; i++) {  
        // Print shape name.  
        // Determine object type with  
        // switch & accordingly call  
        // calcArea() method.  
    }  
}
```

Required Switch Logic

```
switch ( _shape[i]->getType() )
{
    case 'L' :
        static_cast<Line*>(_shape[i])
            ->calcArea();    break;
    case 'C' :
        static_cast<Circle*>(_shape[i])
            ->calcArea();    break;
    ...
}
```

...Delocalized Code

- The above switch logic is same as was in function `drawArray()`
- Further we may need to draw shapes or calculate area at more than one places in code

Other Problems

- Programmer may forget a check
- May forget to test all the possible cases
- Hard to maintain

Solution?

- To avoid switch, we need a mechanism that can select the message target automatically!

Polymorphism Revisited

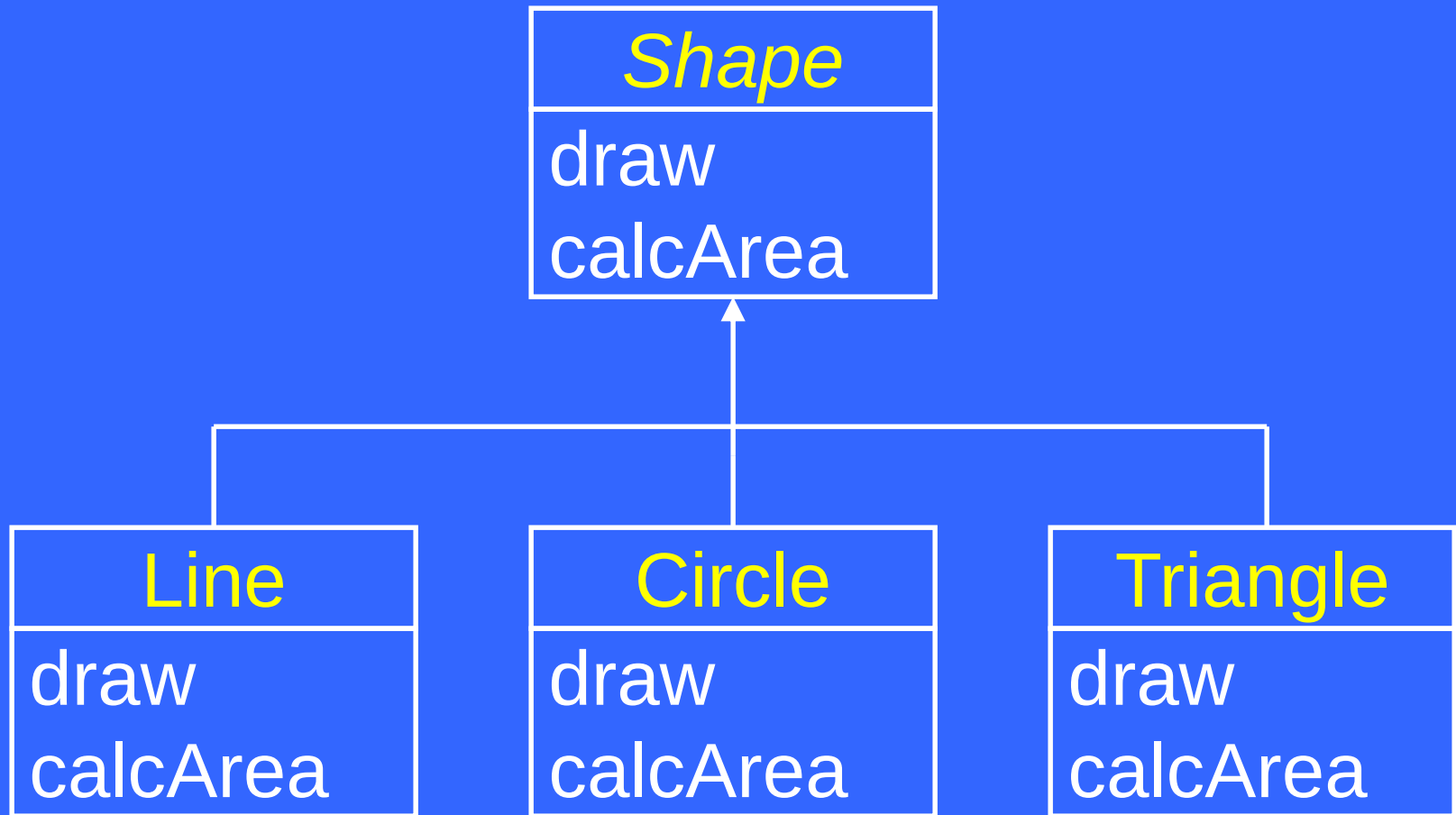
- In OO model, polymorphism means that different objects can behave in different ways for the same message (stimulus)
- Consequently, sender of a message does not need to know the exact class of receiver

Virtual Functions

- Target of a virtual function call is determined at run-time
- In C++, we declare a function virtual by preceding the function header with keyword “virtual”

```
class Shape {  
    ...  
    virtual void draw();  
}
```

Shape Hierarchy



...Shape Hierarchy Revisited

No type field

```
class Shape {  
    ...  
    virtual void draw();  
    virtual int calcArea();  
}  
class Line : public Shape {  
    ...  
    virtual void draw();  
}
```

... Shape Hierarchy Revisited

```
class Circle : public Shape {  
    ...  
    virtual void draw();  
    virtual int calcArea();  
}  
class Triangle : public Shape {  
    ...  
    virtual void draw();  
    virtual int calcArea();  
}
```

Function drawShapes()

```
void drawShapes (Shape* _shape[],  
                 int size) {  
    for (int i = 0; i < size; i++) {  
        _shape[i]->draw();  
    }  
}
```

Sample Output

Line

Circle

Triangle

Circle

...

Function printArea

```
void printArea(Shape* _shape[],  
               int size) {  
    for (int i = 0; i < size; i++) {  
        // Print shape name  
        cout<< _shape[i]  
              ->calcArea();  
        cout << endl;  
    }  
}
```


Static vs Dynamic Binding

- Static binding means that target function for a call is selected at compile time
- Dynamic binding means that target function for a call is selected at run time

Static vs Dynamic Binding

```
Line _line;  
_line.draw();      // Always Line::draw  
                   // called
```

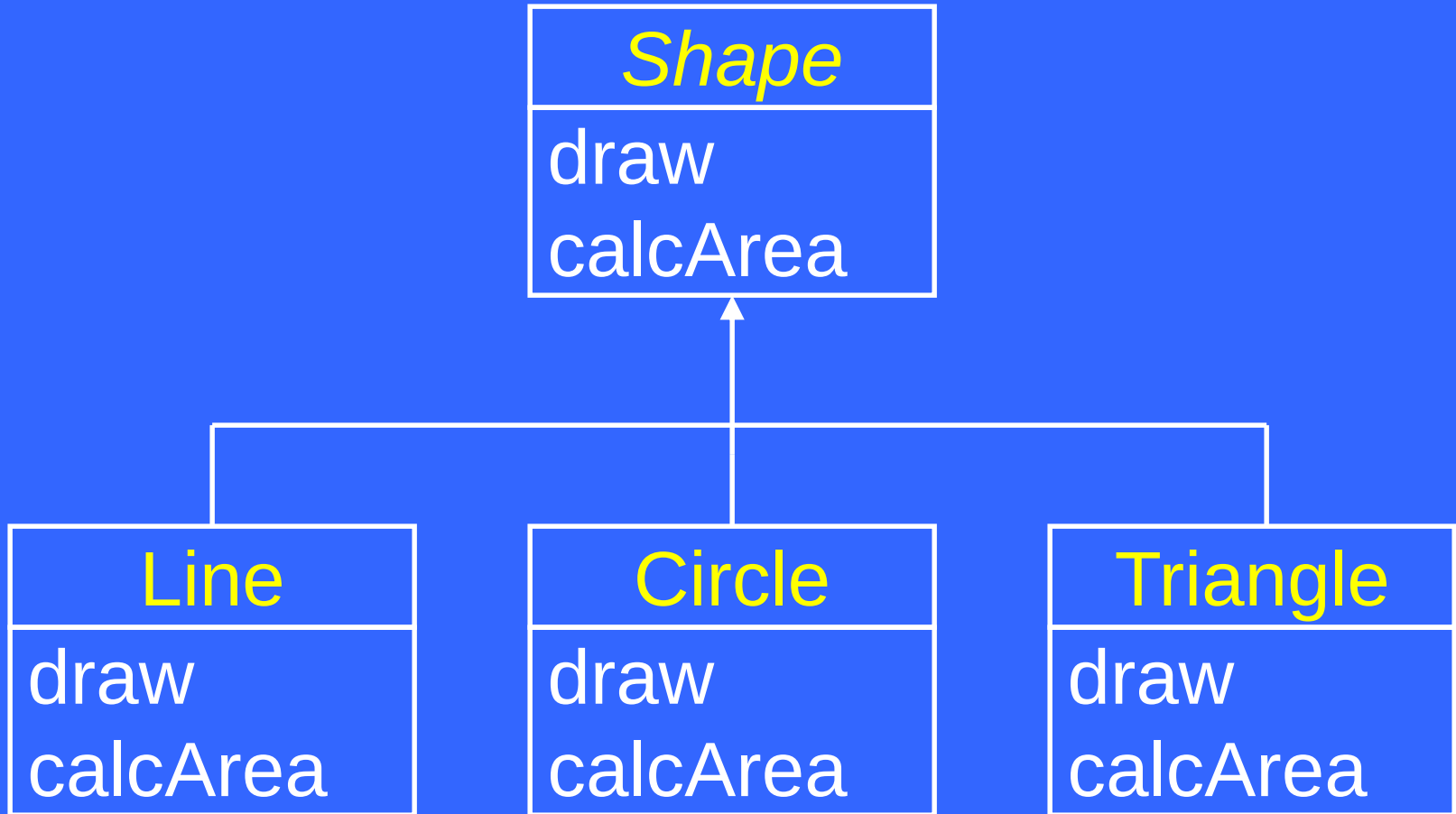
```
Shape* _shape = new Line();  
_shape->draw(); // Shape::draw called  
               // if draw() is not virtual
```

```
Shape* _shape = new Line();  
_shape->draw(); // Line::draw called  
               // if draw() is virtual
```

Object-Oriented Programming (OOP)

Lecture No. 29

Abstract Class



Abstract Class

- Implements an abstract concept
- Cannot be instantiated
- Used for inheriting interface and/or implementation

Concrete Class

- Implements a concrete concept
- Can be instantiated
- May inherit from an abstract class or another concrete class

Abstract Classes in C++

- In C++, we can make a class abstract by making its function(s) pure virtual
- Conversely, a class with no pure virtual function is a concrete class

Pure Virtual Functions function

- A pure virtual represents an abstract behavior and therefore may not have its implementation (body)
- A function is declared pure virtual by following its header with “= 0”

```
virtual void draw() = 0;
```


... Pure Virtual Functions

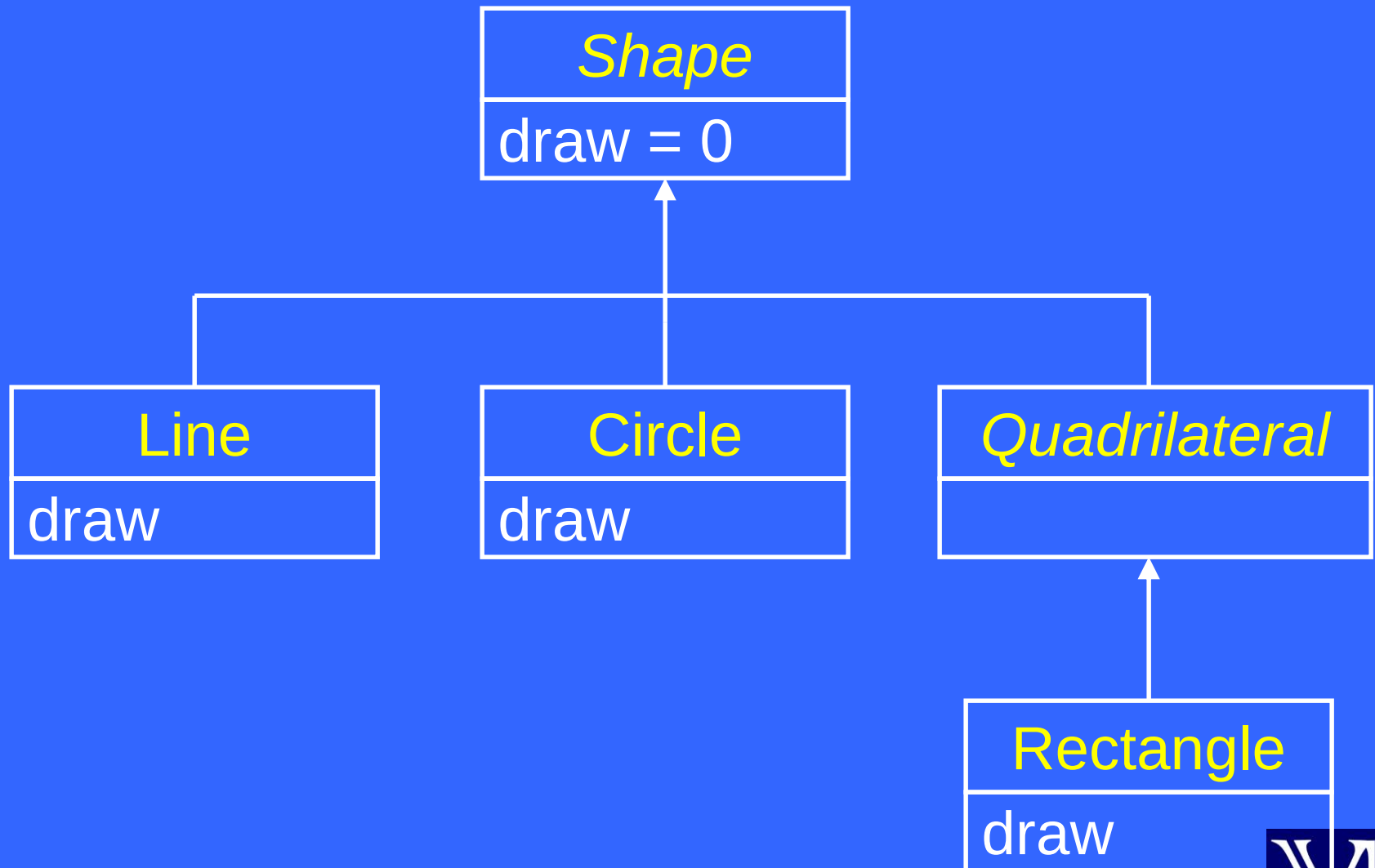
- A class having pure virtual function(s) becomes abstract

```
class Shape {  
    ...  
public:  
    virtual void draw() = 0;  
}  
...  
Shape s;           // Error!
```

... Pure Virtual Functions

- A derived class of an abstract class remains abstract until it provides implementation for all pure virtual functions

Shape Hierarchy



... Pure Virtual Functions

```
class Quadrilateral : public Shape {  
    ...  
    // No overriding draw() method  
}  
...  
Quadrilateral q; // Error!
```

... Pure Virtual Functions

```
class Rectangle:public Quadrilateral{
    ...
public:
    // void draw()
    virtual void draw() {
        ... // function body
    }
}
...
Rectangle r;           // OK
```

Virtual Destructors

```
class Shape {  
    ...  
public:  
    ~Shape() {  
        cout << "Shape destructor  
                                   called\n";  
    }  
}
```

...Virtual Destructors

```
class Quadrilateral : public Shape {  
    ...  
public:  
    ~Quadrilateral() {  
        cout << "Quadrilateral destructor  
                called\n";  
    }  
}
```

...Virtual Destructors

```
class Rectangle : public
                    Quadrilateral {
...
public:
~Rectangle() {
    cout << "Rectangle destructor
                    called\n";
}
}
```


...Virtual Destructors

- When delete operator is applied to a base class pointer, base class destructor is called regardless of the object type

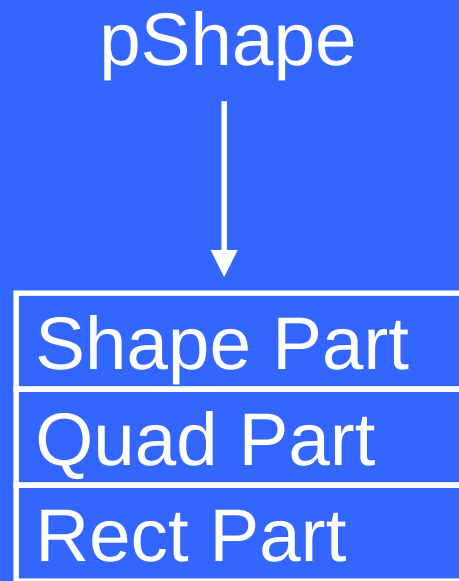
...Virtual Destructors

```
int main() {  
    Shape* pShape = new Rectangle();  
    delete pShape;  
    return 0;  
}
```

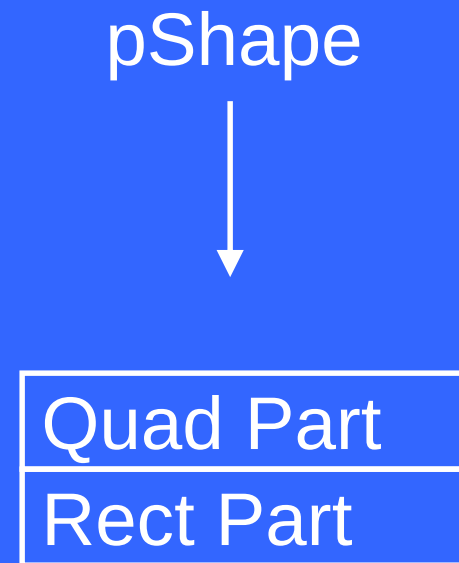
- Output

Shape destructor called

Result



Before



After

Virtual Destructors

- Make the base class destructor virtual

```
class Shape {  
    ...  
public:  
    virtual ~Shape() {  
        cout << "Shape destructor  
                called\n"; }  
}
```

...Virtual Destructors

```
class Quadrilateral : public Shape {  
    ...  
public:  
    virtual ~Quadrilateral() {  
        cout << "Quadrilateral destructor  
                called\n";  
    }  
}
```

...Virtual Destructors

```
class Rectangle : public
                    Quadrilateral {
...
public:
virtual ~Rectangle() {
    cout << "Rectangle destructor
                                called\n";
}
}
```

...Virtual Destructors

- Now base class destructor will run after the derived class destructor

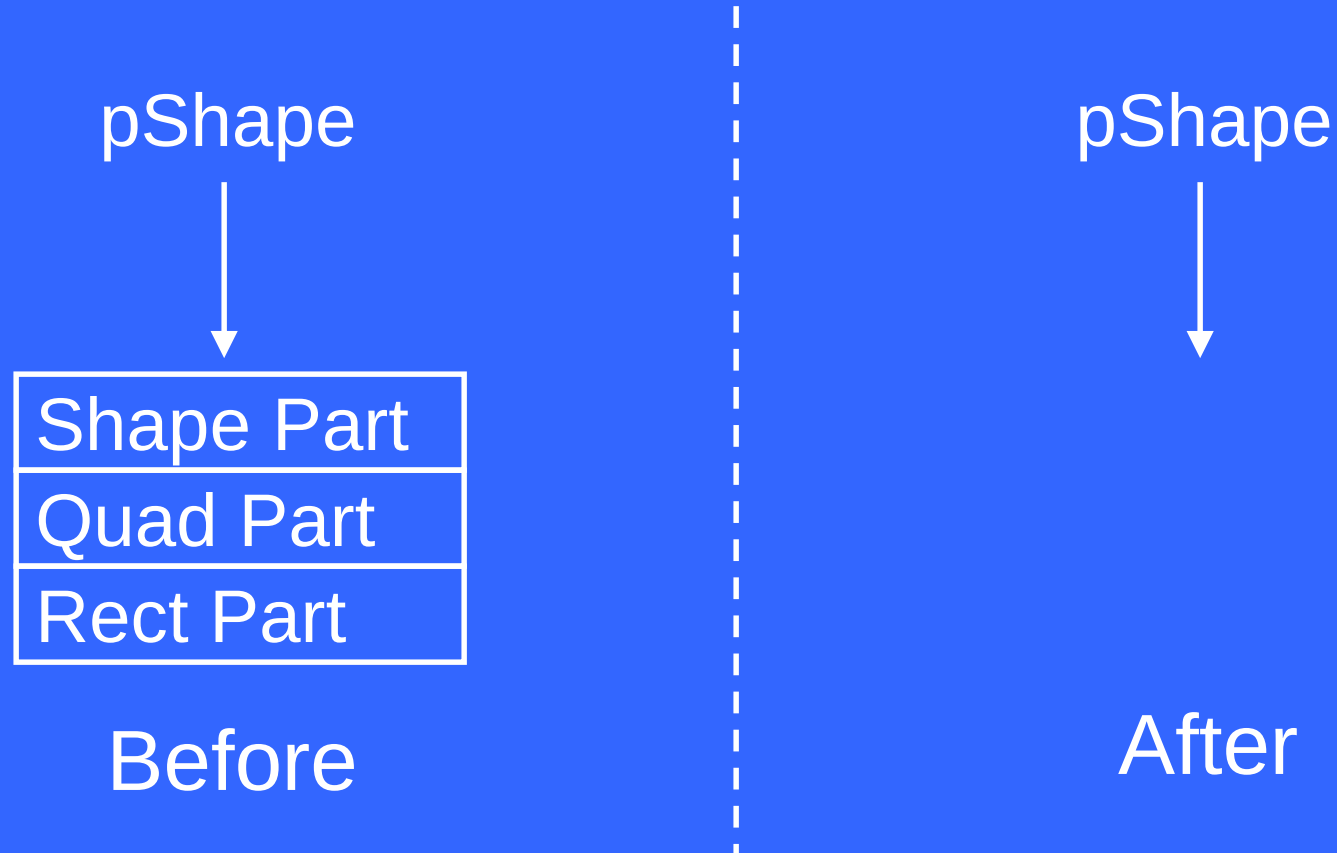
...Virtual Destructors

```
int main() {  
    Shape* pShape = new Rectangle();  
    delete pShape;  
    return 0;  
}
```

- Output

Rectangle destructor called
Quadrilateral destructor called
Shape destructor called

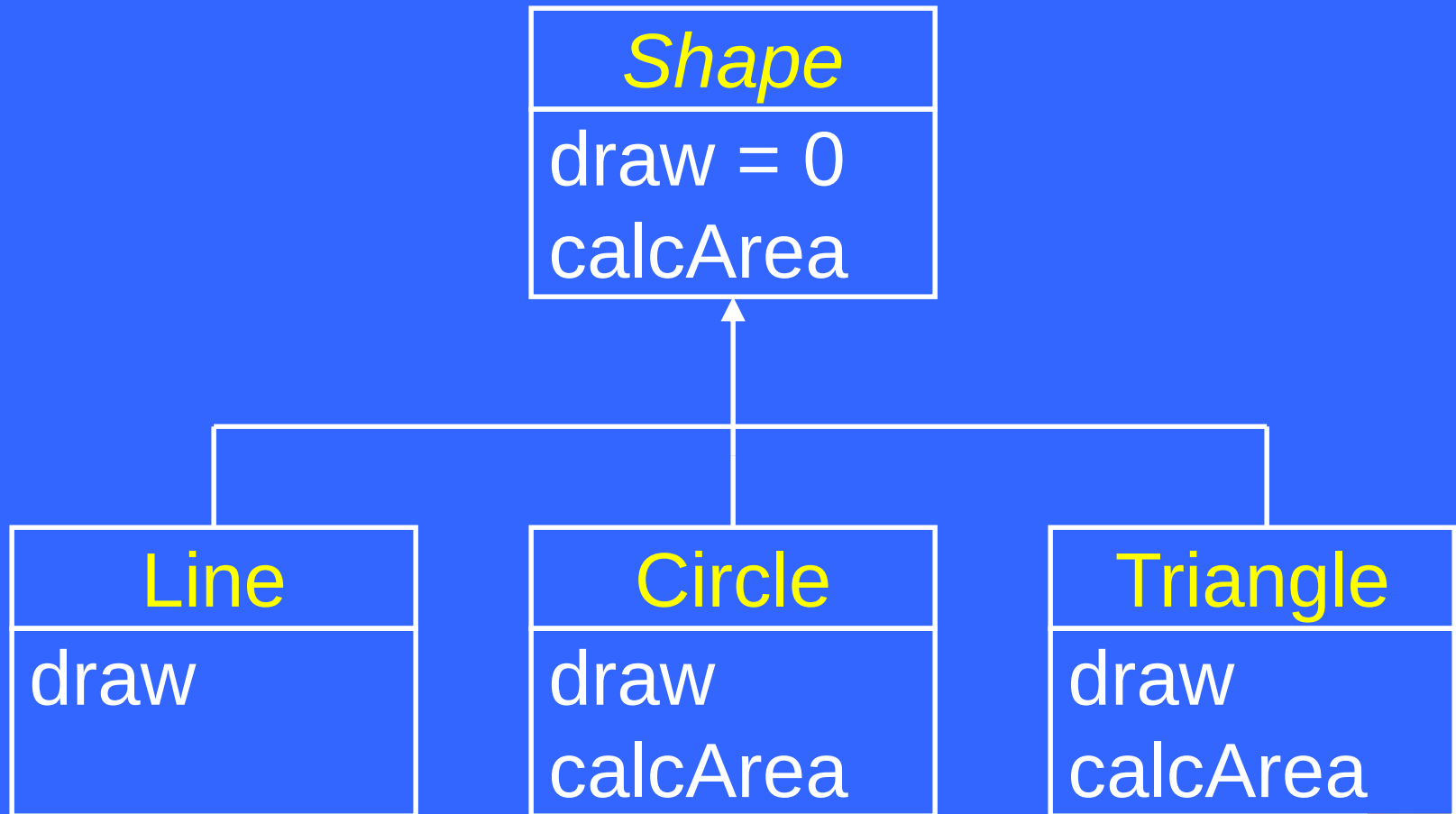
Result



Virtual Functions – Usage

- Inherit interface and implementation
- Just inherit interface (Pure Virtual)

Inherit interface and implementation



...Inherit interface and implementation

```
class Shape {  
...  
    virtual void draw() = 0;  
  
    virtual float calcArea() {  
        return 0;  
    }  
}
```

...Inherit interface and implementation

- Each derived class of `Shape` inherits default implementation of `calcArea()`
- Some may override this, such as `Circle` and `Triangle`
- Others may not, such as `Point` and `Line`

...Inherit interface and implementation

- Each derived class of `Shape` inherits interface (prototype) of `draw()`
- Each concrete derived class has to provide body of `draw()` by overriding it

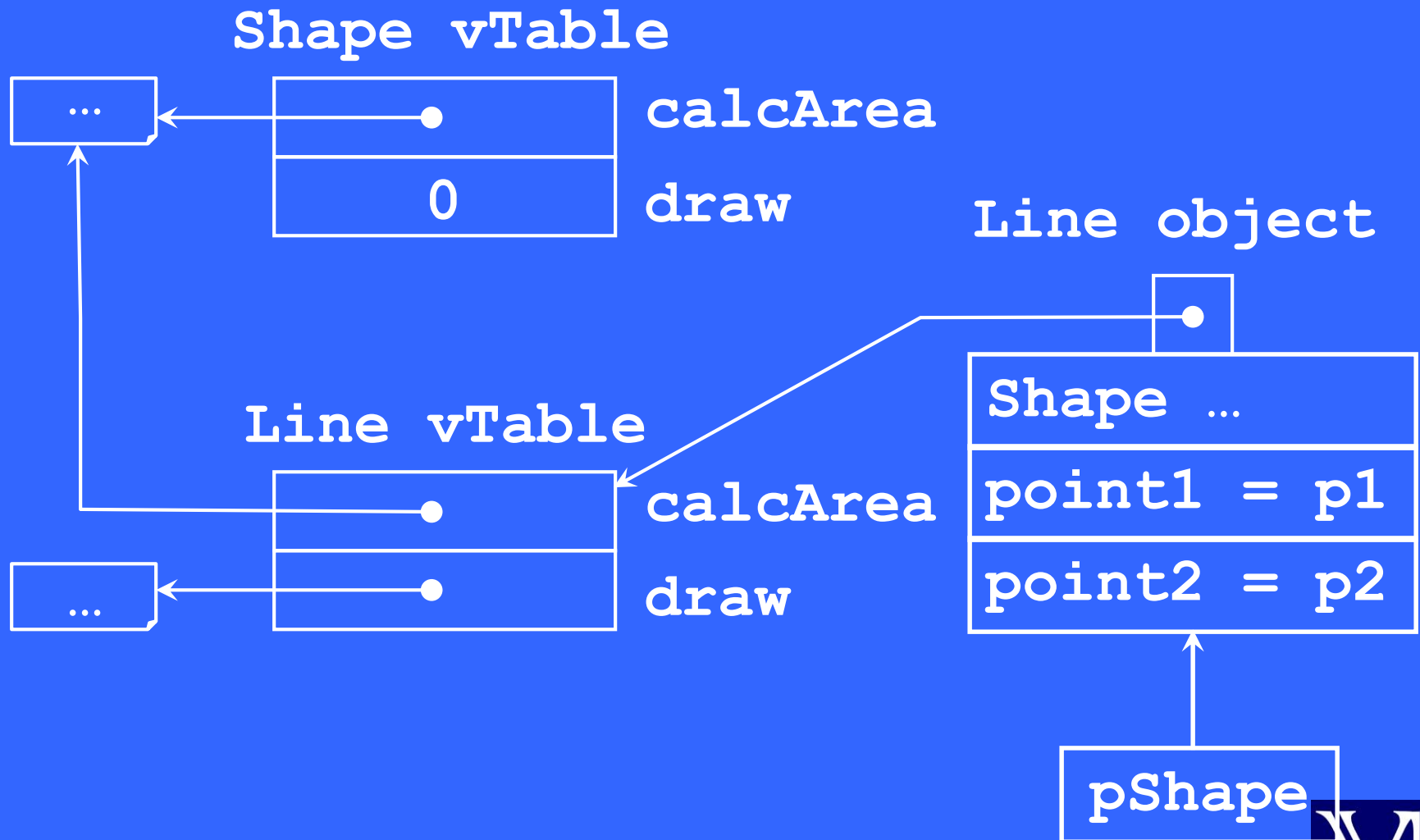
V Table

- Compiler builds a virtual function table (vTable) for each class having virtual functions
- A vTable contains a pointer for each virtual function

Example – V Table

```
int main() {  
    Point p1( 10, 10 ), p2( 30, 30 );  
    Shape* pShape;  
  
    pShape = new Line( p1, p2 );  
    pShape->draw();  
    pShape->calcArea();  
}
```


Example – V Table



Dynamic Dispatch

- For non-virtual functions, compiler just generates code to call the function
- In case of virtual functions, compiler generates code to
 - access the object
 - access the associated vTable
 - call the appropriate function

Conclusion

- Polymorphism adds
 - Memory overhead due to vTables
 - Processing overhead due to extra pointer manipulation
- However, this overhead is acceptable for many of the applications
- Moral: “Think about performance requirements before making a function virtual”

Object-Oriented Programming (OOP)

Lecture No. 30

Polymorphism – Case Study

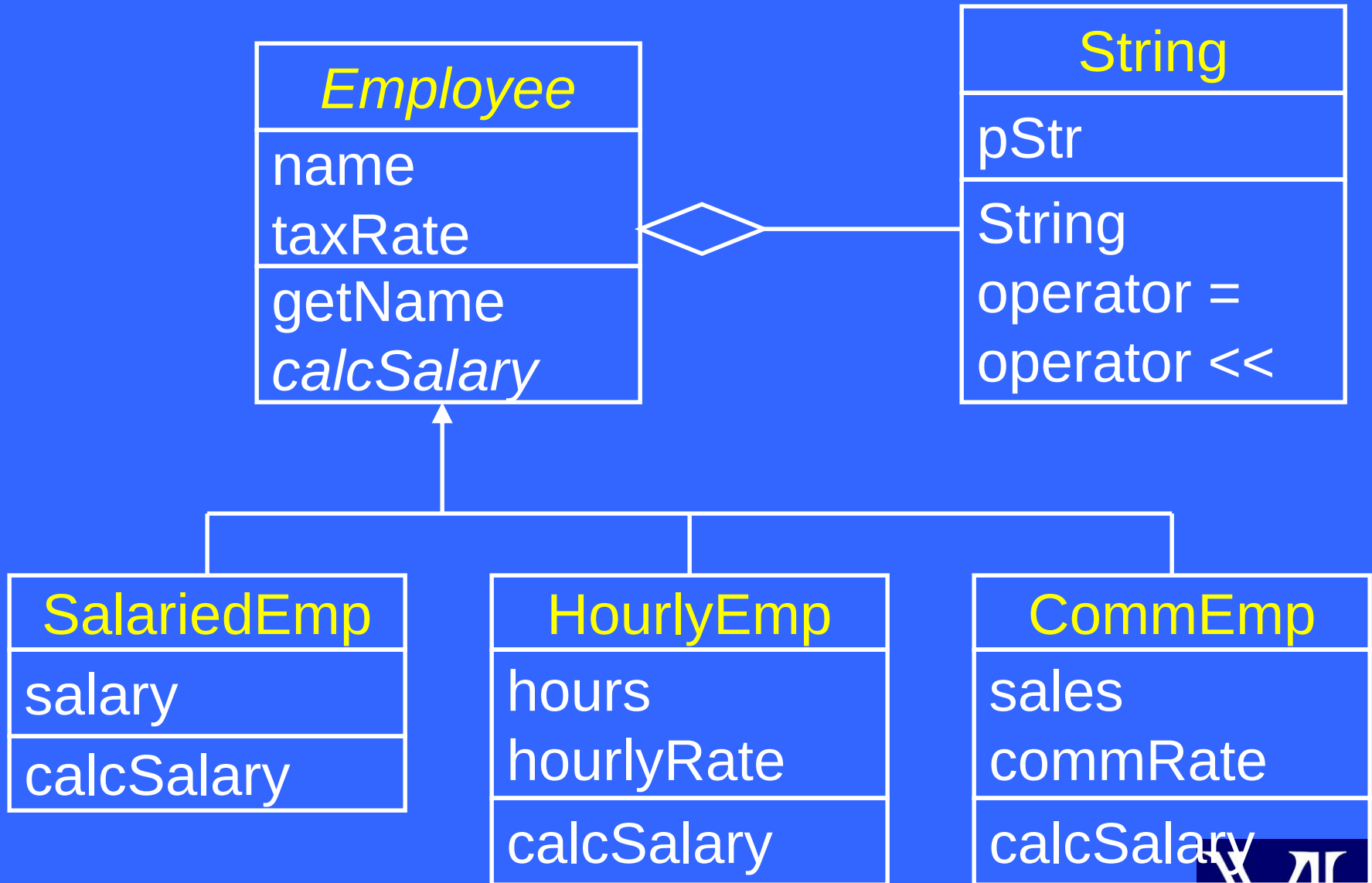
A Simple Payroll Application



Problem Statement

- Develop a simple payroll application. There are three kinds of employees in the system: salaried employee, hourly employee, and commissioned employee. The system takes as input an array containing employee objects, calculates salary polymorphically, and generates report.

OO Model



Class *Employee*

```
class Employee {  
    private:  
        String name;  
        double taxRate;  
    public:  
        Employee( String&, double );  
        String getName();  
        virtual double calcSalary() = 0;  
}
```



... Class *Employee*

```
Employee::Employee( String& n,  
                   double tr ): name(n) {  
    taxRate = tr;  
}
```

```
String Employee::getName() {  
    return name;  
}
```

Class SalariedEmp

```
class SalariedEmp : public Employee
{
private:
    double salary;
public:
    SalariedEmp(String&, double, double) ;
    virtual double calcSalary() ;
}
```



... Class SalariedEmp

```
SalariedEmp::SalariedEmp(String& n,  
                           double tr, double sal)  
    : Employee( n, tr ) {  
    salary = sal;  
}
```

```
double SalariedEmp::calcSalary() {  
    double tax = salary * taxRate;  
    return salary - tax;  
}
```



Class HourlyEmp

```
class HourlyEmp : public Employee {  
private:  
    int hours;  
    double hourlyRate;  
public:  
    HourlyEmp(string&, double, int, double) ;  
    virtual double calcSalary() ;  
}
```



... Class HourlyEmp

```
HourlyEmp ::HourlyEmp( String& n,  
    double tr, int h, double hr )  
    : Employee( n, tr ) {  
    hours = h;  
    hourlyRate = hr;  
}
```

... Class HourlyEmp

```
double HourlyEmp::calcSalary()  
{  
    double grossPay, tax;  
  
    grossPay = hours * hourlyRate;  
    tax = grossPay * taxRate;  
  
    return grossPay - tax;  
}
```

Class CommEmp

```
class CommEmp : public Employee
{
private:
    double sales;
    double commRate;
public:
    CommEmp( String&, double, double,
            double
    );
    virtual double calcSalary();
}
```



... Class CommEmp

```
CommEmp::CommEmp( String& n,  
    double tr, double s, double cr )  
    : Employee( n, tr ) {  
    sales = s;  
    commRate = cr;  
}
```


... Class CommEmp

```
double CommEmp::calcSalary()  
{  
    double grossPay = sales * commRate;  
    double tax = grossPay * taxRate;  
  
    return grossPay - tax;  
}
```

A Sample Payroll

```
int main() {  
    Employee* emp[10];  
    emp[0] = new SalariedEmp( "Aamir",  
                               0.05, 15000 );  
    emp[1] = new HourlyEmp( "Faakhir",  
                             0.06, 160, 50 );  
    emp[2] = new CommEmp( "Fuaad",  
                           0.04, 150000, 10 );  
    ...  
    generatePayroll( emp, 10 );  
    return 0;  
}
```



...A Sample Payroll

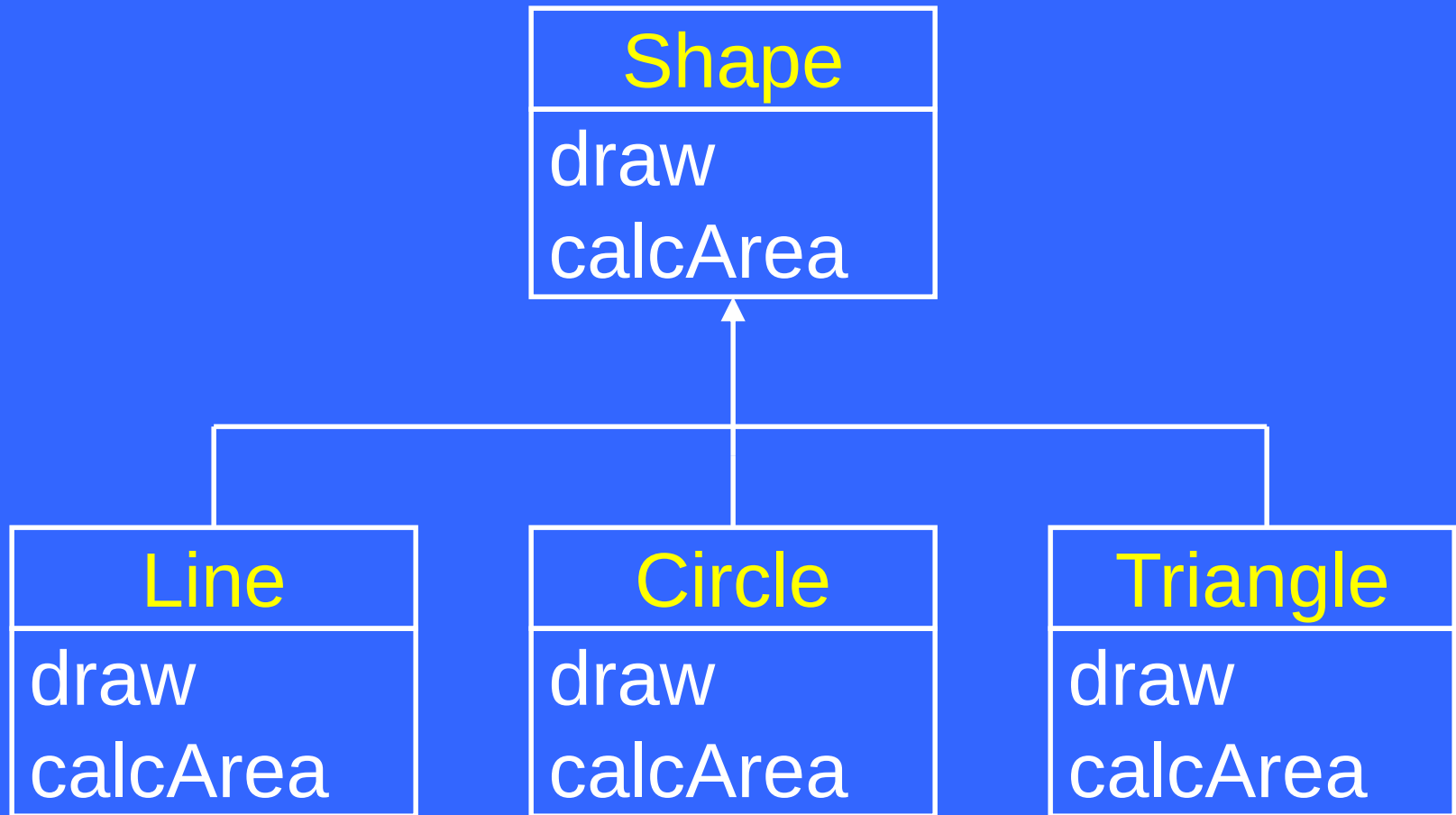
```
void generatePayroll(Employee* emp[],  
                    int size) {  
  
    cout << "Name\tNet Salary\n\n";  
  
    for (int i = 0; i < size; i++) {  
        cout << emp[i]->getName() << '\t'  
              << emp[i]->calcSalary()  
              << '\n';  
    }  
}
```

Sample Output

Name	Net Salary
Aamir	14250
Fakhir	7520
Fuaad	14400
...	

Never Treat Arrays
Polymorphically

Shape Hierarchy Revisited



Shape Hierarchy

```
class Shape {  
    ...  
public:  
    Shape();  
    virtual void draw() {  
        cout << "Shape\n";  
    }  
    virtual int calcArea() { return 0; }  
};
```



... Shape Hierarchy

```
class Line : public Shape {  
    ...  
public:  
    Line(Point p1, Point p2);  
    void draw() { cout << "Line\n"; }  
}
```


drawShapes()

```
void drawShapes( Shape _shape[],  
                int size ) {  
    for (int i = 0; i < size; i++) {  
        _shape[i].draw();  
    }  
}
```

Polymorphism & Arrays

```
int main() {  
    Shape _shape[ 10 ];  
    _shape[ 0 ] = Shape();  
    _shape[ 1 ] = Shape();  
    ...  
    drawShapes( _shape, 10 );  
    return 0;  
}
```

Sample Output

Shape

Shape

Shape

...

...Polymorphism & Arrays

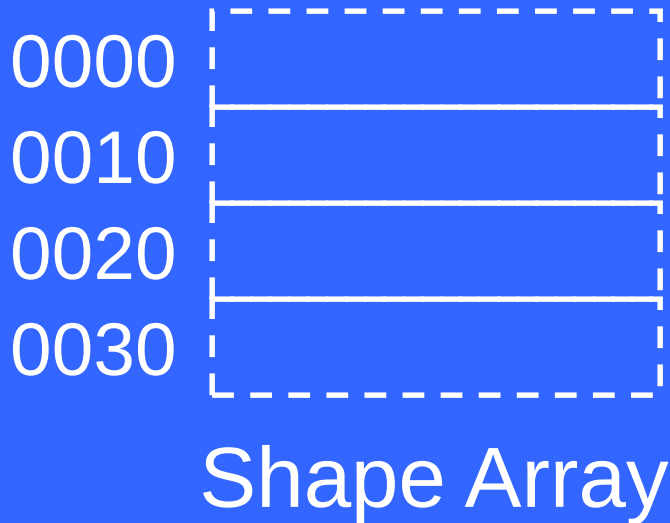
```
int main() {  
    Point p1(10, 10), p2(20, 20), ...  
    Line _line[ 10 ];  
    _line[ 0 ] = Line( p1, p2 );  
    _line[ 1 ] = Line( p3, p4 );  
    ...  
    drawShapes( _line, 10 );  
    return 0;  
}
```

Sample Output

```
Shape
```

```
// Run-time error
```

Because



```
_shape[ i ].draw();  
*( _shape + (i * sizeof(Shape)) ).draw();
```

Original drawShapes()

```
void drawShapes (Shape* _shape[],  
                 int size) {  
    for (int i = 0; i < size; i++) {  
        _shape[i]->draw();  
    }  
}
```

Sample Output

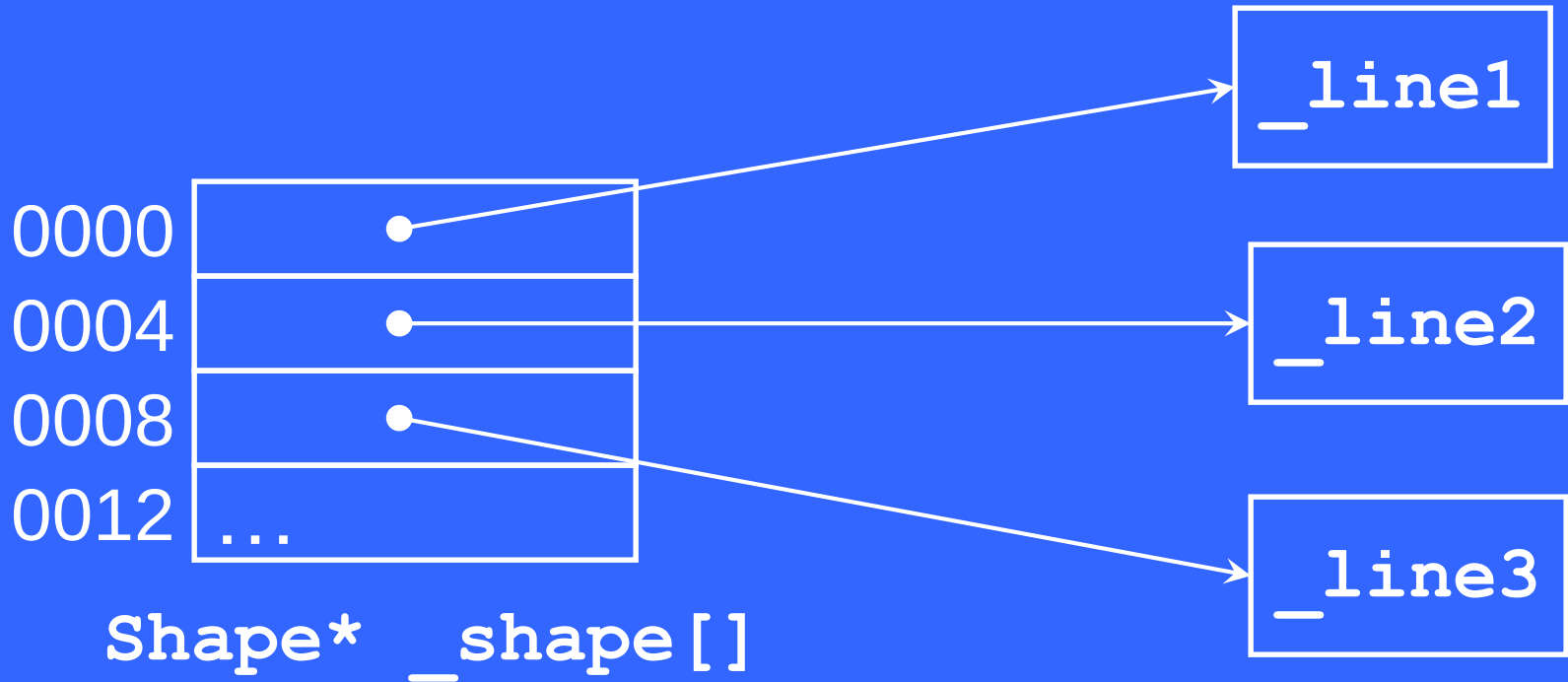
Line

Line

Line

...

Because



```
_shape[i]->draw();
```

```
(_shape + (i * sizeof(Shape*))) ->draw();
```

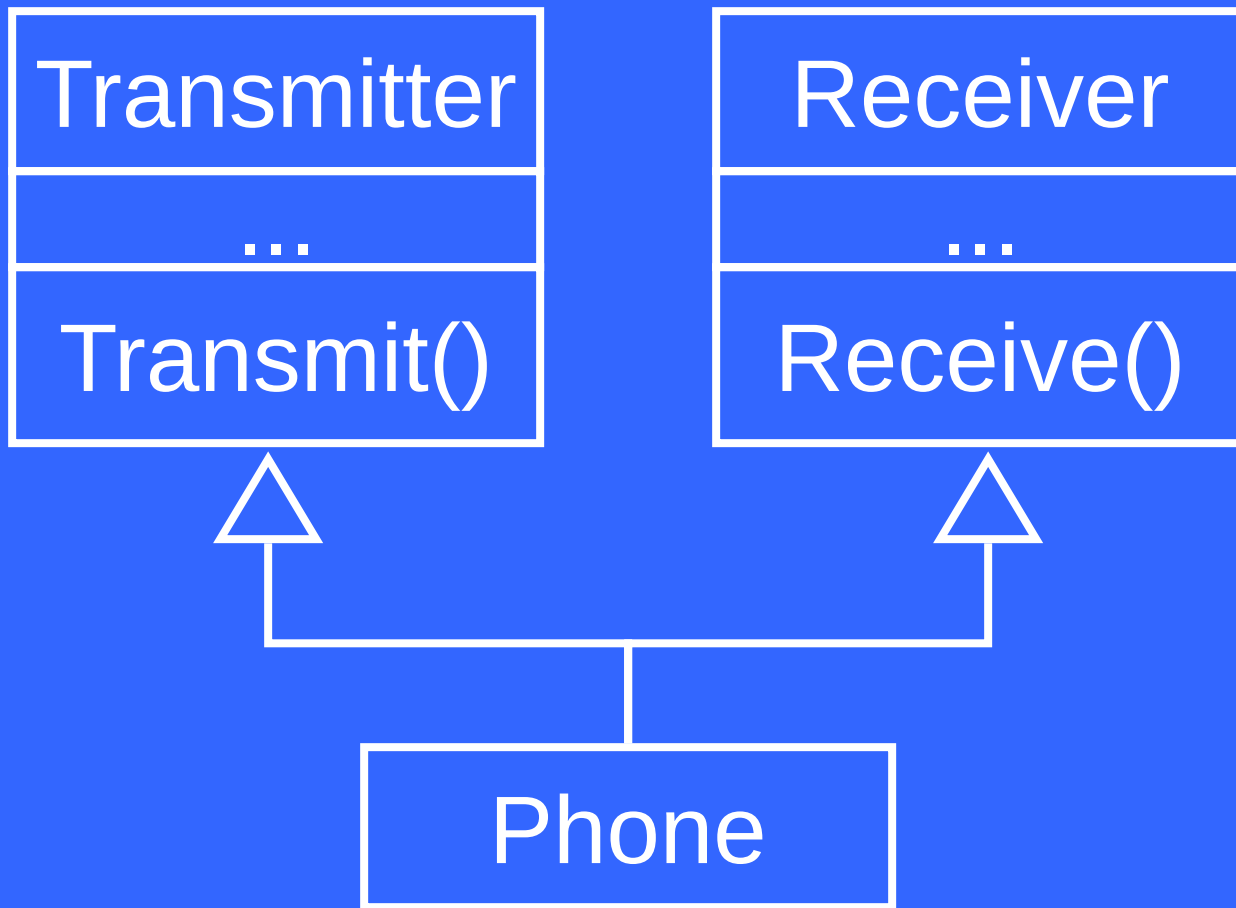
Object-Oriented Programming (OOP)

Lecture No. 31

Multiple Inheritance

- A class can inherit from more than one class

Multiple Inheritance



Example

```
class Phone:    public Transmitter,  
                public Receiver  
{  
    ...  
};
```

Multiple Inheritance

- Derived class can inherit from public base class as well as private and protected base classes

class Mermaid:

private Woman, private Fish

Multiple Inheritance

- The derived class inherits data members and functions from all the base classes
- Object of derived class can perform all the tasks that an object of base class can perform

Example

```
int main(){  
    Phone obj;  
    obj.Transmit();  
    obj.Receive();  
    return 0;  
}
```


Multiple Inheritance

- When using public multiple inheritance, the object of derived class can replace the objects of all the base classes

Example

```
int main(){  
    Phone obj;  
    Transmitter * tPtr = &obj;  
    Receiver * rPtr = &obj;  
    return 0;  
}
```

Multiple Inheritance

- The pointer of one base class cannot be used to call the function of another base class
- The functions are called based on static type

Example

```
int main(){  
    Phone obj;  
    Transmitter * tPtr = &obj;  
    tPtr->Transmit();  
    tPtr->Receive();    //Error  
    return 0;  
}
```

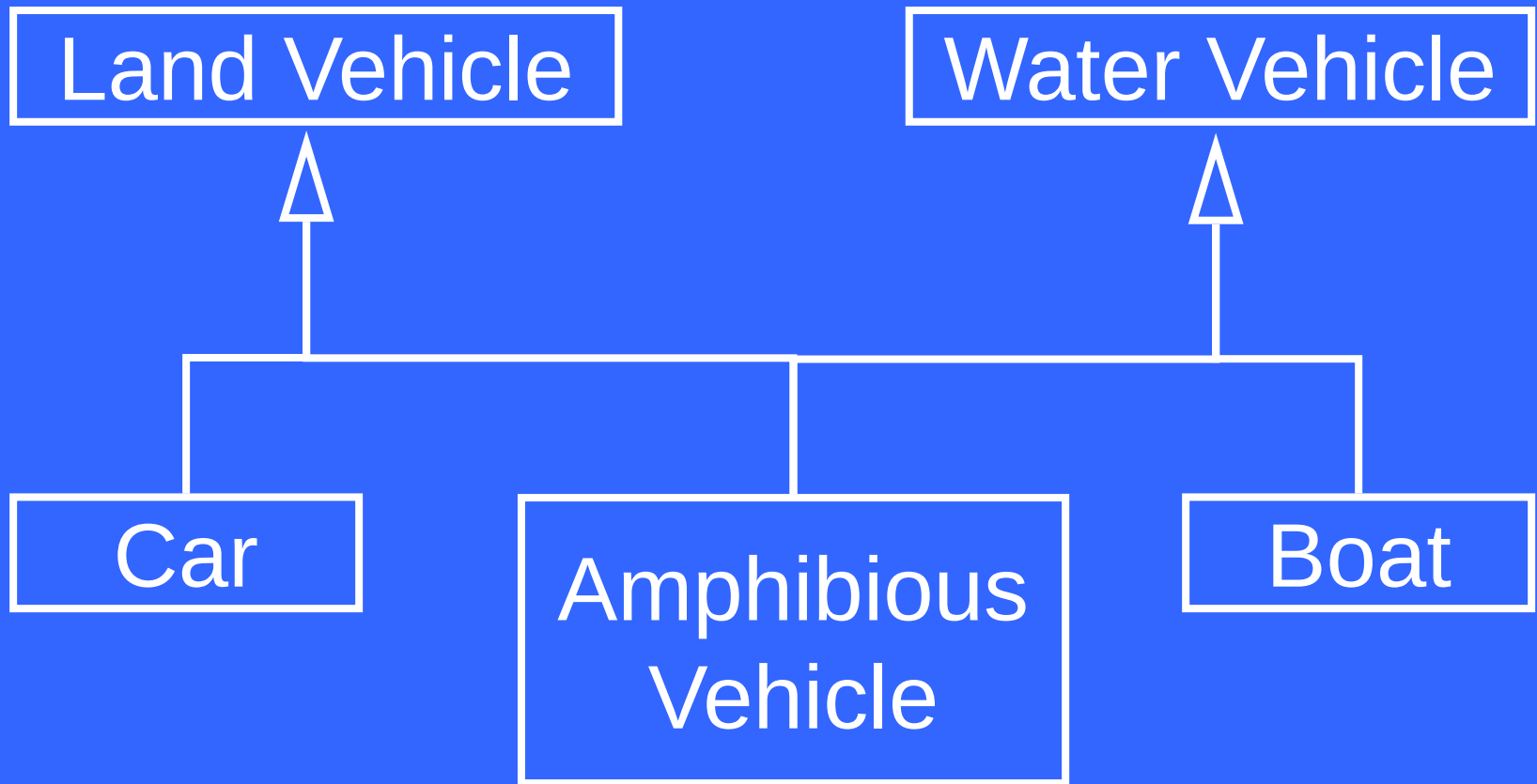
Example

```
int main(){  
    Phone obj;  
    Receiver * rPtr = &obj;  
    rPtr->Receive();  
    rPtr->Transmit();    //Error  
    return 0;  
}
```

Multiple Inheritance

- If more than one base class have a function with same signature then the child will have two copies of that function
- Calling such function will result in ambiguity

Multiple Inheritance



Example

```
class LandVehicle{  
public:  
    int GetMaxLoad();  
};  
class WaterVehicle{  
public:  
    int GetMaxLoad();  
};
```


Example

```
class AmphibiousVehicle:
    public LandVehicle,
    public WaterVehicle{
};

int main(){
    AmphibiousVehicle obj;
    obj.GetMaxLoad();           // Error
    return 0;
}
```

Multiple Inheritance

- Programmer must explicitly specify the class name when calling ambiguous function

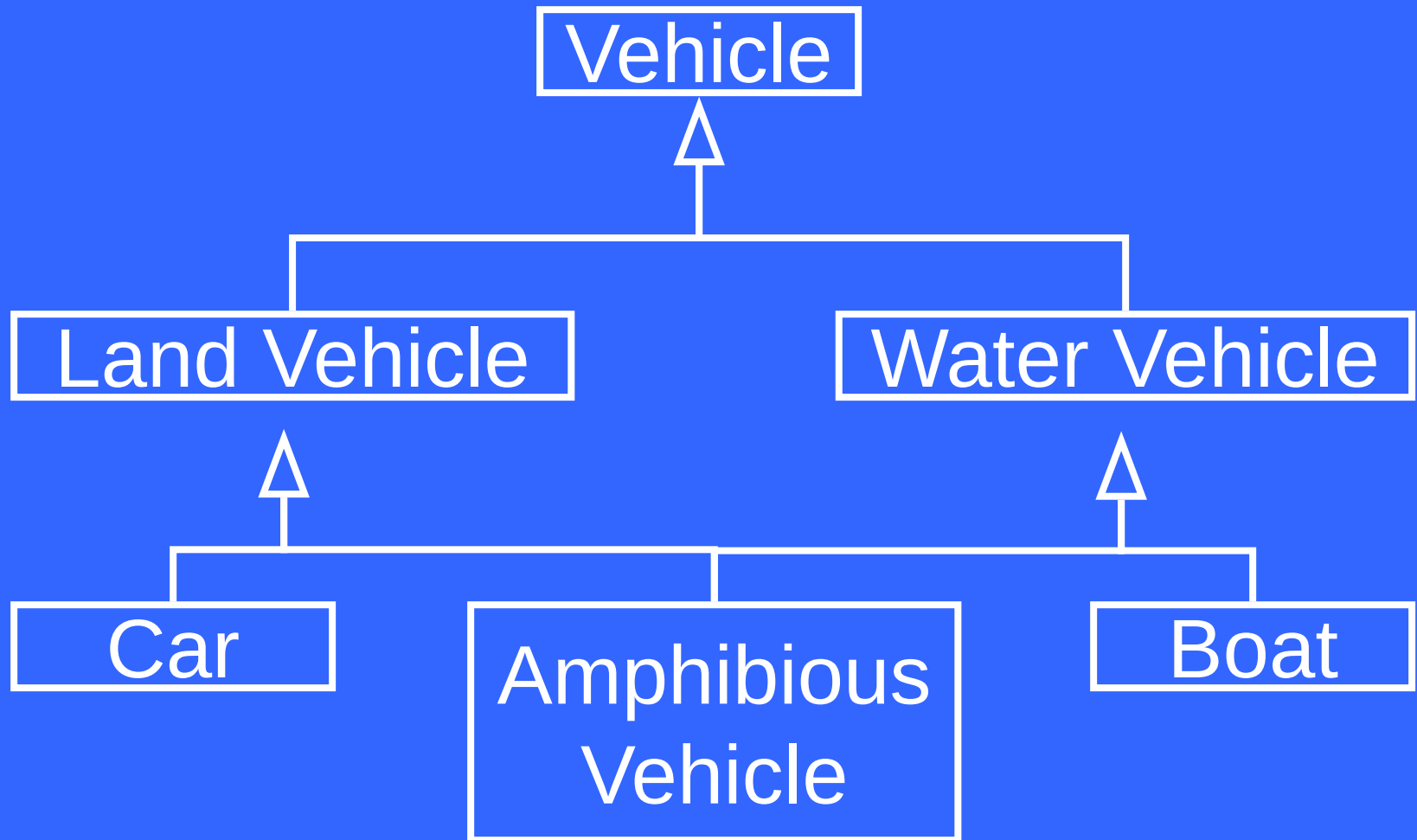
Example

```
int main(){  
    AmphibiousVehicle obj;  
    obj.LandVehicle::GetMaxLoad();  
    obj.WaterVehicle::GetMaxLoad();  
    return 0;  
}
```

Multiple Inheritance

- The ambiguous call problem can arise when dealing with multiple level of multiple inheritance

Multiple Inheritance



Example

```
class Vehicle{  
public:  
    int GetMaxLoad();  
};  
class LandVehicle : public Vehicle{  
};  
class WaterVehicle : public Vehicle{  
};
```

Example

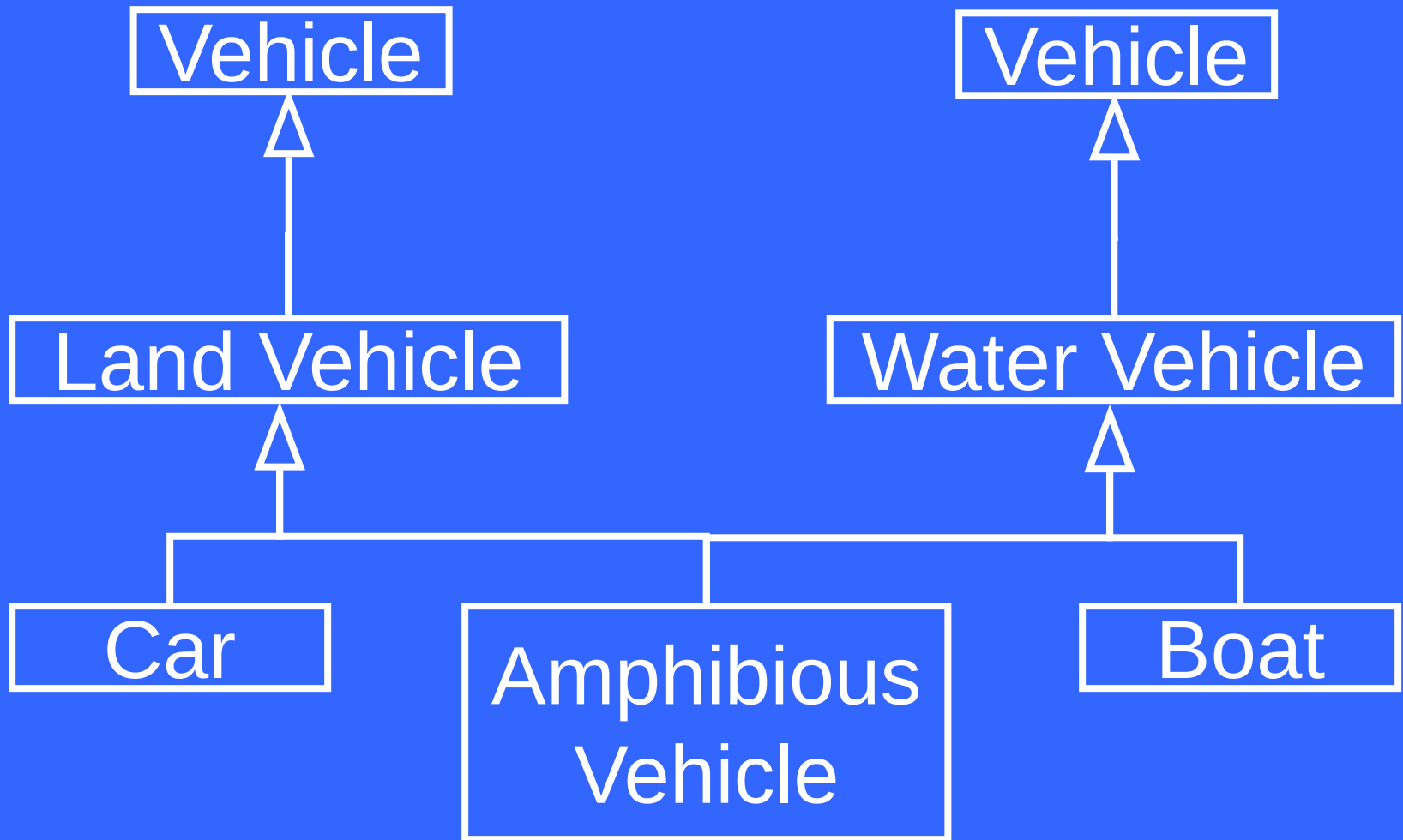
```
class AmphibiousVehicle:  
    public LandVehicle,  
    public WaterVehicle{  
};  
int main(){  
    AmphibiousVehicle obj;  
    obj.GetMaxLoad();           // Error  
    return 0;  
}
```

Example

```
int main()
{
    AmphibiousVehicle obj;
    obj.Vehicle::GetMaxLoad(); //Error
    return 0;
}
```

- Vehicle is accessible through two paths

Multiple Inheritance



Example

```
int main(){  
    AmphibiousVehicle obj;  
    obj.LandVehicle::GetMaxLoad();  
    obj.WaterVehicle::GetMaxLoad();  
    return 0;  
}
```

Multiple Inheritance

- Data member must be used with care when dealing with more than one level on inheritance

Example

```
class Vehicle{
```

```
protected:
```

```
    int weight;
```

```
};
```

```
class LandVehicle : public Vehicle{
```

```
};
```

```
class WaterVehicle : public Vehicle{
```

```
};
```

Example

```
class AmphibiousVehicle:  
    public LandVehicle,  
    public WaterVehicle{  
public:  
    AmphibiousVehicle(){  
        LandVehicle::weight = 10;  
        WaterVehicle::weight = 10;  
    }  
};
```

- There are multiple copies of data member weight

Memory View

Data Members of Vehicle	Data Members of Vehicle
Data Members of LandVehicle	Data Members of WaterVehicle
Data Members of AmphibiousVehicle	

Virtual Inheritance

- In virtual inheritance there is exactly one copy of the anonymous base class object

Example

```
class Vehicle{
protected:
    int weight;
};
class LandVehicle :
    public virtual Vehicle{
};
class WaterVehicle :
    public virtual Vehicle{
};
```


Example

```
class AmphibiousVehicle:  
    public LandVehicle,  
    public WaterVehicle{  
public:  
    AmphibiousVehicle(){  
        weight = 10;  
    }  
};
```

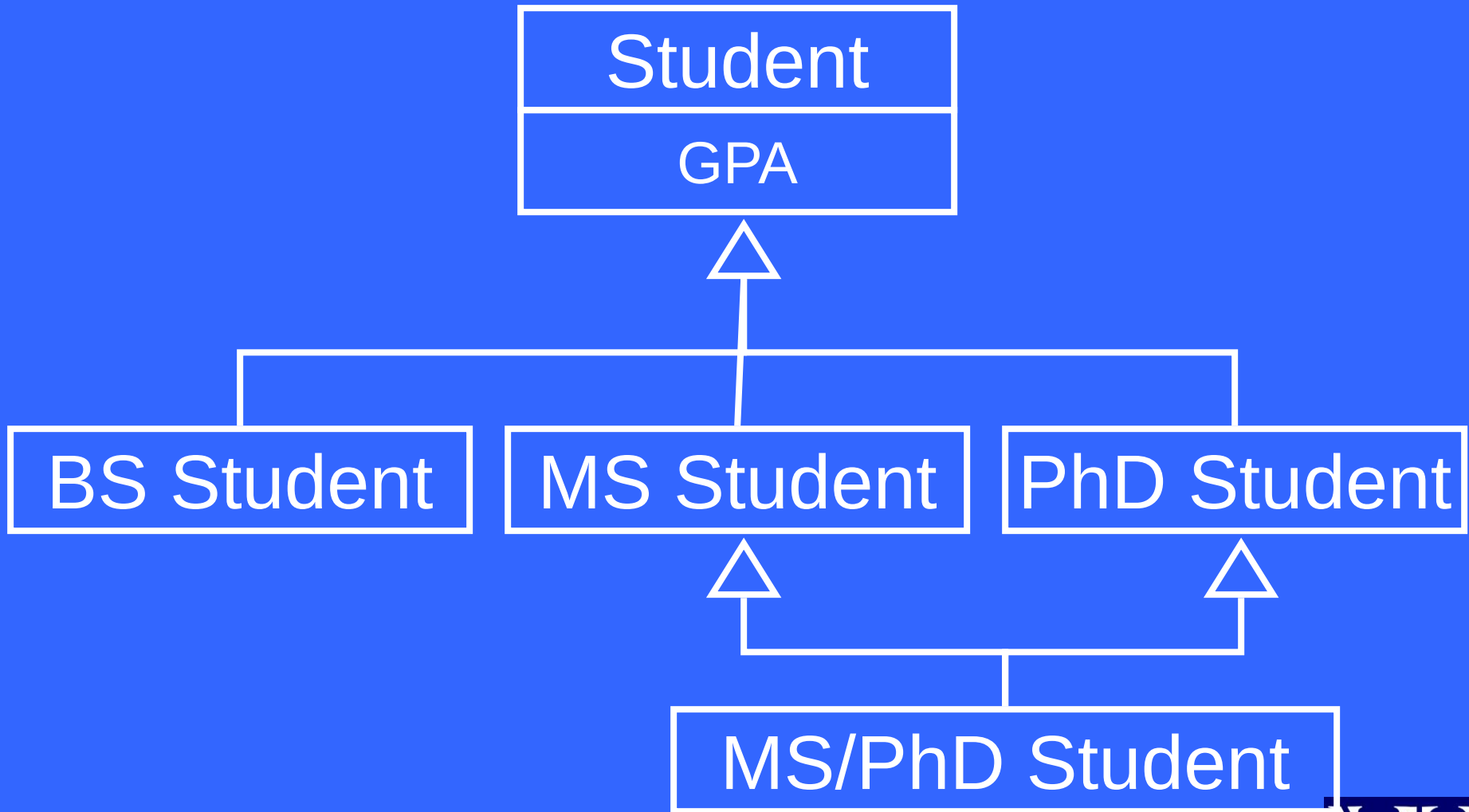
Memory View

Data Members of Vehicle	
Data Members of LandVehicle	Data Members of WaterVehicle
Data Members of AmphibiousVehicle	

Virtual Inheritance

- Virtual inheritance must be used when necessary
- There are situation when programmer would want to use two distinct data members inherited from base class rather than one

Example



Object-Oriented Programming (OOP)

Lecture No. 32

Motivation

- Following function prints an array of integer elements:

```
void printArray(int* array, int size)
{
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << ", ";
}
```

...Motivation

- What if we want to print an array of characters?

```
void printArray(char* array,  
                int size)  
{  
    for ( int i = 0; i < size; i++ )  
        cout << array[ i ] << ", ";  
}
```

...Motivation

- What if we want to print an array of doubles?

```
void printArray(double* array,  
                int size)  
{  
    for ( int i = 0; i < size; i++ )  
        cout << array[ i ] << ", ";  
}
```


...Motivation

- Now if we want to change the way function prints the array. e.g. from

1, 2, 3, 4, 5

to

1 - 2 - 3 - 4 - 5

...Motivation

- Now consider the `Array` class that wraps an array of integers

```
class Array {  
    int* pArray;  
    int size;  
public:  
    ...  
};
```

...Motivation

- What if we want to use an **Array** class that wraps arrays of double?

```
class Array {  
    double* pArray;  
    int size;  
public:  
    ...  
};
```

...Motivation

- What if we want to use an **Array** class that wraps arrays of boolean variables?

```
class Array {  
    bool* pArray;  
    int size;  
public:  
    ...  
};
```

...Motivation

- Now if we want to add a function `sum` to `Array` class, we have to change all the three classes

Generic Programming

- Generic programming refers to programs containing generic abstractions
- A generic program abstraction (function, class) can be parameterized with a type
- Such abstractions can work with many different types of data

Advantages

- Reusability
- Writability
- Maintainability

Templates

- In C++ generic programming is done using templates
- Two kinds
 - Function Templates
 - Class Templates
- Compiler generates different type-specific copies from a single template

Function Templates

- A function template can be parameterized to operate on different types of data



Declaration

```
template< class T >  
void funName( T x );  
// OR
```

```
template< typename T >  
void funName( T x );  
// OR
```

```
template< class T, class U, ... >  
void funName( T x, U y, ... );
```



Example – Function Templates

- Following function template prints an array having almost any type of elements:

```
template< typename T >
void printArray( T* array, int size )
{
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << ", ";
}
```

...Example – Function Templates

```
int main() {  
    int iArray[5] = { 1, 2, 3, 4, 5 };  
    void printArray( iArray, 5 );  
        // Instantiated for int[]  
  
    char cArray[3] = { 'a', 'b', 'c' };  
    void printArray( cArray, 3 );  
        // Instantiated for char[]  
    return 0;  
}
```

Explicit Type Parameterization

- A function template may not have any parameter

```
template <typename T>
T getInput() {
    T x;
    cin >> x;
    return x;
}
```

...Explicit Type Parameterization

```
int main() {  
    int x;  
    x = getInput();           // Error!  
  
    double y;  
    y = getInput();           // Error!  
}
```

...Explicit Type Parameterization

```
int main() {  
    int x;  
    x = getInput< int >();  
  
    double y;  
    y = getInput< double >();  
}
```

User-defined Specializations

- A template may not handle all the types successfully
- Explicit specializations need to be provided for specific type(s)

Example – User Specializations

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

... Example – User Specializations

```
int main {  
    isEqual( 5, 6 );    // OK  
    isEqual( 7.5, 7.5 );    // OK  
    isEqual( "abc", "xyz" );  
        // Logical Error!  
    return 0;  
}
```

... Example – User Specializations

```
template< >
bool isEqual< const char* >(
    const char* x, const char* y ) {
    return ( strcmp( x, y ) == 0 );
}
```

... Example – User Specializations

```
int main {  
    isEqual( 5, 6 );  
    // Target: general template  
    isEqual( 7.5, 7.5 );  
    // Target: general template  
  
    isEqual( "abc", "xyz" );  
    // Target: user specialization  
    return 0;  
}
```

Object-Oriented Programming (OOP)

Lecture No. 33

Recap

- Templates are generic abstractions
- C++ templates are of two kinds
 - Function Templates
 - Class Templates
- A general template can be specialized to specifically handle a particular type

Multiple Type Arguments

```
template< typename T, typename U >
T my_cast( U u ) {
    return (T)u;
}
```

```
int main() {
    double d = 10.5674;
    int j = my_cast( d );    //Error
    int i = my_cast< int >( d );
    return 0;
}
```



User-Defined Types

- Besides primitive types, user-defined types can also be passed as type arguments to templates
- Compiler performs static type checking to diagnose type errors

...User-Defined Types

- Consider the String class without overloaded operator “==”

```
class String {  
    char* pStr;  
  
    ...  
    // Operator “==” not defined  
};
```

... User-Defined Types

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

```
int main() {
    String s1 = "xyz", s2 = "xyz";
    isEqual( s1, s2 ); // Error!
    return 0;
}
```



...User-Defined Types

```
class String {  
    char* pStr;  
    ...  
    friend bool operator ==(  
        const String&, const String& );  
};
```



... User-Defined Types

```
bool operator ==( const String& x,  
                  const String& y  
    ) {  
    return strcmp(x.pStr, y.pStr) == 0;  
}
```



... User-Defined Types

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

```
int main() {
    String s1 = "xyz", s2 = "xyz";
    isEqual( s1, s2 ); // OK
    return 0;
}
```



Overloading vs Templates

- Different data types, similar operation
 - Needs function overloading
- Different data types, identical operation
 - Needs function templates

Example

Overloading vs Templates

- '+' operation is overloaded for different operand types
- A single function template can calculate sum of array of many types

...Example

Overloading vs Templates

```
String operator +( const String& x,  
                  const String& y ) {  
    String tmp;  
    tmp.pStr = new char[strlen(x.pStr) +  
                        strlen(y.pStr) + 1 ];  
    strcpy( tmp.pStr, x.pStr );  
    strcat( tmp.pStr, y.pStr );  
    return tmp;  
}
```


...Example

Overloading vs Templates

```
String operator +( const char * str1,  
                  const String& y ) {  
    String tmp;  
    tmp.pStr = new char[ strlen(str1) +  
                        strlen(y.pStr) + 1 ];  
    strcpy( tmp.pStr, str1 );  
    strcat( tmp.pStr, y.pStr );  
    return tmp;  
}
```

...Example

Overloading vs Templates

```
template< class T >
T sum( T* array, int size ) {
    T sum = 0;

    for (int i = 0; i < size; i++)
        sum = sum + array[i];

    return sum;
}
```



Template Arguments as Policy

- Policy specializes a template for an operation (behavior)



Example – Policy

- Write a function that compares two given character strings
- Function can perform either case-sensitive or non-case sensitive comparison

First Solution

```
int caseSencompare( char* str1,
                    char* str2 )
{
    for (int i = 0; i < strlen( str1 )
          && i < strlen( str2 );
        ++i)
        if ( str1[i] != str2[i] )
            return str1[i] - str2[i];

    return strlen(str1) - strlen(str2);
}
```



...First Solution

```
int nonCaseSencompare( char* str1,
                        char* str2 )
{
    for (int i = 0; i < strlen( str1 )
        && i < strlen( str2 );
        i++)
        if ( toupper( str1[i] ) !=
            toupper( str2[i] ) )
            return str1[i] - str2[i];

    return strlen(str1) - strlen(str2);
}
```



Second Solution

```
int compare( char* str1, char* str2,
             bool caseSen
)
{
    for (int i = 0; i < strlen( str1 )
          && i < strlen( str2 );
        i++)
        if ( ... )
            return str1[i] - str2[i];

    return strlen(str1) - strlen(str2);
}
```



...Second Solution

```
// if condition:
```

```
(caseSen && str1[i] != str2[i])  
    || (!caseSen &&  
        toupper(str1[i])  
    !=  
        toupper(str2[i]))
```


Third Solution

```
class CaseSenCmp {  
public:  
    static int isEqual( char x, char y )  
    {  
        return x == y;  
    }  
};
```



... Third Solution

```
class NonCaseSenCmp {  
public:  
    static int isEqual( char x, char y )  
    {  
        return toupper(x) == toupper(y) ;  
    }  
};
```

...Third Solution

```
template< typename C >
int compare( char* str1, char* str2 )
{
    for (int i = 0; i < strlen( str1 )
          && i < strlen( str2 );
         i++)
        if ( !C::isEqual
              (str1[i], str2[i]) )
            return str1[i] - str2[i];

    return strlen(str1) - strlen(str2);
}
```



...Third Solution

```
int main() {  
    int i, j;  
    char *x = "hello", *y = "HELLO";  
    i = compare< CaseSenCmp >(x, y);  
    j = compare< NonCaseSenCmp >(x, y);  
    cout << "Case Sensitive: " << i;  
    cout << "\nNon-Case Sensitive: "  
          << j << endl;  
    return 0;  
}
```



Sample Output

Case Sensitive: 32 // Not Equal

Non-case Sensitive: 0 // Equal

Default Policy

```
template< typename C = CaseSenCmp >
int compare( char* str1, char* str2 )
{
    for (int i = 0; i < strlen( str1 )
        && i < strlen( str2 );
        i++)
        if ( !C::isEqual
            (str1[i], str2[i]) )
            return str1[i] - str2[i];

    return strlen(str1) - strlen(str2);
}
```



...Third Solution

```
int main() {  
    int i, j;  
    char *x = "hello", *y = "HELLO";  
    i = compare(x, y);  
    j = compare< NonCaseSenCmp >(x, y);  
    cout << "Case Sensitive: " << i;  
    cout << "\nNon-Case Sensitive: "  
          << j << endl;  
    return 0;  
}
```



Object-Oriented Programming (OOP)

Lecture No. 34

Generic Algorithms

A Case Study

Print an Array

```
template< typename T >
void printArray( T* array, int size )
{
    for ( int i = 0; i < size; i++ )
        cout << array[ i ] << ", ";
}
```

Generic Algorithms

```
const int* find( const int* array,
                  int _size, int x
                ) {
    const int* p = array;
    for (int i = 0; i < _size; i++) {
        if ( *p == x )
            return p;
        p++;
    }
    return 0;
}
```

...Generic Algorithms

```
template< typename T >
T* find( T* array, int _size,
        const T& x ) {
    T* p = array;
    for (int i = 0; i < _size; i++) {
        if ( *p == x )
            return p;
        p++;
    }
    return 0;
}
```

...Generic Algorithms

```
template< typename T >
T* find( T* array, T* beyond,
        const T& x ) {
    T* p = array;
    while ( p != beyond ) {
        if ( *p == x )
            return p;
        p++;
    }
    return 0;
}
```

...Generic Algorithms

```
template< typename T >
T* find( T* array, T* beyond,
        const T& x ) {
    T* p = array;
    while ( p != beyond ) {
        if ( *p == x )
            return p;
        p++;
    }
    return beyond;
}
```

...Generic Algorithms

```
template< typename T >
T* find( T* array, T* beyond,
        const T& x ) {
    T* p = array;
    while ( p != beyond && *p != x )
        p++;
    return p;
}
```

...Generic Algorithms

- This algorithm works fine for arrays of any type
- We can make it work for any generic container that supports two operations
 - Increment operator (++)
 - Dereference operator (*)

...Generic Algorithms

```
template< typename P, typename T >
P find( P start, P beyond,
        const T& x ) {
    while ( start != beyond &&
            *start != x )
        start++;
    return start;
}
```

...Generic Algorithms

```
int main() {  
    int iArray[5];  
    iArray[0] = 15;  
    iArray[1] = 7;  
    iArray[2] = 987;  
    ...  
    int* found;  
    found = find(iArray, iArray + 5, 7);  
    return 0;  
}
```

Class Templates

- A single class template provides functionality to operate on different types of data
- Facilitates reuse of classes
- Definition of a class template follows
 - `template< class T > class XYZ { ... }; or`
 - `template< typename T > class XYZ { ... };`



Example – Class Template

- A **Vector** class template can store data elements of different types
- Without templates, we need a separate **Vector** class for each data type

...Example – Class Template

```
template< class T >
class Vector {
private:
    int size;
    T* ptr;
public:
    Vector<T>( int = 10 );
    Vector<T>( const Vector< T >& );
    ~Vector<T>();
    int getSize() const;
```



...Example – Class Template

```
const Vector< T >& operator =(  
    const Vector< T >& );  
T& operator [] ( int );  
};
```

...Example – Class Template

```
template< class T >
Vector<T>::Vector<T>( int s ) {
    size = s;
    if ( size != 0 )
        ptr = new T[size];
    else
        ptr = 0;
}
```

...Example – Class Template

```
template< class T >
Vector<T>:: Vector<T>(
    const Vector<T>& copy ) {
    size = copy.getSize();
    if (size != 0) {
        ptr = new T[size];
        for (int i = 0; i < size; i++)
            ptr[i] = copy.ptr[i];
    }
    else ptr = 0;
}
```



...Example – Class Template

```
template< class T >
Vector<T>::~~Vector<T>() {
    delete [] ptr;
}
```

```
template< class T >
int Vector<T>::getSize() const {
    return size;
}
```



...Example – Class Template

```
template< class T >
const Vector<T>& Vector<T>::operator
    =( const Vector<T>& right) {
    if ( this != &right ) {
        delete [] ptr;
        size = right.size;
```

...Example – Class Template

```
if ( size != 0 ) {  
    ptr = new T[size];  
    for(int i = 0; i < size;i++)  
        ptr[i] = right.ptr[i];  
}  
else  
    ptr = 0;  
}  
return *this;  
}
```

...Example – Class Template

```
template< class T >
T& Vector< T >::operator [](
                                int index ) {
    if ( index < 0 || index >= size ) {
        cout << "Error: index out of
                                   range\n";
        exit( 1 );
    }
    return ptr[index];
}
```



...Example – Class Template

- A customization of above class template can be instantiated as

```
Vector< int > intVector;
```

```
...
```

```
Vector< char > charVector;
```

Object-Oriented Programming (OOP)

Lecture No. 35

Member Templates

- A class or class template can have member functions that are themselves templates

...Member Templates

```
template<typename T> class Complex {  
    T real, imag;  
public:  
    // Complex<T>( T r, T im )  
    Complex( T r, T im ) :  
        real(r), imag(im) {}  
    // Complex<T>(const Complex<T>& c)  
    Complex(const Complex<T>& c) :  
        real( c.real ), imag( c.imag ) {}  
    ...  
};
```


...Member Templates

```
int main() {  
    Complex< float > fc( 0, 0 );  
    Complex< double > dc = fc; // Error  
    return 0;  
}
```

Because

```
class Complex<double> {  
    double real, imag;  
public:  
    Complex( double r, double im ) :  
        real(r), imag(im) {}  
    Complex(const Complex<double>& c) :  
        real( c.real ), imag( c.imag ) {}  
    ...  
};
```

...Member Templates

```
template<typename T> class Complex {  
    T real, imag;  
public:  
    Complex( T r, T im ) :  
        real(r), imag(im) {}  
    template <typename U>  
    Complex(const Complex<U>& c) :  
        real( c.real ), imag( c.imag ) {}  
    ...  
};
```



...Member Templates

```
int main() {  
    Complex< float > fc( 0, 0 );  
    Complex< double > dc = fc; // OK  
    return 0;  
}
```

Because

```
class Complex<double> {  
    double real, imag;  
public:  
    Complex( double r, double im ) :  
        real(r), imag(im) {}  
    template <typename U>  
    Complex(const Complex<U>& c) :  
        real( c.real ), imag( c.imag ) {}  
    ...  
};
```



<float> Instantiation

```
class Complex<float> {  
    float real, imag;  
public:  
    Complex( float r, float im ) :  
        real(r), imag(im) {}  
    // No Copy Constructor  
    ...  
};
```

Class Template Specialization

- Like function templates, a class template may not handle all the types successfully
- Explicit specializations are provided to handle such types

...Class Template Specialization

```
int main() {  
    Vector< int > iv1( 2 );  
    iv1[0] = 15;  
    iv1[1] = 27;  
    Vector< int > iv2( iv1 );  
    Vector< int > iv3( 2 );  
    iv3 = iv1;  
    return 0;  
}
```



...Class Template Specialization

```
int main() {  
    Vector< char* > sv1( 2 );  
    sv1[0] = "Aamir";  
    sv1[1] = "Nasir";  
    Vector< char* > sv2( sv1 );  
    Vector< char* > sv3( 2 );  
    sv3 = sv1;  
    return 0;  
}
```



...Class Template Specialization

```
template<>
class Vector< char* > {
private:
    int size;
    char** ptr;
public:
    // Vector< char* >( int = 10 );
    Vector( int = 10 );
    Vector( const Vector< char* >& );
    virtual ~Vector();
```



...Class Template Specialization

```
int getSize() const;  
const Vector< char* >& operator =(  
    const Vector< char* >&  
);  
const char*& operator [] ( int );  
void insert( char*, int );  
};
```

...Class Template Specialization

```
template<>
Vector<char*>::Vector(int s) {
    size = s;
    if ( size != 0 ) {
        ptr = new char*[size];
        for (int i = 0; i < size; i++)
            ptr[i] = 0;
    }
    else
        ptr = 0;
}
```



...Class Template Specialization

```
template<>
Vector< char* >::Vector(
    const Vector<char*>& copy ) {

    size = copy.getSize();
    if ( size == 0 ) {
        ptr = 0;
        return;
    }
}
```

...Class Template Specialization

```
ptr = new char*[size];
for (int i = 0; i < size; i++)
    if ( copy.ptr[i] != 0 ) {
        ptr[i] = new char[ strlen(
                                copy.ptr[i] ) +
1 ];
        strcpy(ptr[i], copy.ptr[i]);
    }
    else
        ptr[i] = 0;
}
```



...Class Template Specialization

```
template<>
Vector<char*>::~~Vector() {
    for (int i = 0; i < size; i++)
        delete [] ptr[i];

    delete [] ptr;
}
```

...Class Template Specialization

```
template<>
int Vector<char*>::getSize() const {
    return size;
}
```


...Class Template Specialization

```
template<>
const Vector<char*>& Vector<char*>::
operator=(const Vector<char*>& right)
{
    if ( this == &right )
        return *this;
    for (int i = 0; i < size; i++)
        delete [] ptr[i];
    delete [] ptr;
```



...Class Template Specialization

```
size = right.size;  
if ( size == 0 ) {  
    ptr = 0;  
    return *this;  
}  
ptr = new char*[size];
```

...Class Template Specialization

```
for (int i = 0; i < size; i++)  
    if ( right.ptr[i] != 0 ) {  
        ptr[i] = new char[strlen(  
                                right.ptr[i] ) + 1];  
        strcpy( ptr[i], right.ptr[i] );  
    }  
    else  
        ptr[i] = 0;  
}
```



...Class Template Specialization

```
template<>
const char*& Vector<char*>::
    operator [] ( int index ) {
    if ( index < 0 || index >= size ) {
        cout << "Error: index out of
                                   range\n";
        exit( 1 );
    }
    return ptr[index];
}
```



...Class Template Specialization

```
template<>
void Vector< char* >::insert(
                        char* str, int i ) {
    delete [] ptr[i];
    if ( str != 0 ) {
        ptr[i] = new char[strlen(str)+1];
        strcpy( ptr[i], str );
    }
    else
        ptr[i] = 0;
}
```



...Class Template Specialization

```
int main() {  
    Vector< char* > sv1( 2 );  
    sv1[0] = "Aamir"; // Error  
    sv1.insert( "Aamir", 0 );  
    sv1.insert( "Nasir", 1 );  
    Vector< char* > sv2( sv1 );  
    Vector< char* > sv3( 2 );  
    sv3 = sv1;  
    return 0;  
}
```



Object-Oriented Programming (OOP)

Lecture No. 36

Recap – Member Templates

- A class template may have member templates
- They can be parameterized independent of the class template



Recap –Template Specializations

- A class template may not handle all the types successfully
- Explicit specializations are required to deal such types

Member Templates Revisited

- An ordinary class can also have member templates

```
class ComplexSet {  
...  
    template< class T >  
    insert( Complex< T > c )  
    { // Add "c" to the set }  
};
```

... Member Templates Revisited

```
int main() {  
    Complex< int > ic( 10, 5 );  
    Complex< float > fc( 10.5, 5.7 );  
    Complex< double > dc( 9.567898, 5 );  
    ComplexSet cs;  
    cs.insert( ic );  
    cs.insert( fc );  
    cs.insert( dc );  
    return 0;  
}
```



Partial Specialization

- A partial specialization of a template provides more information about the type of template arguments than that of template
- The number of template arguments remains the same

Example – Partial Specialization

```
template< class T >  
class Vector { };
```

```
template< class T >  
class Vector< T* > { };
```

Example – Partial Specialization

```
template< class T, class U, class V >  
class A {};
```

```
template< class T, class V >  
class A< T, T*, V > {};
```

```
template< class T, class U, int I >  
class A< T, U, I > {};
```

```
template< class T >  
class A< int, T*, 5 > {};
```



Example – Complete Specialization

```
template< class T >  
class Vector { };
```

```
template< >  
class Vector< char* > { };
```

Example – Complete Specialization

```
template< class T, class U, class V >  
class A {};
```

```
template< >  
class A< int, char*, double > {};
```



Function Templates

- A function template may also have partial specializations

Example – Partial Specialization

```
template< class T, class U, class V >  
void func( T, U, V );
```

```
template< class T, class V >  
void func( T, T*, V );
```

```
template< class T, class U, int I >  
void func( T, U );
```

```
template< class T >  
void func( int, T, 7 );
```



Example

- Consider the following template

```
template< typename T >  
bool isEqual( T x, T y ) {  
    return ( x == y );  
}
```

Complete Specialization

- We have already used this complete specialization

```
template< >
bool isEqual< const char* >(
    const char* x, const char* y ) {
    return ( strcmp( x, y ) == 0 );
}
```

Partial Specialization

- Following partial specialization deals with pointers to objects

```
template< typename T >
bool isEqual( T* x, T* y ) {
    return ( *x == *y );
}
```

Using Different Specializations

```
int main() {  
    int i, j;  
    char* a, b;  
    Shape *s1 = new Line();  
    Shape *s2 = new Circle();  
    isEqual( i, j );    // Template  
    isEqual( a, b );    // Complete Sp.  
    isEqual( s1, s2 ); // Partial Sp.  
    return 0;  
}
```



Non-type Parameters

- Template parameters may include non-type parameters
- The non-type parameters may have default values
- They are treated as constants
- Common use is static memory allocation

Example – Non-type Parameters

```
template< class T >
class Array {
private:
    T* ptr;
public:
    Array( int size );
    ~Array() ;
    ...
};
```


Example – Non-type Parameters

```
template< class T >
Array<T>::Array() {
    if (size > 0)
        ptr = new T[size];
    else
        ptr = NULL;
}
```

Example – Non-type Parameters

```
int main() {  
    Array< char > cArray( 10 );  
    Array< int > iArray( 15 );  
    Array< double > dArray( 20 );  
  
    return 0;  
}
```

Example – Non-type Parameters

```
template< class T, int SIZE >
class Array {
private:
    T ptr[SIZE];
public:
    Array() ;
    ...
};
```

...Example – Non-type Parameters

```
int main() {  
    Array< char, 10 > cArray;  
    Array< int, 15 > iArray;  
    Array< double, 20 > dArray;  
  
    return 0;  
}
```

Default Non-type Parameters

```
template< class T, int SIZE = 10 >
class Array {
private:
    T ptr[SIZE];
public:
    void doSomething();
    ...
}
```



... Default Non-type Parameters

```
int main() {  
    Array< char, 15 > cArray;  
    return 0;  
}  
// OR
```

```
int main() {  
    Array< char > cArray;  
    return 0;  
}
```

Default Type Parameters

- A type parameter can specify a default type

```
template< class T = int >  
class Vector {  
    ...  
}
```

```
Vector< > v;    // Vector< int > v;
```



Object-Oriented Programming (OOP)

Lecture No. 37



Resolution Order

```
template< typename T >  
class Vector { ... };
```

```
template< typename T >  
class Vector< T* > { ... };
```

```
template< >  
class Vector< char* > { ... };
```

Resolution Order

- Compiler searches a complete specialization whose type matches exactly with that of declaration
- If it fails then it searches for some partial specialization
- In the end it searches for some general template

...Example – Resolution Order

```
int main() {  
    Vector< char* > strVector;  
        // Vector< char* > instantiated  
  
    Vector< int* > iPtrVector;  
        // Vector< T* > instantiated  
  
    Vector< int > intVector;  
        // Vector< T > instantiated  
    return 0;  
}
```



Function Template Overloading

```
template< typename T >  
void sort( T );
```

```
template< typename T >  
void sort( Vector< T > & );
```

```
template< >  
void sort< Vector<char*> >(  
    Vector< char* > & );
```

```
void sort( char* );
```



Resolution Order

- Compiler searches target of a function call in the following order
 - Ordinary Function
 - Complete Specialization
 - Partial Specialization
 - Generic Template

Example – Resolution Order

```
int main() {  
    char* str = "Hello World!";  
    sort(str); // sort( char* )  
  
    Vector<char*> v1 = {"ab", "cd", ... };  
    sort(v1); //sort( Vector<char*> & )  
}
```

...Example – Resolution Order

```
Vector<int> v2 = { 5, 10, 15, 20 };  
sort(v2); // sort( Vector<T> &)
```

```
int iArray[] = { 5, 2, 6 , 70 };  
sort(iArray); // sort( T )
```

```
return 0;
```

```
}
```



Templates and Inheritance

- We can use inheritance comfortably with templates or their specializations
- But we must follow one rule:
 - “Derived class must take at least as many template parameters as the base class requires for an instantiation”

Derivations of a Template

- A class template may inherit from another class template

```
template< class T >  
class A  
{ ... };
```

```
template< class T >  
class B : public A< T >  
{ ... };
```

...Derivations of a Template

```
int main() {  
    A< int > obj1;  
    B< int > obj2;  
    return 0;  
}
```

...Derivations of a Template

- A partial specialization may inherit from a class template

```
template< class T >  
class B< T* > : public A< T >  
{ ... };
```

...Derivations of a Template

```
int main() {  
    A< int > obj1;  
    B< int* > obj2;  
    return 0;  
}
```

...Derivations of a Template

- Complete specialization or ordinary class cannot inherit from a class template

```
template< >  
class B< char* > : public A< T >  
{ ... }; // Error: 'T' undefined
```

```
class B : public A< T >  
{ ... }; // Error: 'T' undefined
```

Derivations of a Partial Sp.

- A class template may inherit from a partial specialization

```
template< class T >  
class A  
{ ... };
```

```
template< class T >  
class A< T* >  
{ ... };
```



...Derivations of a Partial Sp.

```
template< class T >  
class B : public A< T* >  
{ ... }
```

```
int main() {  
    A< int* > obj1;  
    B< int > obj2;  
    return 0;  
}
```

...Derivations of a Partial Sp.

- A partial specialization may inherit from a partial specialization

```
template< class T >  
class B< T* > : public A< T* >  
{ ... };
```


...Derivations of a Partial Sp.

```
int main() {  
    A< int* > obj1;  
    B< int* > obj2;  
    return 0;  
}
```

...Derivations of a Partial Sp.

- Complete specialization or ordinary class cannot inherit from a partial specialization

```
template< >  
class B< int* > : public A< T* >  
{ ... } // Error: Undefined 'T'
```

```
class B : public A< T* >  
{ ... } // Error: Undefined 'T'
```

Derivations of a Complete Sp.

- A class template may inherit from a complete specialization

```
template< class T > class A  
{ ... };
```

```
template< >  
class A< float* >  
{ ... };
```



...Derivations of a Complete Sp.

```
template< class T >  
class B : public A< float* >  
{ ... };
```

```
int main() {  
    A< float* > obj1;  
    B< int > obj2;  
    return 0;  
}
```

...Derivations of a Complete Sp.

- A partial specialization may inherit from a complete specialization

```
template< class T >  
class B< T* > : public A< float* >  
{ ... };
```

...Derivations of a Complete Sp.

```
int main() {  
    A< float* > obj1;  
    B< int* > obj2;  
    return 0;  
}
```

... Derivations of a Complete Sp.

- A complete specialization may inherit from a complete specialization

```
template< >
class B< double* > :
    public A< float* >
{ ... };
```

... Derivations of a Complete Sp.

```
int main() {  
    A< float* > obj1;  
    B< double* > obj2;  
    return 0;  
}
```


... Derivations of a Complete Sp.

- An ordinary class may inherit from a complete specialization

```
class B : public A< float* >  
{ ... };
```

... Derivations of a Complete Sp.

```
int main() {  
    A< float* > obj1;  
    B obj2;  
    return 0;  
}
```

Derivations of Ordinary Class

- A class template may inherit from an ordinary class

```
class A  
{ ... };
```

```
template< class T >  
class B : public A  
{ ... };
```

...Derivations of Ordinary Class

```
int main() {  
    A obj1;  
    B< int > obj2;  
    return 0;  
}
```

...Derivations of Ordinary Class

- A partial specialization may inherit from an ordinary class

```
template< class T >  
class B< T* > : public A  
{ ... };
```

...Derivations of Ordinary Class

```
int main() {  
    A obj1;  
    B< int* > obj2;  
    return 0;  
}
```

...Derivations of Ordinary Class

- A complete specialization may inherit from an ordinary class

```
template< >  
class B< char* > : public A  
{ ... };
```

...Derivations of Ordinary Class

```
int main() {  
    A obj1;  
    B< char* > obj2;  
    return 0;  
}
```


Object-Oriented Programming (OOP)

Lecture No. 38



Templates and Friends

- Like inheritance, templates or their specializations are compatible with friendship feature of C++

Templates and Friends – Rule 1

- When an ordinary function or class is declared as friend of a class template then it becomes friend of each instantiation of that template

...Templates and Friends – Rule 1

```
void doSomething( B< char >& );
```

```
class A { ... };
```

```
template< class T > class B {  
    int data;  
    friend void doSomething( B<char>& );  
    friend A;  
    ...  
};
```

...Templates and Friends – Rule 1

```
void doSomething( B< char >& cb ) {  
    B< int > ib;  
    ib.data = 5;    // OK  
    cb.data = 6;    // OK  
}
```

...Templates and Friends – Rule 1

```
class A {  
    void method() {  
        B< int > ib;  
        B< char > cb  
        ib.data = 5; // OK  
        cb.data = 6; // OK  
    }  
};
```

Templates and Friends – Rule 2

- When a friend function / class template is instantiated with the type parameters of class template granting friendship then its instantiation for a specific type is a friend of that class template instantiation for that particular type

...Templates and Friends – Rule 2

```
template< class U >  
void doSomething( U );  
template< class V >  
class A { ... };
```

```
template< class T > class B {  
    int data;  
    friend void doSomething( T );  
    friend A< T >;  
};
```


...Templates and Friends – Rule 2

```
template< class U >
void doSomething( U u ) {
    B< U > ib;
    ib.data = 78;
}
```

...Templates and Friends – Rule 2

```
int main() {  
    int i = 5;  
    char c = 'x';  
    doSomething( i );    // OK  
    doSomething( c );    // OK  
    return 0;  
}
```

...Templates and Friends – Rule 2

```
template< class U >
void doSomething( U u ) {
    B< int > ib;
    ib.data = 78;
}
```

...Templates and Friends – Rule 2

```
int main() {  
    int i = 5;  
    char c = 'x';  
    doSomething( i );    // OK  
    doSomething( c );    // Error!  
    return 0;  
}
```

...Templates and Friends – Rule 2

- Because `doSomething()` always instantiates `B< int >`

```
class B< int > {  
    int data;  
    friend void doSomething( int );  
    friend A< int >;  
};
```

...Templates and Friends – Rule 2

```
template< class T >
class A {
    void method() {           // Error!
        B< char > cb;
        cb.data = 8;
        B< int > ib;
        ib.data = 9;
    }
};
```

Templates and Friends – Rule 3

- When a friend function / class template takes different type parameters from the class template granting friendship then its each instantiation is a friend of each instantiation of the class template granting friendship

...Templates and Friends – Rule 3

```
template< class U >
void doSomething( U );
template< class V >
class A { ... };
template< class T > class B {
    int data;
    template< class W >
        friend void doSomething( W );
    template< class S >
        friend class A;
};
```


...Templates and Friends – Rule 3

```
template< class U >
void doSomething( U u ) {
    B< int > ib;
    ib.data = 78;
}
```

...Templates and Friends – Rule 3

```
int main() {  
    int i = 5;  
    char c = 'x';  
    doSomething( i );    // OK  
    doSomething( c );    // OK  
    return 0;  
}
```

...Templates and Friends – Rule 3

```
template< class T >
class A {
    void method() {           // OK!
        B< char > cb;
        cb.data = 8;
        B< int > ib;
        ib.data = 9;
    }
};
```

Templates and Friends – Rule 4

- Declaring a template as friend implies that all kinds of its specializations – explicit, implicit and partial, are also friends of the class granting friendship

...Templates and Friends – Rule 4

```
template< class T >
class B {
    T data;
    template< class U >
        friend class A;
};
```

...Templates and Friends – Rule 4

```
template< class U >
class A {
    A() {
        B< int > ib;
        ib.data = 10;    // OK
    }
};
```

...Templates and Friends – Rule 4

```
template< class U >
class A< U* > {
    A() {
        B< int > ib;
        ib.data = 10;    // OK
    }
};
```

...Templates and Friends – Rule 4

```
template< class T >
class B {
    T data;
    template< class U >
        friend void doSomething( U );
};
```


...Templates and Friends – Rule 4

```
template< class U >
void doSomething( U u ) {
    B< int > ib;
    ib.data = 56;    // OK
}
```

...Templates and Friends – Rule 4

```
template< >
void doSomething< char >( char u ) {
    B< int > ib;
    ib.data = 56;    // OK
}
```

Object-Oriented Programming (OOP)

Lecture No. 39

Templates & Static Members

- Each instantiation of a class template has its own copy of static members
- These are usually initialized at file scope

...Templates & Static Members

```
template< class T >
class A {
public:
    static int data;
    static void doSomething( T & );
};
```

...Templates & Static Members

```
int main() {  
    A< int > ia;  
    A< char > ca;  
    ia.data = 5;  
    ca.data = 7;  
    cout << "ia.data = " << ia.data  
        << endl  
        << "ca.data = " << ca.data;  
    return 0;  
}
```



...Templates & Static Members

- Output

```
ia.data = 5
```

```
ca.data = 7
```

Templates – Conclusion

- Templates provide
 - Reusability
 - Writability
- But can consume memory if used without care

...Templates – Conclusion

- Templates affect reliability of a program

```
template< typename T >
bool isEqual( T x, T y ) {
    return ( x == y );
}
```

...Templates – Conclusion

- One may use it erroneously

```
int main() {  
    char* str1 = "Hello ";  
    char* str2 = "World!";  
    isEqual( str1, str2 );  
    // Compiler accepts!  
}
```

Generic Algorithms Revisited

```
template< typename P, typename T >
P find( P start, P beyond,
        const T& x ) {
    while ( start != beyond &&
            *start != x )
        ++start;

    return start;
}
```

...Generic Algorithms Revisited

```
int main() {  
    int iArray[5];  
    iArray[0] = 15;  
    iArray[1] = 7;  
    iArray[2] = 987;  
    ...  
    int* found;  
    found = find(iArray, iArray + 5, 7);  
    return 0;  
}
```



...Generic Algorithms Revisited

- We claimed that this algorithm is generic
- Because it works for any aggregate object (container) that defines following three operations
 - Increment operator (++)
 - Dereferencing operator (*)
 - Inequality operator (!=)

...Generic Algorithms Revisited

- Let us implement these operations in `vector` to examine the generality of the algorithm
- Besides these operations we need a kind of pointer to track the traversal

Example – Vector

```
template< class T >
class Vector {
private:
    T* ptr;
    int size;
    int index; // initialized with zero
public:
    ...
    Vector( int = 10 );
```



...Example – Vector

```
Vector( const Vector< T >& );  
T& operator [] (int);  
  
int getIndex() const;  
void setIndex( int i );  
T& operator *();  
bool operator !=(  
    const Vector< T >& v );  
Vector< T >& operator ++();  
};
```


...Example – Vector

```
template< class T >
int Vector< T >::getIndex() const {
    return index;
}

template< class T >
void Vector< T >::setIndex( int i ) {
    if ( index >= 0 && index < size )
        index = i;
}
```



...Example – Vector

```
template< class T >
Vector<T>& Vector<T>::operator ++() {
    if ( index < size )
        ++index;
    return *this;
}

template< class T >
T& Vector< T >::operator *() {
    return ptr[index];
}
```



...Example – Vector

```
template< class T >
bool Vector<T>::operator !=(
                                Vector<T>& v ) {
    if ( size != v.size ||
        index != v.index )
        return true;
```

...Example – Vector

```
for ( int i = 0; i < size; i++ )  
    if ( ptr[i] != v.ptr[i] )  
        return true;  
  
return false;  
}
```

...Example – Vector

```
int main() {  
    Vector<int> iv( 3 );  
    iv[0] = 10;  
    iv[1] = 20;  
    iv[2] = 30;  
    Vector<int> beyond( iv ), found( 3 );  
    beyond.setIndex( iv.getSize() );  
    found = find( iv, beyond, 20 );  
    cout<<"Index: "<<found.getIndex();  
    return 0;  
}
```

Generic Algorithm

```
template< typename P, typename T >
P find( P start, P beyond,
        const T& x ) {
    while ( start != beyond &&
            *start != x )
        ++start;

    return start;
}
```

Problems

- Our generic algorithm now works fine with container `vector`
- However there are some problems with the iteration approach provided by class `Vector`

...Problems

- No support for multiple traversals
- Inconsistent behavior
 - We use pointers to mark a position in a data structure of some primitive type
 - Here we use the whole container as marker e.g. **found** in the **main** program
- Supports only a single traversal strategy

Object-Oriented Programming (OOP)

Lecture No. 40

Recap

- Generic algorithm requires three operations (`++`, `*`, `!=`)
- Implementation of these operations in `Vector` class
- Problems
 - No support for multiple traversals
 - Supports only a single traversal strategy
 - Inconsistent behavior
 - Operator `!=`

Cursors

- A better way is to use *cursors*
- A cursor is a pointer that is declared outside the container / aggregate object
- Aggregate object provides methods that help a cursor to traverse the elements
 - T* first()
 - T* beyond()
 - T* next(T*)

Vector

```
template< class T >
class Vector {
private:
    T* ptr;
    int size;
public:
    Vector( int = 10 );
    Vector( const Vector< T >& );
    ~Vector();
    int getSize() const;
```

...Vector

```
const Vector< T >& operator =( const  
                                Vector< T >& );
```

```
T& operator [] ( int );
```

```
T* first();
```

```
T* beyond();
```

```
T* next( T* );
```

```
};
```

...Vector

```
template< class T >
T* Vector< T >::first() {
    return ptr;
}
```

```
template< class T >
T* Vector< T >::beyond() {
    return ( ptr + size );
}
```

...Vector

```
template< class T >
T* Vector< T >::next( T* current )
{
    if ( current < (ptr + size) )
        return ( current + 1 );
    // else
    return current;
}
```

Example – Cursor

```
int main() {  
    Vector< int > iv( 3 );  
    iv[0] = 10;  
    iv[1] = 20;  
    iv[2] = 30;  
    int* first = iv.first();  
    int* beyond = iv.beyond();  
    int* found = find(first,beyond,20);  
    return 0;  
}
```


Generic Algorithm

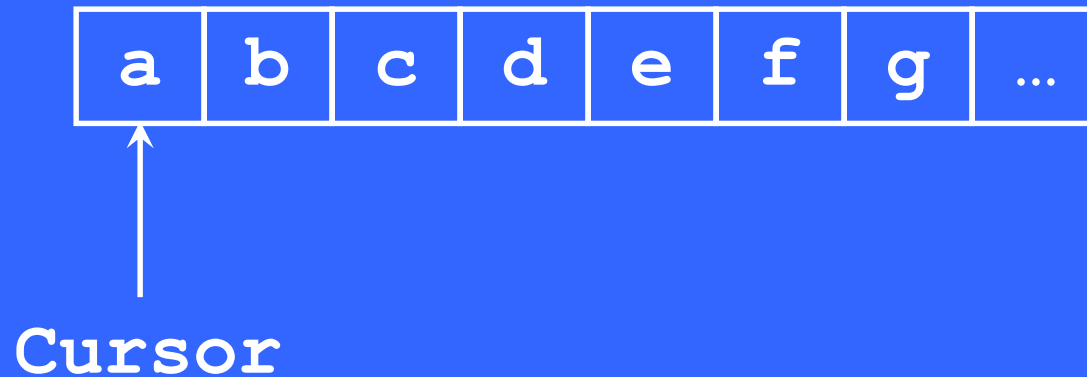
```
template< typename P, typename T >
P find( P start, P beyond,
        const T& x ) {
    while ( start != beyond &&
            *start != x )
        ++start;

    return start;
}
```

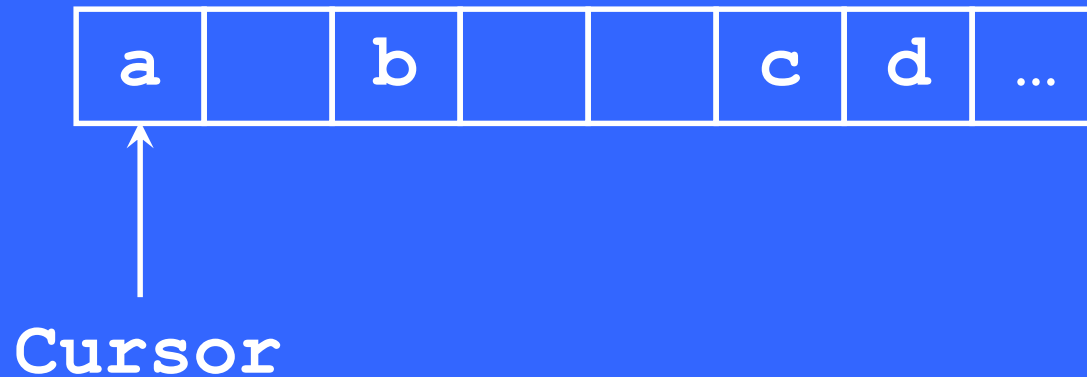
...Cursors

- This technique works fine for a contiguous sequence such as Vector
- However it does not work with containers that use complicated data structures
- There we have to rely on the container traversal operations

Example – Works Fine



Example – Problem



Example – Problem

```
int main() {  
    Set< int > is( 3 );  
    is.add( 10 );  
    is.add( 20 );  
    is.add( 30 );  
    ET* first = iv.first();  
    ET* beyond = iv.beyond();  
    ET* found = find(first, beyond, 20);  
    return 0;  
}
```

...Example – Problem

```
template< typename P, typename T >
P find( P start, P beyond,
        const T& x ) {
    while ( start != beyond &&
            *start != x )
        ++start; // Error

    return start;
}
```

Works Fine

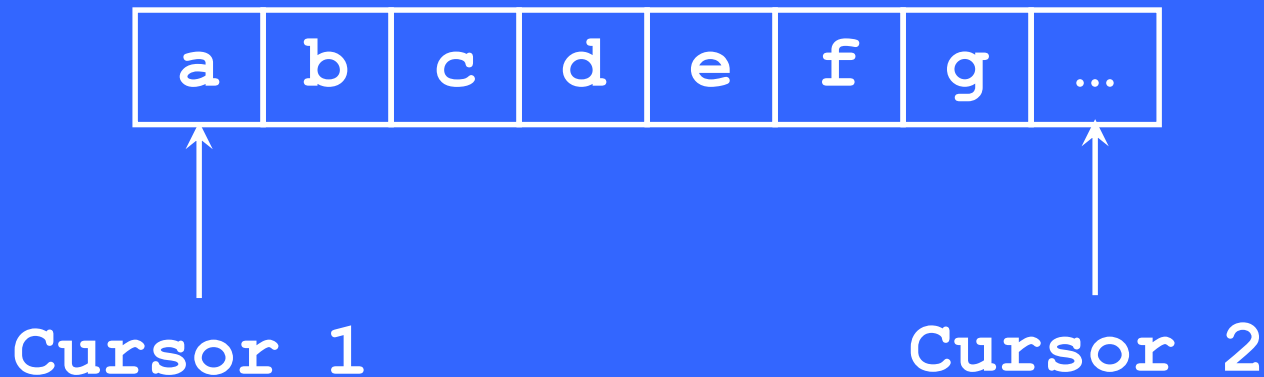
```
template< typename CT, typename ET >
P find( CT& cont, const ET& x ) {
    ET* start = cont.first();
    ET* beyond = cont.beyond();
    while ( start != beyond &&
            *start != x )
        start = cont.next( start );
    return start;
}
```

...Works Fine

```
int main() {  
    Set< int > is( 3 );  
    is.add( 10 );  
    is.add( 20 );  
    is.add( 30 );  
    int* found = find( is, 20 );  
    return 0;  
}
```


Cursors – Conclusion

- Now we can have more than one traversal pending on the aggregate object



...Cursors – Conclusion

- However we are unable to use cursors in place of pointers for all containers

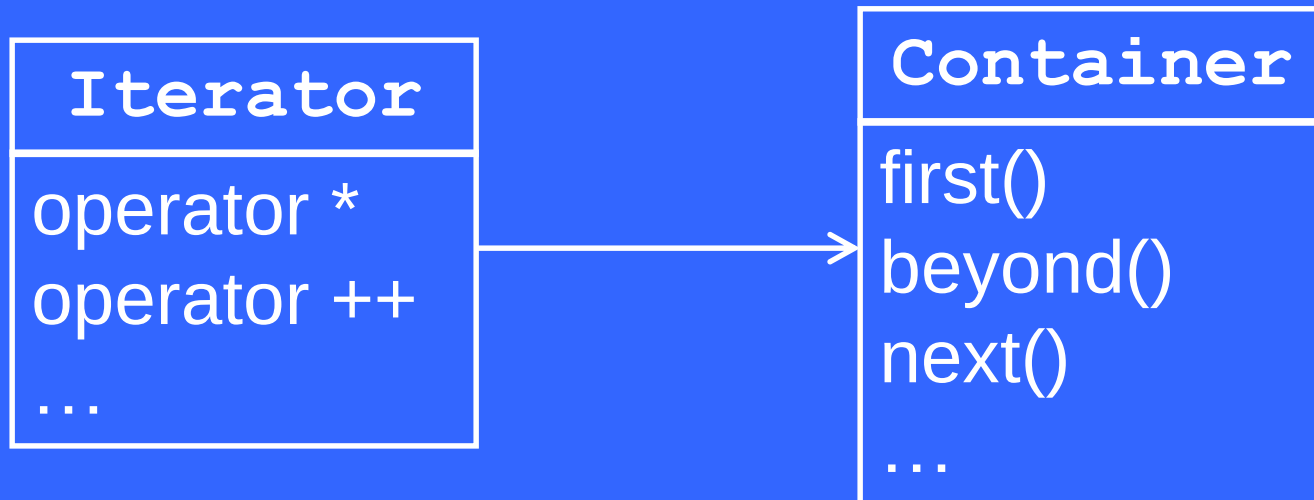
Iterators

- Iterator is an object that traverses a container without exposing its internal representation
- Iterators are for containers exactly like pointers are for ordinary data structures

Generic Iterators

- A generic iterator works with any kind of container
- To do so a generic iterator requires its container to provide three operations
 - T^* first()
 - T^* beyond()
 - T^* next(T^*)

Example – Generic Iterator



Generic Iterator

```
template< class CT, class ET >
class Iterator {
    CT* container;
    ET* index;
public:
    Iterator( CT* c,
              bool pointAtFirst = true );
    Iterator( Iterator< CT, ET >& it );
    Iterator& operator ++();
    ET& operator *();
```

...Generic Iterator

```
bool operator !=(  
    Iterator< CT, ET >& it );  
};
```

...Generic Iterator

```
template< class CT, class ET >
Iterator< CT, ET >::Iterator( CT* c,
                             bool pointAtFirst )
{
    container = c;
    if ( pointAtFirst )
        index = container->first();
    else
        index = container->beyond();
}
```


...Generic Iterator

```
template< class CT, class ET >
Iterator< CT, ET >::Iterator(
    Iterator< CT, ET >& it ) {
    container = it.container;
    index = it.index;
}
```

...Generic Iterator

```
template< class CT, class ET >
Iterator<CT,ET>& Iterator<CT,ET>::
                                operator
++() {
    index = container->next( index );
    return *this;
}
```

...Generic Iterator

```
template< class CT, class ET >  
ET& Iterator< CT, ET >::operator *()  
{  
    return *index;  
}
```

...Generic Iterator

```
template< class CT, class ET >
bool Iterator< CT, ET >::operator !=(
    Iterator< CT, ET >& it ) {
    if ( container != it.container ||
        index !=
        it.index )
        return true;
    // else
    return false;
}
```

...Generic Iterator

```
int main() {  
    Vector< int > iv( 2 );  
    Iterator< Vector<int>, int >  
        it( &iv ), beyond( &iv, false );  
    iv[0] = 10;  
    iv[1] = 20;  
    Iterator< Vector<int>, int > found  
        = find( it, beyond, 20 );  
    return 0;  
}
```

...Generic Iterator

```
template< typename P, typename T >
P find( P start, P beyond,
        const T& x ) {
    while ( start != beyond &&
            *start != x )
        ++start;

    return start;
}
```

Iterators – Conclusion

- With iterators more than one traversal can be pending on a single container
- Iterators allow to change the traversal strategy without changing the aggregate object
- They contribute towards data abstraction by emulating pointers

Object-Oriented Programming (OOP)

Lecture No. 41

Standard Template Library

- C++ programmers commonly use many data structures and algorithms
- So C++ standard committee added the STL to C++ standard library
- STL is designed to operate efficiently across many different applications

...Standard Template Library

- STL consists of three key components
 - Containers
 - Iterators
 - Algorithms

STL Promotes Reuse

- Software reuse saves development time and cost
- Once tested component can be reused several times

STL Containers

- Container is an object that contains a collection of data elements
- STL provides three kinds of containers
 - Sequence Containers
 - Associative Containers
 - Container Adapters

Sequence Containers

- A sequence organizes a finite set of objects, all of the **same type**, into a strictly **linear arrangement**

...Sequence Containers

- **vector**
 - Rapid insertions and deletions at back end
 - Random access to elements
- **deque**
 - Rapid insertions and deletions at front or back
 - Random access to elements
- **list**
 - Doubly linked list
 - Rapid insertions and deletions anywhere

Example – STL Vector

```
#include <vector>
int main() {
    std::vector< int > iv;
    int x, y;
    char ch;
    do {
        cout<<"Enter the first integer:";
        cin >> x;
        cout<<"Enter the second
                                     integer:";
        cin >> y;
```



...Example – STL Vector

```
iv.push_back( x );
iv.push_back( y );
cout << "Current capacity of iv =
      " << iv.capacity() << endl;
cout << "Current size of iv ="
      << iv.size() << endl;
cout<<"Do you want to continue?";
cin >> ch;
} while ( ch == 'y' );
}
```


Sample Output

Enter the first integer: 1

Enter the second integer: 2

Current capacity of iv = 2

Current size of iv = 2

Do you want to continue? y



...Sample Output

Enter the first integer: 3

Enter the second integer: 4

Current capacity of iv = 4

Current size of iv = 4

Do you want to continue? y



...Sample Output

Enter the first integer: 5

Enter the second integer: 6

Current capacity of iv = 8

Current size of iv = 6

Do you want to continue? n



Example – STL Deque

```
#include <deque>

int main() {
    std::deque< int > dq;
    dq.push_front( 3 );
    dq.push_back( 5 );

    dq.pop_front();
    dq.pop_back();
    return 0;
}
```



Example – STL List

```
#include <list>

int main() {
    std::list< float > _list;
    _list.push_back( 7.8 );
    _list.push_back( 8.9 );
    std::list< float >::iterator it
                                = _list.begin();
    _list.insert( ++it, 5.3 );
    return 0;
}
```



Associative Containers

- An associative container provide fast retrieval of data based on keys

...Associative Containers

- **set**
 - No duplicates
- **multiset**
 - Duplicates allowed
- **map**
 - No duplicate keys
- **multimap**
 - Duplicate keys allowed

Example – STL Set

```
#include <set>

int main() {
    std::set< char > cs;
    cout << "Size before insertions: "
          << cs.size() << endl;
    cs.insert( 'a' );
    cs.insert( 'b' );
    cs.insert( 'b' );
    cout << "Size after insertions: "
          << cs.size();
    return 0;
}
```


Output

Size before insertions: 0

Size after insertions: 2

Example – STL Multi-Set

```
#include <set>

int main() {
    std::multiset< char > cms;
    cout << "Size before insertions: "
          << cms.size() << endl;
    cms.insert( 'a' );
    cms.insert( 'b' );
    cms.insert( 'b' );
    cout << "Size after insertions: "
          << cms.size();
    return 0;
}
```

Output

Size before insertions: 0

Size after insertions: 3

Example – STL Map

```
#include <map>
int main() {
    typedef std::map< int, char > MyMap;
    MyMap m;
    m.insert(MyMap::value_type(1, 'a'));
    m.insert(MyMap::value_type(2, 'b'));
    m.insert(MyMap::value_type(3, 'c'));
    MyMap::iterator it = m.find( 2 );
    cout << "Value @ key " << it->first
          << " is " << it->second;
    return 0;
}
```



Output

Value @ key 2 is b

Example – STL Multi-Map

```
#include <map>
```

```
int main() {
```

```
    typedef std::multimap< int, char >
```

```
        MyMap;
```

```
    MyMap m;
```

```
    m.insert(MyMap::value_type(1, 'a'));
```

```
    m.insert(MyMap::value_type(2, 'b'));
```

```
    m.insert(MyMap::value_type(3, 'b'));
```



...Example – STL Multi-Map

```
MyMap::iterator it1 = m.find( 2 );  
MyMap::iterator it2 = m.find( 3 );  
cout << "Value @ key " << it1->first  
      << " is " << it1->second << endl;  
cout << "Value @ key " << it2->first  
      << " is " << it2->second << endl;  
return 0;  
}
```

Output

Value @ key 2 is b

Value @ key 3 is b

First-class Containers

- Sequence and associative containers are collectively referred to as the first-class containers

Container Adapters

- A container adapter is a constrained version of some first-class container

...Container Adapters

- **stack**
 - Last in first out (LIFO)
 - Can adapt **vector**, **deque** or **list**
- **queue**
 - First in first out (FIFO)
 - Can adapt **deque** or **list**
- **priority_queue**
 - Always returns element with highest priority
 - Can adapt **vector** or **deque**

Common Functions for All Containers

- Default constructor
- Copy Constructor
- Destructor
- **empty()**
 - Returns true if container contains no elements
- **max_size()**
 - Returns the maximum number of elements

...Common Functions for All Containers

- **size()**
 - Return current number of elements
- **operator = ()**
 - Assigns one container instance to another
- **operator < ()**
 - Returns true if the first container is less than the second container

...Common Functions for All Containers

- **operator <= ()**
 - Returns true if the first container is less than or equal to the second container
- **operator > ()**
 - Returns true if the first container is greater than the second container
- **operator >= ()**
 - Returns true if the first container is greater than or equal to the second container

...Common Functions for All Containers

- **operator == ()**
 - Returns true if the first container is equal to the second container
- **operator != ()**
 - Returns true if the first container is not equal to the second container
- **swap ()**
 - swaps the elements of the two containers

Functions for First-class Containers

- `begin()`
 - Returns an `iterator` object that refers to the first element of the container
- `end()`
 - Returns an `iterator` object that refers to the next position beyond the last element of the container

...Functions for First-class Containers

- **rbegin()**
 - Returns an **iterator** object that refers to the last element of the container
- **rend()**
 - Returns an **iterator** object that refers to the position before the first element

...Functions for First-class Containers

- `erase( iterator )`
 - Removes an element pointed to by the `iterator`
- `erase( iterator, iterator )`
 - Removes the range of elements specified by the first and the second `iterator` parameters

...Functions for First-class Containers

- `clear()`
 - erases all elements from the container

Container Requirements

- Each container requires element type to provide a minimum set of functionality e.g.
- When an element is inserted into a container, a copy of that element is made
 - Copy Constructor
 - Assignment Operator

...Container Requirements

- Associative containers and many algorithms compare elements
 - Operator `==`
 - Operator `<`

Object-Oriented Programming (OOP)

Lecture No. 42

Iterators

- Iterators are types defined by STL
- Iterators are for containers like pointers are for ordinary data structures
- STL iterators provide pointer operations such as `*` and `++`

Iterator Categories

- Input Iterators
- Output Iterators
- Forward Iterators
- Bidirectional Iterators
- Random-access Iterators

Input Iterators

- Can only read an element
- Can only move in forward direction one element at a time
- Support only one-pass algorithms

Output Iterators

- Can only write an element
- Can only move in forward direction one element at a time
- Support only one-pass algorithms

Forward Iterators

- Combine the capabilities of both input and output iterators
- In addition they can bookmark a position in the container

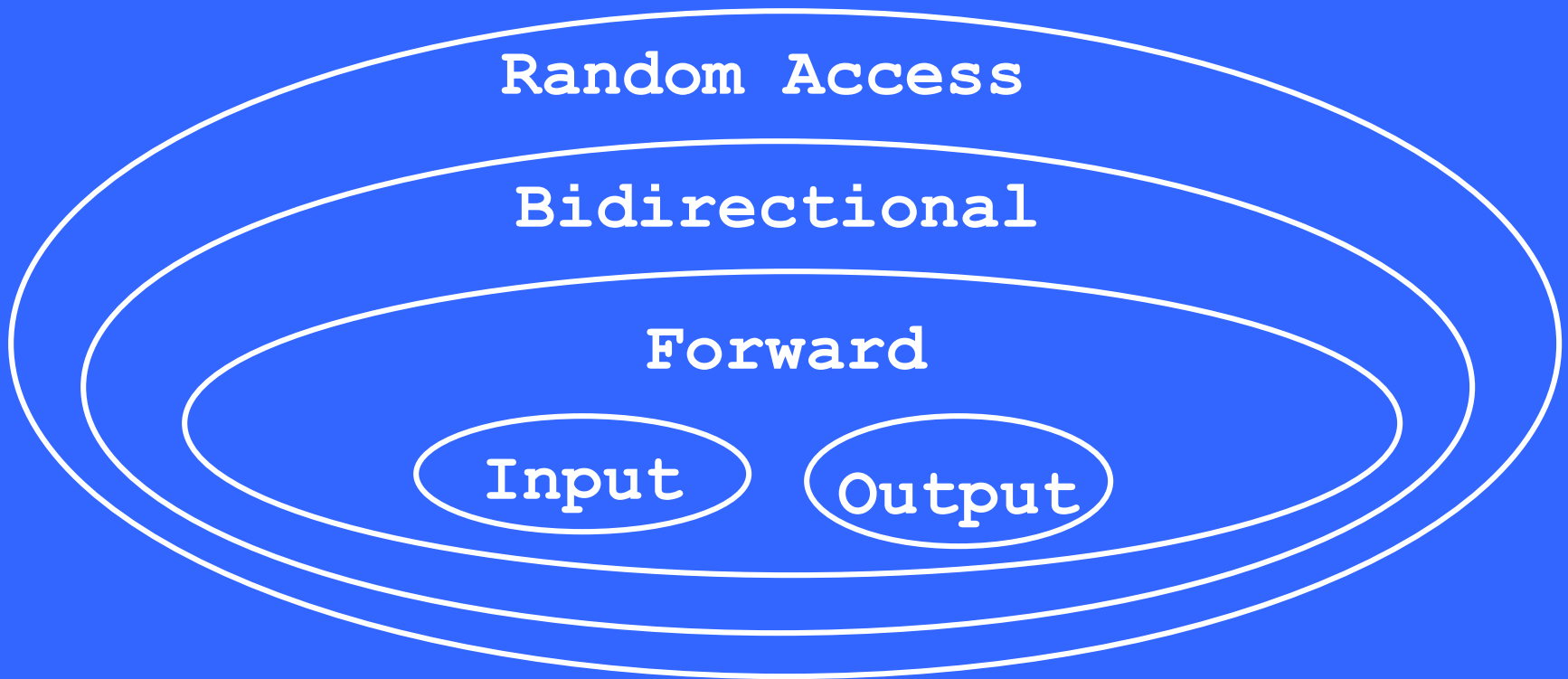
Bidirectional Iterators

- Provide all the capabilities of forward iterators
- In addition, they can move in backward direction
- As a result they support multi-pass algorithms

Random Access Iterators

- Provide all the capabilities of bidirectional iterators
- In addition they can directly access any element of a container

Iterator Summary



Container and Iterator Types

- Sequence Containers
 - vector -- random access
 - deque -- random access
 - list -- bidirectional
- Associative Containers
 - set -- bidirectional
 - multiset -- bidirectional
 - map -- bidirectional
 - multimap -- bidirectional

...Container and Iterator Types

- Container Adapters

-- stack	-- (none)
-- queue	-- (none)
-- priority_queue	-- (none)

Iterator Operations

All Iterators

- `++p`
 - pre-increment an iterator
- `p++`
 - post-increment an iterator

Input Iterators

- `*p`
 - Dereference operator (used as rvalue)
- `p1 = p2`
 - Assignment
- `p1 == p2`
 - Equality operator
- `p1 != p2`
 - Inequality operator
- `p->`
 - Access Operator

Output Iterators

- $*p$
 - Dereference operator (can be used as lvalue)
- $p1 = p2$
 - Assignment

Forward Iterators

- Combine the operations of both input and output iterators

Bidirectional Iterators

- Besides the operations of forward iterators they also support
- $--p$
 - Pre-increment operator
- $p--$
 - post-decrement operator

Random-access Iterators

- Besides the operations of bidirectional iterators, they also support
 - $p + i$
 - Result is an iterator pointing at $p + i$
 - $p - i$
 - Result is an iterator pointing at $p - i$

...Random-access Iterators

- $p += i$
 - Increment iterator p by i positions
- $p -= i$
 - Decrement iterator p by i positions
- $p[i]$
 - Returns a reference of element at $p + i$
- $p1 < p2$
 - Returns true if $p1$ is before $p2$ in the container

...Random-access Iterators

- $p1 \leq p2$
 - Returns true if $p1$ is before $p2$ in the container or $p1$ is equal to $p2$
- $p1 > p2$
 - Returns true if $p1$ is after $p2$ in the container
- $p1 \geq p2$
 - Returns true if $p1$ is after $p2$ in the container or $p1$ is equal to $p2$

Example – Random Access Iterator

```
typedef std::vector< int > IntVector;  
int main() {  
    const int SIZE = 3;  
    int iArray[ SIZE ] = { 1, 2, 3 };  
    IntVector iv(iArray, iArray + SIZE);  
    IntVector::iterator it = iv.begin();  
    cout << "Vector contents: ";  
    for ( int i = 0; i < SIZE; ++i )  
        cout << it[i] << ", ";  
    return 0;  
}
```



...Sample Output

Vector contents: 1, 2, 3,

Example – Bidirectional Iterator

```
typedef std::set< int > IntSet;
int main() {
    const int SIZE = 3;
    int iArray[ SIZE ] = { 1, 2, 3 };
    IntSet is( iArray, iArray + SIZE );
    IntSet::iterator it = is.begin();
    cout << "Set contents: ";
    for (int i = 0; i < SIZE; ++i)
        cout << it[i] << ", "; // Error
    return 0;
}
```



...Example – Bidirectional Iterator

```
typedef std::set< int > IntSet;
int main() {
    const int SIZE = 3;
    int iArray[ SIZE ] = { 1, 2, 3 };
    IntSet is( iArray, iArray + SIZE );
    IntSet::iterator it = is.begin();
    cout << "Set contents: ";
    for ( int i = 0; i < SIZE; ++i )
        cout << *it++ << ", ";    // OK
    return 0;
}
```

...Sample Output

Set contents: 1, 2, 3,

...Example – Bidirectional Iterator

```
typedef std::set< int > IntSet;
int main() {
    const int SIZE = 3;
    int iArray[ SIZE ] = { 1, 2, 3 };
    IntSet is( iArray, iArray + SIZE );
    IntSet::iterator it = is.end();
    cout << "Set contents: ";
    for (int i = 0; i < SIZE; ++i)
        cout << *--it << ", ";
    return 0;
}
```

...Sample Output

Set contents: 3, 2, 1,

Example – Input Iterator

```
#include <iostream>
using std::cin;
using std::cout;
using std::endl;
#include <iterator>

int main() {
    int x, y, z;
    cout << "Enter three integers:\n";
```



...Example – Input Iterator

```
std::istream_iterator< int >  
    inputIt( cin );  
  
x = *inputIt++;  
y = *inputIt++;  
z = *inputIt;  
  
cout << "x = " << x << endl;  
cout << "y = " << y << endl;  
cout << "z = " << z << endl;  
return 0;  
}
```

...Example – Input Iterator

```
int main() {  
    int x = 5;  
    std::istream_iterator< int >  
                                inputIt( cin );  
    *inputIt = x;    // Error  
    return 0;  
}
```

Example – Output Iterator

```
int main() {  
    int x = 1, y = 2, z = 3;  
    std::ostream_iterator< int >  
        outputIt( cout, ", " );  
  
    *outputIt++ = x;  
    *outputIt++ = y;  
    *outputIt++ = z;  
    return 0;  
}
```

...Example – Output Iterator

```
int main() {  
    int x = 1, y = 2, z = 3;  
    std::ostream_iterator< int >  
        outputIt( cout, ", " );  
    x = *outputIt++;    // Error  
    return 0;  
}
```

Algorithms

- STL includes 70 standard algorithms
- These algorithms may use iterators to manipulate containers
- STL algorithms also work for ordinary pointers and data structures

...Algorithms

- An algorithm works with a particular container only if that container supports a particular iterator category
- A multi-pass algorithm for example, requires bidirectional iterator(s) at least

Examples

Mutating-Sequence Algorithms

copy
copy_backward
fill
fill_n
generate
generate_n
iter_swap
partition
...

Non-Mutating-Sequence Algorithms

`adjacent_find`

`count`

`count_if`

`equal`

`find`

`find_each`

`find_end`

`find_first_of`

`...`



Numeric Algorithms

`accumulate`

`inner_product`

`partial_sum`

`adjacent_difference`

Example – copy Algorithm

```
#include <iostream>
using std::cout;
#include <vector>
#include <algorithm>
typedef std::vector< int > IntVector;

int main() {
    int iArray[] = {1, 2, 3, 4, 5, 6};
    IntVector iv( iArray, iArray + 6 );
```



...Example – copy Algorithm

```
std::ostream_iterator< int >  
    output( cout, ", " );  
std::copy( begin, end, output );  
  
return 0;  
}
```

Output

1, 2, 3, 4, 5, 6,

Example – fill Algorithm

```
#include <iostream>
using std::cout;
using std::endl;
#include <vector>
#include <algorithm>
typedef std::vector< int > IntVector;
int main() {
    int iArray[] = { 1, 2, 3, 4, 5 };
    IntVector iv( iArray, iArray + 5 );
```



...Example – fill Algorithm

```
std::ostream_iterator< int >  
    output( cout, " , " );  
std::copy( iv.begin(), iv.end(),  
           output  
);  
std::fill(iv.begin(), iv.end(), 0);  
cout << endl;  
std::copy( iv.begin(), iv.end(),  
           output  
);  
return 0;
```



Object-Oriented Programming (OOP)

Lecture No. 43

Techniques for Error Handling

- Abnormal termination
- Graceful termination
- Return the illegal value
- Return error code from a function
- Exception handling

Example – Abnormal Termination

```
void GetNumbers( int &a, int &b ) {  
    cout << "\nEnter two integers";  
    cin >> a >> b;  
}  
int Quotient( int a, int b ){  
    return a / b;  
}  
void OutputQuotient( int a, int b, int quo ) {  
    cout << "Quotient of " << a << " and "  
        << b << " is " << quo << endl;  
}
```

Example – Abnormal Termination

```
int main(){
    int sum = 0, quot;
    int a, b;
    for (int i = 0; i < 10; i++){
        GetNumbers(a,b);
        quot = Quotient(a,b);
        sum += quot;
        OutputQuotient(a,b,quot);
    }
    cout << "\nSum of ten quotients is " << sum;
    return 0;
}
```

Output

Enter two integers

10

10

Quotient of 10 and 10 is 1

Enter two integers

10

0

Program terminated abnormally

Graceful Termination

- Program can be designed in such a way that instead of abnormal termination, that causes the wastage of resources, program performs clean up tasks

Example – Graceful Termination

```
int Quotient (int a, int b ) {  
    if(b == 0){  
        cout << "Denominator can't "  
              << " be zero" << endl;  
        // Do local clean up  
        exit(1);  
    }  
    return a / b;  
}
```

Output

Enter two integers

10

10

Quotient of 10 and 10 is 1

Enter two integers

10

0

Denominator can't be zero

Error Handling

- The clean-up tasks are of local nature only
- There remains the possibility of information loss

Example – Return Illegal Value

```
int Quotient(int a, int b){
    if(b == 0)
        b = 1;
    OutputQuotient(a, b, a/b);
    return a / b ;
}

int main() {
    int a,b,quot;      GetNumbers(a,b);
    quot = Quotient(a,b);
    return 0;
}
```

Output

Enter two integers

10

0

Quotient of 10 and 1 is 10

Error Handling

- Programmer has avoided the system crash but the program is now in an inconsistent state

Example – Return Error Code

```
bool Quotient ( int a, int b, int & retVal ) {  
    if(b == 0){  
        return false;  
    }  
    retVal = a / b;  
    return true;  
}
```

Part of main Function

```
for(int i = 0; i < 10; i++){  
    GetNumbers(a,b);  
    while ( ! Quotient(a, b, quot) ) {  
        cout << "Denominator can't be "  
        << "Zero. Give input again \n";  
        GetNumbers(a,b);  
    }  
    sum += quot;  
    OutputQuotient(a, b, quot);  
}
```

Output

Enter two integers

10

0

Denominator can't be zero. Give input again.

Enter two integers

10

10

Quotient of 10 and 10 is 1

...//there will be exactly ten quotients

Error Handling

- Programmer sometimes has to change the design to incorporate error handling
- Programmer has to check the return type of the function to know whether an error has occurred

Error Handling

- Programmer of calling function can ignore the return value
- The result of the function might contain illegal value, this may cause a system crash later

Program's Complexity Increases

- The error handling code increases the complexity of the code
 - Error handling code is mixed with program logic
 - The code becomes less readable
 - Difficult to modify

Example

```
int main() {  
    function1();  
    function2();  
    function3();  
  
    return 0;  
}
```

Example

```
int main(){
    if( function1() ) {
        if( function2() ) {
            if( function3() ) {
                ...
            }
            else    cout << "Error Z has occurred";
        }
        else    cout << "Error Y has occurred";
    }
    else    cout << "Error X has occurred";
    return 0;
}
```

Exception Handling

- Exception handling is a much elegant solution as compared to other error handling mechanisms
- It enables separation of main logic and error handling code

Exception Handling Process

- Programmer writes the code that is suspected to cause an exception in **try block**
- Code section that encounters an error **throws** an object that is used to represent exception
- **Catch blocks** follow try block to catch the object thrown

Syntax - Throw

- The keyword **throw** is used to throw an exception
- Any expression can be used to represent the exception that has occurred

throw X;

throw (X);

Examples

```
int a;
```

```
Exception obj;
```

```
throw 1;           // literal
```

```
throw (a);         // variable
```

```
throw obj;         // object
```

```
throw Exception();  
                // anonymous object
```

```
throw 1+2*9;  
                // mathematical expression
```


Throw

- Primitive data types may be avoided as throw expression, as they can cause ambiguity
- Define new classes to represent the exceptions that has occurred
 - This way there are less chances of ambiguity

Syntax – Try and Catch

```
int main () {  
    try {  
        ...  
    }  
    catch ( Exception1 ) {  
        ...  
    }  
    catch ( Exception2 obj ) {  
        ...  
    }  
    return 0;  
}
```

Catch Blocks

- Catch handler must be preceded by a try block or an other catch handler
- Catch handlers are only executed when an exception has occurred
- Catch handlers are differentiated on the basis of argument type

Catch Handler

- The catch blocks are tried in order they are written
- They can be seen as switch statement that do not need break keyword

Example

```
class DivideByZero {  
public:  
    DivideByZero() {  
    }  
};  
int Quotient(int a, int b){  
    if(b == 0){  
        throw DivideByZero();  
    }  
    return a / b;  
}
```

Body of main Function

```
for(int i = 0; i < 10; i++) {  
    try{  
        GetNumbers(a,b);  
        quot = Quotient(a,b);  
        OutputQuotient(a,b,quot); sum += quot;  
    }  
    catch(DivideByZero) {  
        i--;  
        cout << "\nAttempt to divide  
            numerator with zero";  
    }  
}
```

Output

Enter two integers

10

10

Quotient of 10 and 10 is 1

Enter two integers

10

0

Attempt to divide numerator with zero

...

// there will be sum of exactly ten quotients

Catch Handler

- The catch handler catches the DivideByZero object through anonymous object
- Program logic and error handling code are separated
- We can modify this to use the object to carry information about the cause of error

Separation of Program Logic and Error Handling

```
int main() {  
    try {  
        function1();  
        function2();  
        function3();  
    }  
    catch( ErrorX) { ... }  
    catch( ErrorY) { ... }  
    catch( ErrorZ) { ... }  
    return 0;  
}
```

Object-Oriented Programming (OOP)

Lecture No. 44

Example

```
class DivideByZero {  
public:  
    DivideByZero() {  
    }  
};  
int Quotient(int a, int b){  
    if(b == 0){  
        throw DivideByZero();  
    }  
    return a / b;  
}
```

main Function

```
int main() {  
    try{ ...  
        quot = Quotient(a,b);  
        ...  
    }  
    catch(DivideByZero) {  
        ...  
    }  
    return 0;  
}
```

Stack Unwinding

- The flow control of throw is referred to as stack unwinding
- Stack unwinding is more complex than return statement
- Return can be used to transfer the control to the calling function only
- Stack unwinding can transfer the control to any function in nested function calls

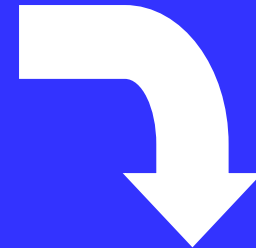
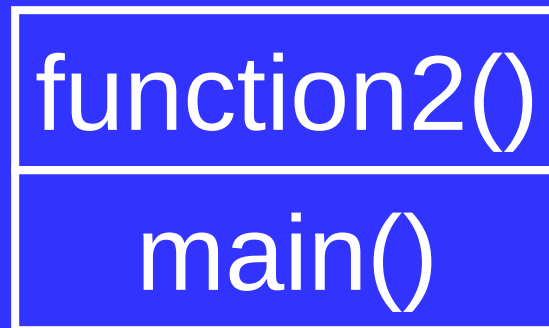
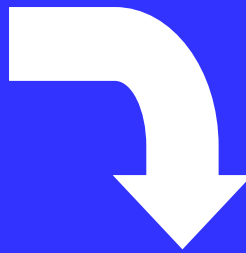
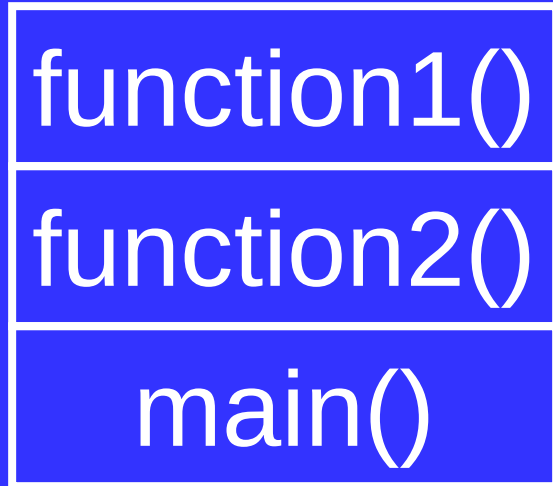
Stack Unwinding

- All the local objects of a executing block are destroyed when an exception is thrown
- Dynamically allocated memory is not destroyed automatically
- If no catch handler catches the exception the function terminate is called, which by default calls function abort

Example

```
void function1() { ...  
    throw Exception(); ...  
}  
void function2() {...  
    function1();...  
}  
int main() {  
    try{  
        function2();  
    } catch( Exception ) { }  
    return 0;  
}
```

Function-Call Stack



Stack Unwinding

- The stack unwinding is also performed if we have nested try blocks

Example

```
int main() {  
    try {  
        try {  
            throw 1;  
        }  
        catch( float ) { }  
    }  
    catch( int ) { }  
    return 0;  
}
```

Stack Unwinding

- Firstly the catch handler with float parameter is tried
- This catch handler will not be executed as its parameter is of different type – no coercion
- Secondly the catch handler with int parameter is tried and executed

Catch Handler

- We can modify this to use the object to carry information about the cause of error
- The object thrown is copied to the object given in the handler
- Use the reference in the catch handler to avoid problem caused by shallow copy

Example

```
class DivideByZero {  
    int numerator;  
public:  
    DivideByZero(int i) {  
        numerator = i;  
    }  
    void Print() const{  
        cout << endl << numerator  
        << " was divided by zero";  
    }  
};
```

Example

```
int Quotient(int a, int b) {  
    if(b == 0){  
        throw DivideByZero(a);  
    }  
    return a / b;  
}
```

Body of main Function

```
for ( int i = 0; i < 10; i++ ) {  
    try {  
        GetNumbers(a, b);  
        quot = Quotient(a, b);    ...  
    } catch(DivideByZero & obj) {  
        obj.Print();  
        i--;  
    }  
}  
  
cout << "\nSum of ten quotients is "  
      << sum;
```

Output

Enter two integers

10

10

Quotient of 10 and 10 is 1

Enter two integers

10

0

10 was divided by zero

...

// there will be sum of exactly ten quotients

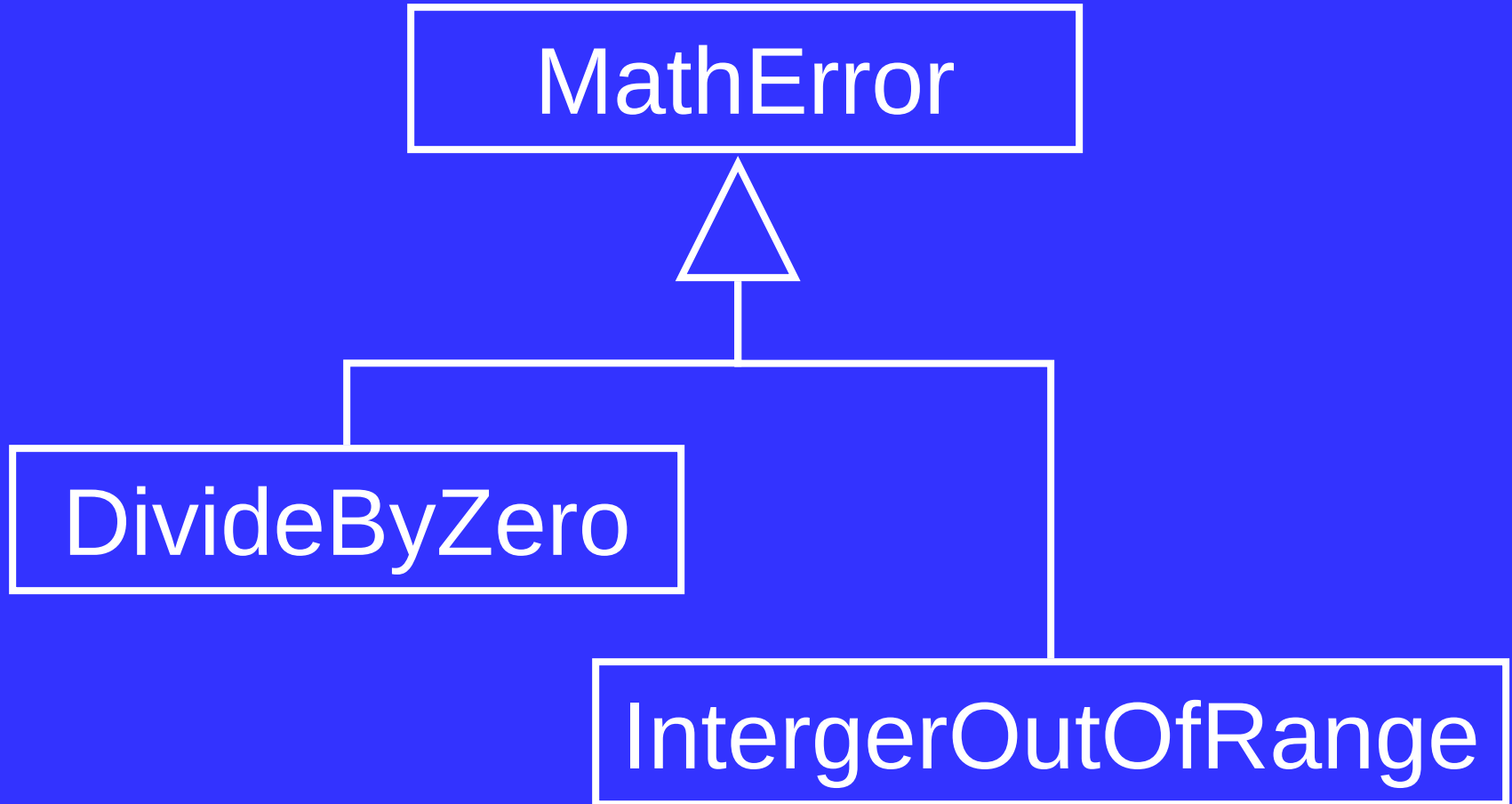
Catch Handler

- The object thrown as exception is destroyed when the execution of the catch handler completes

Avoiding too many Catch Handlers

- There are two ways to catch more than one object in a single catch handler
 - Use inheritance
 - Catch every exception

Inheritance of Exceptions



Grouping Exceptions

```
try{  
    ...  
}  
catch(DivideByZero){  
    ...  
}  
catch(IntergerOutOfRange){  
    ...  
}  
catch (InputStreamError){  
}
```

Example—With Inheritance

```
try{
```

```
    ...
```

```
}
```

```
catch (MathError){
```

```
}
```

```
catch (InputStreamError){
```

```
}
```

Catch Every Exception

- C++ provides a special syntax that allows to catch every object thrown

```
catch ( ... )  
{  
    //...  
}
```

Re-Throw

- A function can catch an exception and perform partial handling
- Re-throw is a mechanism of throw the exception again after partial handling

throw; /*without any expression*/

Example

```
int main () {  
    try {  
        Function();  
    }  
    catch(Exception&) {  
        ...  
    }  
    return 0;  
}
```


Example

```
void Function() {  
    try {  
        /*Code that might throw  
        an Exception*/  
    } catch(Exception&){  
        if( can_handle_completely ) {  
            // handle exception  
        } else {  
            // partially handle exception  
            throw; //re-throw exception  
        }  
    }  
}
```

Order of Handlers

- Order of the more than one catch handlers can cause logical errors when using inheritance or catch all

Example

```
try{  
    ...  
}  
catch (...) { ...  
}  
catch ( MathError ) { ...  
}  
catch ( DivideByZero ) { ...  
}  
// last two handler can never be invoked
```

Object-Oriented Programming (OOP)

Lecture No. 45

Resource Management

- Function acquiring a resource must properly release it
- Throwing an exception can cause resource wastage

Example

```
int function1(){  
    FILE *fileptr =  
        fopen("filename.txt","w");  
  
    ...  
    throw exception();  
  
    ...  
    fclose(fileptr);  
    return 0;  
}
```

Resource Management

- In case of exception the call to close will be ignored

First Attempt

```
int function1(){
    try{
        FILE *fileptr = fopen("filename.txt","w");
        fwrite("Hello World",1,11,fileptr);
        ...
        throw exception();
        fclose(fileptr);
    } catch(...) {
        fclose(fileptr);
        throw;
    }
    return 0;
}
```


Resource Management

- There is code duplication

Second Attempt

```
class FilePtr{  
    FILE * f;  
public:  
    FilePtr(const char *name,  
            const char * mode)  
        { f = fopen(name, mode);}  
    ~FilePtr()          { fclose(f);    }  
    operator FILE * () { return f;      }  
};
```

Example

```
int function1(){  
    FilePtr file("filename.txt","w");  
    fwrite("Hello World",1,11,file);  
    throw exception();  
    ...  
    return 0;  
}
```

Resource Management

- The destructor of the FilePtr class will close the file
- Programmer does not have to close the file explicitly in case of error as well as in normal case

Exception in Constructors

- Exception thrown in constructor cause the destructor to be called for any object built as part of object being constructed before exception is thrown
- Destructor for partially constructed object is not called

Example

```
class Student{  
    String FirstName;  
    String SecondName;  
    String EmailAddress;  
    ...  
}
```

- If the constructor of the SecondName throws an exception then the destructor for the First Name will be called

Exception in Initialization List

- Exception due to constructor of any contained object or the constructor of a parent class can be caught in the member initialization list

Example

```
Student::Student (String aName) :  
    name(aName)
```

```
/*The constructor of String can  
   throw a exception*/
```

```
{
```

```
    ...
```

```
}
```


Exception in Initialization List

- The programmer may want to catch the exception and perform some action to rectify the problem

Example

```
Student::Student (String aName)
```

```
try
```

```
    : name(aName) {
```

```
        ...
```

```
    }
```

```
catch(...) {
```

```
}
```

Exceptions in Destructors

- Exception should not leave the destructor
- If a destructor is called due to stack unwinding, and an exception leaves the destructor then the function `std::terminate()` is called, which by default calls the `std::abort()`

Example

```
class Exception;  
class Complex{  
    ...  
public:  
    ~Complex(){  
        throw Exception();  
    }  
};
```

Example

```
int main(){  
    try{  
        Complex obj;  
        throw Exception();  
        ...  
    }  
    catch(...){  
    }  
    return 0;  
}  
// The program will terminate abnormally
```

Example

```
Complex::~~Complex()
{
    try{
        throw Exception();
    }
    catch(...){
    }
}
```

Exception Specification

- Program can specify the list of exceptions a function is allowed to throw
- This list is also called throw list
- If we write empty list then the function wont be able to throw any exception

Syntax

```
void Function1() {...}
```

```
void Function2() throw () {...}
```

```
void Function3()  
    throw (Exception1, ...) {}
```

- Function1 can throw any exception
- Function2 cannot throw any Exception
- Function3 can throw any exception of type Exception1 or any class derived from it

Exception Specification

- If a function throws exception other than specified in the throw list then the function **unexpected** is called
- The function **unexpected** calls the function **terminate**

Exception Specification

- If programmer wants to handle such cases then he must provide a handler function
- To tell the compiler to call a function use `set_unexpected`

Course Review

Object Orientation

- What is an object
- Object-Oriented Model
 - Information Hiding
 - Encapsulation
 - Abstraction
- Classes

Object Orientation

- Inheritance
 - Generalization
 - Sub-Typing
 - Specialization
- “IS-A” relationship
- Abstract classes
- Concrete classes

Object Orientation

- Multiple inheritance
- Types of association
 - Simple association
 - Composition
 - Aggregation
- Polymorphism

Classes – C++ Constructs

- Classes
 - Data members
 - Member functions
- Access specifier
- Constructors
- Copy Constructors
- Destructors

Classes – C++ Constructs

- this pointer
- Constant objects
- Static data member
- Static member function
- Dynamic allocation

Classes – C++ Constructs

- Friend classes
- Friend functions
- Operator overloading
 - Binary operator
 - Unary operator
 - operator []
 - Type conversion

Inheritance – C++ Constructs

- Public inheritance
- Private inheritance
- Protected inheritance

- Overriding
- Class hierarchy

Polymorphism – C++ Constructs

- Static type vs. dynamic type
 - Virtual function
 - Virtual destructor
 - V-tables
-
- Multiple inheritance
 - Virtual inheritance

Templates – C++ Constructs

- Generic programming
- Classes template
- Function templates
- Generic algorithm
- Templates specialization
 - Partial Specialization
 - Complete specialization

Templates – C++ Constructs

- Inheritance and templates
- Friends and templates
- STL
 - Containers
 - Iterators
 - Algorithms

Writing Reliable Programs

- Error handling techniques
 - Abnormal termination
 - Graceful termination
 - Return the illegal value
 - Return error code from a function
 - Exception handling